

· Enterprise Linux Patching Runbook

Premium Production-Grade Document | Version 3.0

Document Information Details ----- -----	Document Title Enterprise Linux Patching Runbook
Version 3.0	Author Krishna Yada Senior Linux Infrastructure Team
Classification INTERNAL · CONFIDENTIAL	Review Cycle Quarterly
Last Updated June 2026	Applicable OS RHEL 6, 7, 8, 9, 10 / CentOS 7, 8 Stream, 9 Stream / Oracle Linux / Rocky Linux / AlmaLinux
Environment 2000+ Linux Servers (DEV/UAT/PROD)	Approved By VP Infrastructure / CISO / Change Advisory Board

·**DISCLAIMER:** This runbook contains production procedures. ALL patches MUST be validated in DEV and UAT environments before applying to PRODUCTION. Never apply untested patches directly to production systems. The authors assume no liability for improper use of procedures described herein. Always follow your organization's Change Management process.

·**BEGINNER NOTE:** If you are new to Linux patching, start by reading Section 1 (Overview) completely, then follow the Pre-Patching Checklist (Section 3) step by step. Never skip any validation step.

· Table of Contents

- Section 1 · Patching Overview
 - Section 2 · Patching Strategy
 - Section 3 · Pre-Patching Checklist
 - Section 4 · Patching Using Red Hat Satellite
 - Section 5 · Patching Using SCCM/MECM
 - Section 6 · Patching Using Ansible
 - Section 7 · Patching Using Chef
 - Section 8 · Production Patching Procedure
 - Section 9 · Zero Downtime Patching
 - Section 10 · Reboot Management
 - Section 11 · Post Patch Validation
 - Section 12 · Troubleshooting Guide
 - Section 13 · Rollback Procedures
 - Section 14 · Change Management
 - Section 15 · Appendix
-

Revision History

Version Date Author Changes ----- ----- ----- -----	1.0 Jan 2024 Linux Team Initial Document
2.0 Jul 2025 Linux Team Added RHEL 9/10, Ansible automation	3.0 Jun 2026 Linux Team Complete rewrite, 2000+ server procedures

Section 1 · Patching Overview

1.1 What is Linux Patching?

Linux patching is the process of applying software updates (patches) to the Linux operating system and its components to fix security vulnerabilities, resolve bugs, improve performance, and add new features. In an enterprise environment with 2000+ servers, patching is a critical operational activity that requires careful planning, execution, and validation.

Why is Patching Required?

| Reason | Description | Business Impact | |-----|-----|-----| | **Security** | Fixes known vulnerabilities (CVEs) | Prevents data breaches, ransomware attacks | | **Compliance** | Meets regulatory requirements (PCI-DSS, HIPAA, SOX, ISO 27001) | Avoids fines, audit failures | | **Stability** | Fixes software bugs and crashes | Reduces unplanned downtime | | **Performance** | Optimizes system performance | Better resource utilization | | **Supportability** | Maintains vendor support eligibility | Access to vendor technical support | | **Features** | Adds new functionality | Enables new business capabilities |

Security Benefits of Regular Patching

```
.....
.                               SECURITY PATCHING BENEFITS
.
.....
.  1. Closes known CVE vulnerabilities
.  2. Prevents zero-day exploit expansion
.  3. Hardens system against privilege escalation
.  4. Fixes buffer overflow vulnerabilities
.  5. Updates cryptographic libraries (OpenSSL, GnuTLS)
.  6. Patches kernel-level security flaws
.  7. Updates SELinux/AppArmor policies
.  8. Fixes authentication bypass vulnerabilities
.  9. Patches network stack vulnerabilities
. 10. Updates security scanning signatures
.....
```

Compliance Requirements

| Standard | Patching Requirement | Timeline | |-----|-----|-----| | PCI-DSS v4.0 | Requirement 6.3.3 · Install critical patches within 30 days | 30 days for critical | | HIPAA | §164.308(a)(1) · Security management process | Risk-based timeline | | SOX | Section 404 · IT General Controls | Quarterly minimum | | ISO 27001 | A.12.6.1 · Technical vulnerability management | Based on risk assessment | | NIST 800-53 | SI-2 · Flaw Remediation | 30 days critical, 90 days high | | CIS Benchmarks | Section 1.8 · Software updates | Monthly recommended |

Business Impact of NOT Patching

REAL-WORLD EXAMPLE: In 2017, Equifax suffered a massive data breach affecting 147 million people because they failed to patch a known Apache Struts vulnerability (CVE-2017-5638) that had a patch available for 2 months. The total cost exceeded \$1.4 billion.

Risks of Not Patching:

- 1 **Data Breach** · Unpatched vulnerabilities are the #1 attack vector
- 2 **Ransomware** · WannaCry exploited unpatched SMB vulnerabilities
- 3 **Compliance Violations** · Fines up to \$100,000 per incident (PCI-DSS)
- 4 **System Instability** · Known bugs cause crashes and data corruption

- 5 **Loss of Vendor Support** · Red Hat may refuse support for unpatched systems
- 6 **Reputation Damage** · Customer trust lost after breaches
- 7 **Legal Liability** · Negligence claims for known unpatched vulnerabilities
- 8 **Insurance Invalidation** · Cyber insurance may deny claims

1.2 Types of Patches

Patch Classification Matrix

Patch Type	Description	Priority	Reboot Required	Timeline
Emergency	Usually Yes	24-72 hours	Yes	Security (Critical) Fixes CVE with CVSS 9.0 P1 · Often
Yes	7-14 days	Yes	Yes	Security (Important) Fixes CVE with CVSS 7.0-8.9 P2 · High
Yes	7-14 days	Yes	Yes	Security (Moderate) Fixes CVE with CVSS 4.0-6.9 P3 · Medium
Yes	7-14 days	Yes	Yes	Security (Low) Fixes CVE with CVSS < 4.0 P4 · Low
Yes	7-14 days	Yes	Yes	Bug Fix Fixes software defects P3 · Medium
Yes	7-14 days	Yes	Yes	Kernel Update Linux kernel patches P2 · High
Yes	7-14 days	Yes	Yes	Firmware Hardware firmware updates P3 · Medium
Yes	7-14 days	Yes	Yes	Application Application-level patches Varies
Yes	7-14 days	Yes	Yes	Enhancement New features, improvements P4 · Low

Understanding CVSS Scores

CVSS Score Range:

- ... 0.0 = None (Informational)
- ... 0.1 - 3.9 = Low
- ... 4.0 - 6.9 = Medium
- ... 7.0 - 8.9 = High
- ... 9.0 - 10.0 = Critical

Example CVEs:

- ... CVE-2024-6387 (regreSSHion) = CVSS 8.1 (High) · OpenSSH RCE
- ... CVE-2021-44228 (Log4Shell) = CVSS 10.0 (Critical) · Remote Code Execution
- ... CVE-2023-4911 (Looney Tunables) = CVSS 7.8 (High) · glibc privilege escalation
- ... CVE-2024-1086 (nf_tables) = CVSS 7.8 (High) · Kernel privilege escalation

Detailed Patch Types

Security Patches

```
# View available security patches (RHEL 7/8/9)
yum updateinfo list security
# or
dnf updateinfo list --type=security
# Example Output:
# RHSA-2026:1234 Important/Sec.
kernel-5.14.0-362.24.1.el9_3.x86_64
# RHSA-2026:1235 Critical/Sec.
openssl-3.0.7-25.el9_3.x86_64
# RHSA-2026:1236 Moderate/Sec.
httpd-2.4.57-5.el9.x86_64
```

Kernel Patches

```
# Check current kernel
uname -r
# Example: 5.14.0-362.18.1.el9_3.x86_64
# List available kernel updates
yum list available kernel
# or
dnf list available kernel
# Kernel live patching (no reboot required)
# RHEL 7+
yum install kpatch
kpatch list
```

Bug Fix Patches

```
# View available bug fix patches
yum updateinfo list bugfix
# or
dnf updateinfo list --type=bugfix
# Example Output:
# RHBA-2026:5678  bugfix  systemd-252-14.el9_3.5.x86_64
# RHBA-2026:5679  bugfix  NetworkManager-1.44.0-3.el9.x86_64
```

Enhancement Patches

```
# View available enhancement patches
yum updateinfo list enhancement
# or
dnf updateinfo list --type=enhancement
```

1.3 Minor vs Major Upgrades

Comparison Table

Aspect	Minor Update (e.g., 8.8 · 8.9)	Major Upgrade (e.g., 8 · 9)
Risk Level	Low-Medium	High
Downtime	15-45 minutes	2-4 hours
Reboot	Usually required	Always required
Application Impact	Minimal	Significant (compatibility)
Rollback	Easy (package downgrade)	Complex (requires backup restore)
Testing Required	Standard UAT	Extended UAT + Application testing
Approval	Standard CAB	Emergency CAB + Business Approval
Frequency	Monthly/Quarterly	Every 2-3 years

Minor Update Example (RHEL 8.9 · 8.10)

```
# Check current version
cat /etc/redhat-release
# Output: Red Hat Enterprise Linux release 8.9 (Ootpa)
# Perform minor update
yum update -y
# After reboot
cat /etc/redhat-release
# Output: Red Hat Enterprise Linux release 8.10 (Ootpa)
```

Major Upgrade Example (RHEL 8 · 9)

```
# Using Leapp for in-place upgrade
yum install leapp leapp-upgrade
leapp preupgrade      # Check compatibility
leapp upgrade         # Perform upgrade (requires reboot)
# After upgrade
cat /etc/redhat-release
# Output: Red Hat Enterprise Linux release 9.x (Plow)
```

·**EXPERT TIP:** For enterprise environments with 2000+ servers, major upgrades should be planned as a separate project with dedicated resources, not as part of regular patching cycles. Use a phased approach: 5% pilot · 20% early adopters · remaining servers.

1.4 Patching Frequency Recommendations

Environment	Frequency	Window	Notes	-----	-----	-----	-----	Development	Weekly	Business
hours OK	Flexible scheduling	UAT/Staging	Bi-weekly	Business hours OK	Must mirror production					
Production (Non-Critical)	Monthly	Maintenance window	Standard CAB approval	Production (Critical/DMZ)						
Monthly	Maintenance window	Enhanced CAB + Business approval	Emergency (Critical CVE)	As needed						
Emergency window	Emergency CAB									

1.5 RHEL Version Lifecycle

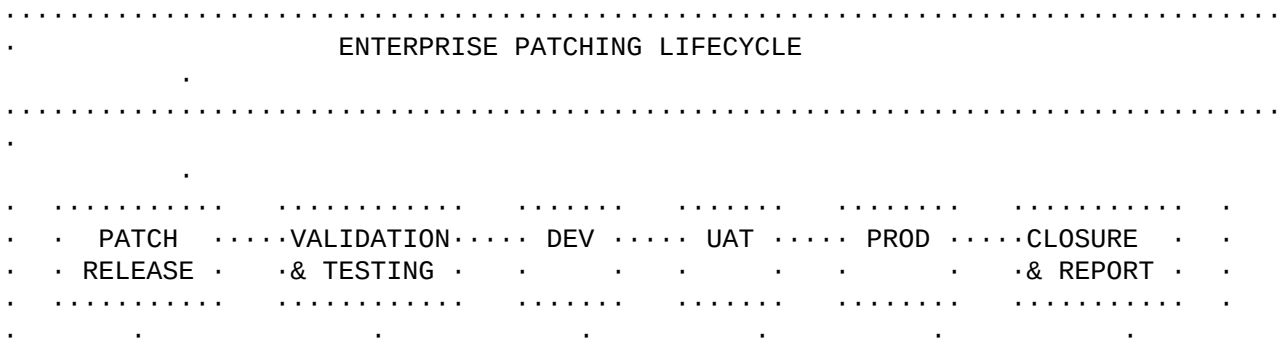
RHEL Version	Release Date	End of Full Support	End of Maintenance	ELS End						
-----	-----	-----	-----	-----	RHEL 6	Nov 2010	May 2016	Nov 2020		
Jun 2024	RHEL 7	Jun 2014	Aug 2019	Jun 2024	Jun 2028	RHEL 8	May 2019	May 2024	May 2029	
May 2032	RHEL 9	May 2022	May 2027	May 2032	May 2035	RHEL 10	May 2025	May 2030	May	
2035	May 2038									

·**WARNING:** RHEL 6 has reached End of Life. If you still have RHEL 6 systems, plan immediate migration to RHEL 8/9. Extended Life Support (ELS) provides limited security patches only.

[Diagram: Enterprise Patching Ecosystem showing Satellite Server, Ansible Tower, Chef Server, SCCM, and 2000+ managed nodes across DEV/UAT/PROD environments]

Section 2 · Patching Strategy

2.1 Enterprise Patching Lifecycle



.	.	.	.	.	.
.	.	.	.	.	.
Vendor pushes Document	Review CVEs	Apply to DEV env	Apply to UAT env	Apply to PROD env	
patches to results	Assess risk				
repository	Test in lab	Validate	Validate	Validate	
Close ticket		2 days	3 days	5 days	
.					
.					
.					

Lifecycle Phases Detail

Phase 1: Patch Release (Day 0)

Actions:

- ... Red Hat releases RHSA/RHBA/RHEA advisory
- ... Satellite synchronizes content from CDN
- ... Security team reviews CVE details and CVSS score
- ... Infrastructure team assesses applicability
- ... Patch is classified (Critical/Important/Moderate/Low)

Phase 2: Validation & Testing (Day 1-3)

Actions:

- ... Apply patch to isolated test server
- ... Run automated regression tests
- ... Verify application compatibility
- ... Check for known issues in Red Hat KB
- ... Document any special procedures required
- ... Create/update runbook if needed

Phase 3: DEV Environment (Day 3-5)

Actions:

- ... Apply patches to all DEV servers
- ... Run application smoke tests
- ... Verify service functionality
- ... Monitor for 24-48 hours
- ... Collect metrics and compare baselines
- ... Document any issues found

Phase 4: UAT Environment (Day 5-8)

Actions:

- ... Apply patches to all UAT servers
- ... Execute full regression test suite
- ... Performance testing (load/stress)
- ... User acceptance testing

- ... Monitor for 48-72 hours
- ... Obtain UAT sign-off
- ... Document results for CAB

Phase 5: Production (Day 8-14)

Actions:

- ... Obtain CAB approval
- ... Obtain Business approval (for critical systems)
- ... Execute production patching in maintenance window
- ... Follow batch strategy (50 · 100 · 250 · remaining)
- ... Validate each batch before proceeding
- ... Monitor applications and services
- ... Business sign-off after completion

Phase 6: Verification & Closure (Day 14-16)

Actions:

- ... Verify all servers patched successfully
- ... Generate compliance report
- ... Update CMDB/asset inventory
- ... Close Change Request
- ... Document lessons learned
- ... Update runbook if needed
- ... Archive patch documentation

2.2 Approval Matrix

Patch Priority	DEV Approval	UAT Approval	PROD Approval	Additional	
P1 · Critical (CVSS ≥ 9)	Team Lead	Manager	Emergency CAB + VP	CISO notification	P2 · High (CVSS 7-8.9)
Team Lead	Manager	Standard CAB	Manager approval	P3 · Medium (CVSS 4-6.9)	Team Lead
Team Lead	Standard CAB	Standard process	P4 · Low (CVSS ≤ 4)	Engineer	Team Lead
Standard CAB	Bundled monthly				

Emergency Patching Approval Flow

Security Team	CISO Approval	Emergency CAB	VP Approval
identifies	(30 min SLA)	(1 hour SLA)	(if needed)
Critical CVE			
Risk Assessment			
Implement			
completed in			within
24-72			
2 hours			hours

2.3 CAB (Change Advisory Board) Approval

CAB Meeting Structure

| Aspect | Standard CAB | Emergency CAB | |-----|-----|-----| | **Frequency** | Weekly (Tuesday 10:00 AM) | As needed (within 1 hour) | | **Attendees** | Change Manager, IT Director, App Owners, DBA, Network, Security | Change Manager, CISO, VP IT, Affected teams | | **Quorum** | 5 members minimum | 3 members minimum | | **Decision** | Approve/Reject/Defer | Approve/Reject | | **Documentation** | Full RFC with test results | Abbreviated RFC with justification | | **Lead Time** | 5 business days before window | Immediate |

CAB Submission Template

```
.....
.                                     CHANGE REQUEST - LINUX PATCHING                                     .
.....
. Change ID:          CHG-2026-XXXXX                                     .
. Priority:           P2 - High                                         .
. Type:              Standard / Emergency                             .
. Category:          Security Patching                                 .
.                                                             .
. DESCRIPTION:                                                .
. Apply RHSA-2026:XXXX security patches to 200 production          .
. servers to address CVE-2026-XXXXX (CVSS 8.5)                      .
.                                                             .
. AFFECTED SYSTEMS:                                           .
. - Web Tier: 50 servers (Apache/Nginx)                             .
. - App Tier: 80 servers (Tomcat/JBoss)                             .
. - DB Tier: 40 servers (PostgreSQL/MySQL)                         .
. - Utility: 30 servers (Monitoring/Backup)                         .
.                                                             .
. IMPLEMENTATION WINDOW:                                       .
. Start: Saturday 2026-06-27 22:00 UTC                             .
. End:   Sunday 2026-06-28 06:00 UTC                               .
. Duration: 8 hours                                             .
.                                                             .
. RISK LEVEL:         Medium                                         .
. IMPACT:             Medium (services rolling restart)           .
. ROLLBACK PLAN:      Yes (package downgrade + snapshot restore)   .
. ROLLBACK TIME:      30 minutes per server                       .
. TEST RESULTS:       Passed in DEV (6/20) and UAT (6/23)         .
.                                                             .
. APPROVALS REQUIRED:                                           .
.   . Change Manager   . IT Director   . Application Owner       .
.   . DBA Team Lead    . Security Team . Network Team           .
.....
```

2.4 Risk Assessment

Risk Assessment Matrix

Likelihood / Impact	Low Impact	Medium Impact	High Impact	Critical Impact
High Likelihood	Medium	High	Critical	Critical
Medium Likelihood	Low	Medium	High	Critical
Low Likelihood	Low	Low	Medium	High
Very Low Likelihood	Low	Low	Low	Medium

Common Risks and Mitigations

#	Risk	Likelihood	Impact	Mitigation
1	Patch causes application failure	Medium	High	Test in DEV/UAT, have rollback plan
2	Server fails to reboot	Low	Critical	Pre-validate GRUB, keep console access
3	Kernel panic after update	Low	Critical	Keep old kernel in GRUB, test first
4	Network loss after patch	Low	High	Document NIC config, validate routing
5	Cluster failover during patch	Medium	High	Patch one node at a time
6	Disk space insufficient	Medium	Medium	Check space before patching
7	Repository unreachable	Low	Medium	Validate connectivity, use local mirror
8	Dependency conflict	Medium	Medium	Test in DEV, use --exclude if needed
9	Patch window exceeded	Medium	Medium	Batch approach, parallel where safe
10	Monitoring false alerts	High	Low	Suppress alerts during window

2.5 Stakeholder Communication

Communication Matrix

Stakeholder	When to Notify	Method	Template
IT Management	5 days before	Email	Patch Summary
Application Owners	5 days before	Email + Meeting	Impact Analysis
DBA Team	5 days before	Email	DB Server List
Network Team	3 days before	Email	Network Requirements
Security Team	Continuous	Ticket updates	Status Report
Business Users	2 days before	Email	Downtime Notice
Service Desk	1 day before	Email + KB article	FAQ Document
NOC/Operations	Day of	Email + Phone	Execution Plan

Pre-Patching Communication Template

Subject: [PLANNED MAINTENANCE] Linux Security Patching - <Date> <Time>

Dear Stakeholders,

This is to notify you of planned maintenance activity:

WHAT: Monthly Linux Security Patching (RHSA-2026-XXXX)

WHEN: Saturday, June 27, 2026, 22:00 - 06:00 UTC

WHERE: Production Linux Servers (200 servers)

IMPACT: Rolling restarts - brief service interruptions (< 5 min per server)

Applications behind load balancers: NO user impact expected

Standalone applications: Brief interruption during restart

AFFECTED SERVICES:

- Web Portal (rolling restart, minimal impact)
- API Gateway (rolling restart, minimal impact)
- Batch Processing (suspended during window)
- Reporting (suspended during window)

ACTIONS REQUIRED:

- Application teams: Confirm no critical batch jobs during window
- DBA team: Confirm database backup completion before 21:00 UTC
- NOC: Suppress monitoring alerts from 22:00-06:00 UTC

ROLLBACK PLAN: Available - 30 min per server if issues detected

CONTACTS:

- Primary: Linux Team Lead - <phone>
- Secondary: On-call Engineer - <phone>
- Escalation: IT Director - <phone>

Please acknowledge receipt of this notification.
 Regards,
 Linux Infrastructure Team

2.6 Batch Strategy for 2000+ Servers

```

.....
.               BATCH PATCHING STRATEGY (2000 Servers)
.
.....
.
.
.  BATCH 1: 50 Servers (2.5%)      .... "Canary Group"
.
.  ... Non-critical servers
.
.  ... Monitoring/utility servers
.
.  ... Wait: 2 hours monitoring
.
.  ... Go/No-Go decision
.
.
.
.  BATCH 2: 100 Servers (5%)      .... "Early Adopters"
.
.  ... Low-risk application servers
.
.  ... Development-adjacent systems
.
.  ... Wait: 2 hours monitoring
.
.  ... Go/No-Go decision
.
.
.
.  BATCH 3: 250 Servers (12.5%)   .... "Mainstream"
.
.  ... Standard application servers
.
.  ... Non-customer-facing
.
.  ... Wait: 1 hour monitoring
.
.  ... Go/No-Go decision
.
.
.
.  BATCH 4: 1600 Servers (80%)    .... "Remaining Fleet"
.

```

-

Go/No-Go Criteria Between Batches

Criteria	Go	No-Go	----- -----	All servers rebooted successfully	· Yes	· Any failures	All services running	· Yes	· Any service down
Application health checks passing	· Yes	· Any failures	No	Unexpected errors in logs	· Yes	· Critical errors found	No	Network connectivity verified	· Yes
Any connectivity issues	No	Monitoring shows normal metrics	· Yes	· Anomalies detected	No	Rollbacks required	· Yes	· Rollback performed	

•**EXPERT TIP:** In a 2000-server environment, the Canary group (Batch 1) should include at least one representative server from each application tier. This ensures you catch tier-specific issues before mass deployment.

Section 3 · Pre-Patching Checklist

•**BEGINNER NOTE:** This checklist **MUST** be completed for **EVERY** server before patching. Save the output of each command to a file for comparison after patching. Use: `command > /tmp/pre_patch_$(hostname)_$(date +%Y%m%d).txt`

3.1 Pre-Patch Data Collection Script

```
#!/bin/bash
# pre_patch_check.sh - Run before patching
# Usage: ./pre_patch_check.sh > /tmp/pre_patch_$(hostname)_$(date +%Y%m%d).log 2>&1
echo "=====
echo "PRE-PATCH CHECK: $(hostname)"
echo "Date: $(date)"
echo "Engineer: $(whoami)"
echo "=====
```

3.2 System Information

```
# Hostname and OS Version
hostname
cat /etc/redhat-release
uname -r
uptime
# Expected Output:
# webserver01.example.com
```

```
# Red Hat Enterprise Linux release 8.9 (Ootpa)
# 4.18.0-513.18.1.el8_9.x86_64
# 10:30:15 up 45 days, 3:22, 2 users, load average: 0.15, 0.20,
0.18
```

3.3 CPU Check

```
# Check CPU utilization
top -bn1 | head -5
# Expected Output:
# top - 10:30:15 up 45 days, 3:22, 2 users, load average: 0.15,
0.20, 0.18
# Tasks: 245 total, 1 running, 244 sleeping, 0 stopped, 0
zombie
# %Cpu(s): 5.2 us, 2.1 sy, 0.0 ni, 92.3 id, 0.2 wa, 0.0 hi,
0.2 si, 0.0 st
# Check load average (should be < number of CPUs)
cat /proc/loadavg
# Expected: 0.15 0.20 0.18 1/245 12345
# Number of CPUs
nproc
# Expected: 4 (or however many CPUs the server has)
# Check for CPU-intensive processes
ps aux --sort=-%cpu | head -10
# .. WARNING: If load average > 2x number of CPUs, investigate before
patching
```

3.4 Memory Check

```
# Check memory usage
free -h
# Expected Output:
#
#          total        used        free        shared
buff/cache  available
# Mem:      15Gi        8.2Gi        2.1Gi        256Mi
5.1Gi      6.8Gi
# Swap:      4.0Gi        0.0Gi        4.0Gi
# Detailed memory info
cat /proc/meminfo | grep -E
"MemTotal|MemFree|MemAvailable|SwapTotal|SwapFree"
# Expected:
# MemTotal:      16384000 kB
# MemFree:       2201600 kB
# MemAvailable:  7168000 kB
# SwapTotal:     4194304 kB
# SwapFree:      4194304 kB
# .. WARNING: If available memory < 500MB, free up memory before
patching
# .. WARNING: If swap usage > 50%, investigate memory pressure
```

3.5 Swap Check

```
# Check swap usage
swapon -s
# Expected Output:
# Filename                Type          Size          Used
Priority
# /dev/dm-1                partition    4194300        0          -2
# Swap usage percentage
free | awk '/Swap/{if($2>0) printf "Swap Usage: %.1f%%\n",
$3/$2*100; else print "No swap configured"}'
# Expected: Swap Usage: 0.0%
# .. ALERT: Swap usage > 20% indicates memory pressure
# Action: Investigate with 'top' and check for memory leaks
```

3.6 Disk Space Check

```
# Check filesystem usage
df -hT
# Expected Output:
# Filesystem              Type      Size  Used Avail Use% Mounted on
# /dev/mapper/vg-root     xfs       50G   22G   28G   44% /
# /dev/mapper/vg-var      xfs       20G    8G   12G   40% /var
# /dev/mapper/vg-tmp      xfs       10G    1G    9G   10% /tmp
# /dev/sda1               xfs        1G   200M   824M   20% /boot
# /dev/mapper/vg-home     xfs       10G    2G    8G   20% /home
# Check /boot specifically (CRITICAL for kernel updates)
df -h /boot
# Expected: Use% should be < 80%
# Check /var (where yum cache and logs reside)
df -h /var
# Expected: Use% should be < 80%
# .. CRITICAL: /boot MUST have at least 100MB free for kernel updates
# .. CRITICAL: /var MUST have at least 2GB free for package
download/install
# .. CRITICAL: / (root) MUST have at least 5GB free
# Check for large files consuming space
du -sh /var/log/* | sort -rh | head -10
du -sh /tmp/* | sort -rh | head -10 2>/dev/null
# Clean old kernels if /boot is full (keep last 2)
# RHEL 7:
package-cleanup --oldkernels --count=2
# RHEL 8/9:
dnf remove --oldinstallonly --setopt installonly_limit=2 kernel
```

3.7 Inode Check

```
# Check inode usage
df -i
# Expected Output:
# Filesystem              Inodes    IUsed    IFree IUse% Mounted on
# /dev/mapper/vg-root     3276800  245000  3031800    8% /
# /dev/mapper/vg-var      1310720   89000  1221720    7% /var
```

```
# /dev/sda1          65536    320    65216    1% /boot
# .. CRITICAL: If IUse% > 90%, patching will FAIL
# Common cause: Too many small files in /var/spool/postfix, /tmp,
# or session files
# Find directories with most inodes
find / -xdev -printf '%h\n' | sort | uniq -c | sort -rn | head -10
```

3.8 Filesystem Check

```
# Check for read-only filesystems
mount | grep -w "ro,"
# Expected: Should return nothing (all should be read-write)
# Exception: /boot/efi might be mounted read-only
# Check for filesystem errors
dmesg | grep -i "ext4\|xfs\|filesystem\|error" | tail -20
# Check filesystem mount options
mount | column -t
# Verify fstab consistency
findmnt --verify
# Expected: No errors
# Check LVM status
lvs
vgs
pvs
# All should show normal status
```

3.9 Running Services Check

```
# List all enabled services (RHEL 7/8/9)
systemctl list-unit-files --state=enabled --type=service
# List running services
systemctl list-units --type=service --state=running
# Check critical services
for svc in sshd crond rsyslog NetworkManager firewalld; do
    echo -n "$svc: "
    systemctl is-active $svc
done
# Expected: All should show "active"
# Record application-specific services
for svc in httpd nginx tomcat postgresql mysqld oracle; do
    if systemctl is-enabled $svc 2>/dev/null; then
        echo "$svc: $(systemctl is-active $svc)"
    fi
done
# RHEL 6 (sysvinit):
chkconfig --list | grep "3:on"
service --status-all 2>/dev/null | grep running
```

3.10 Network Configuration Check

```
# IP addresses
```

```

ip addr show
# or (deprecated but still used)
ifconfig -a
# Expected: All interfaces should have correct IP addresses
# Check default gateway
ip route show default
# Expected: default via 10.0.0.1 dev eth0 proto static metric 100
# Full routing table
ip route show
# Record this for comparison after patching
# Network interfaces status
ip link show
# All operational interfaces should show "state UP"
# Check for bonded/teamed interfaces
cat /proc/net/bonding/bond0 2>/dev/null
teamctl team0 state 2>/dev/null
# Active connections
ss -tuln
# or
netstat -tuln
# Record all listening ports
# Check network connectivity to critical systems
ping -c 3 <gateway_ip>
ping -c 3 <satellite_server>
ping -c 3 <dns_server>
ping -c 3 <ntp_server>

```

3.11 DNS Resolution Check

```

# Check DNS configuration
cat /etc/resolv.conf
# Expected:
# nameserver 10.0.0.53
# nameserver 10.0.0.54
# search example.com
# Test DNS resolution
nslookup $(hostname)
dig $(hostname -f) +short
host $(hostname -f)
# Test external resolution (if applicable)
nslookup satellite.example.com
nslookup repo.example.com
# Test reverse DNS
dig -x $(hostname -i) +short

```

3.12 Routing Table Check

```

# Full routing table
ip route show
# or
route -n
# Expected Output:

```

```
# default via 10.0.0.1 dev eth0 proto static metric 100
# 10.0.0.0/24 dev eth0 proto kernel scope link src 10.0.0.50 metric
100
# 172.16.0.0/16 via 10.0.0.1 dev eth0
# Check for static routes in config
cat /etc/sysconfig/network-scripts/route-* 2>/dev/null      # RHEL 7
nmcli connection show <connection-name> | grep route      # RHEL
8/9
# Verify routing to critical networks
traceroute -n <satellite_server> 2>/dev/null | head -5
```

3.13 Log Check

```
# Check system logs for errors
journalctl -p err --since "1 hour ago" --no-pager | tail -20
# or (RHEL 6)
tail -50 /var/log/messages | grep -i error
# Check for OOM (Out of Memory) events
dmesg | grep -i "oom\|out of memory"
# Expected: No output (no OOM events)
# Check for hardware errors
dmesg | grep -i "hardware\|error\|fail" | tail -10
# Check audit log
tail -20 /var/log/audit/audit.log
# Check secure log (authentication)
tail -20 /var/log/secure
# Verify log rotation is working
ls -la /var/log/messages*
ls -la /var/log/secure*
```

3.14 Current Kernel Check

```
# Running kernel
uname -r
# Expected: 4.18.0-513.18.1.el8_9.x86_64
# All installed kernels
rpm -qa kernel | sort
# Expected: Shows 2-3 kernel versions
# Verify GRUB configuration
# RHEL 7:
cat /boot/grub2/grub.cfg | grep menuentry
grub2-editenv list
# RHEL 8/9:
grubby --default-kernel
grubby --info=ALL
# Check GRUB default
grub2-editenv list
# Expected: saved_entry=<current_kernel_entry>
# .. CRITICAL: Verify you can see current kernel in GRUB menu
# If GRUB is corrupted, the server may not boot after kernel update
```

3.15 Repository Check

```
# List enabled repositories
yum repolist enabled
# or
dnf repolist enabled
# Expected Output (Satellite-managed):
# repo id                                repo name
# rhel-8-for-x86_64-baseos-rpms          Red Hat Enterprise Linux 8 -
BaseOS
# rhel-8-for-x86_64-appstream-rpms       Red Hat Enterprise Linux 8 -
AppStream
# satellite-tools-6.x-for-rhel-8         Red Hat Satellite Tools
# Check repository connectivity
yum repolist -v 2>/dev/null | grep -E "Repo-baseurl|Repo-status"
# Verify subscription status (RHEL)
subscription-manager status
subscription-manager list --consumed
# Expected: Status should be "Current"
# Check for available updates
yum check-update
# or
dnf check-update
# Record the list of available updates
# Count available updates
yum check-update 2>/dev/null | grep -v "^$\\|Loaded\\|repo" | wc -l
```

3.16 Backup Verification

```
# Verify recent backup exists
# (Commands depend on your backup solution)
# NetBackup:
/usr/opensv/netbackup/bin/bpclimagelist -client $(hostname -f) -d
$(date -d "yesterday" +%m/%d/%Y)
# Commvault:
# Check via Commvault console
# Veeam:
# Check via Veeam console
# Verify VM snapshot (if applicable)
# Check via vCenter/VMware console
# Local backup of critical configs
mkdir -p /root/pre_patch_backup_$(date +%Y%m%d)
cp -a /etc/fstab /root/pre_patch_backup_$(date +%Y%m%d)/
cp -a /etc/default/grub /root/pre_patch_backup_$(date +%Y%m%d)/
cp -a /etc/sysconfig/network-scripts/ /root/pre_patch_backup_$(date
+%Y%m%d)/ 2>/dev/null
cp -a /etc/NetworkManager/ /root/pre_patch_backup_$(date +%Y%m%d)/
2>/dev/null
cp -a /boot/grub2/grub.cfg /root/pre_patch_backup_$(date +%Y%m%d)/
echo "Backup verification: COMPLETE"
```

3.17 Snapshot Verification

```

# For VMware VMs - verify snapshot can be taken
# (Usually done via vCenter API or Ansible vmware module)
# For AWS EC2:
aws ec2 create-snapshot --description "Pre-patch $(hostname) $(date +%Y%m%d)"
# For Azure:
az snapshot create --name "prepatch-$(hostname)-$(date +%Y%m%d)"
# For LVM snapshot (bare metal):
# Check VG free space for snapshot
vgs
# Expected: VFree should have enough space (at least 5GB recommended)
# Create LVM snapshot (if using LVM)
lvcreate -L 5G -s -n root_snap /dev/vg/root
# Verify:
lvs | grep snap

```

3.18 Application Health Check

```

# Web Server (Apache)
curl -s -o /dev/null -w "%{http_code}" http://localhost/health
# Expected: 200
# Web Server (Nginx)
curl -s -o /dev/null -w "%{http_code}" http://localhost/status
# Expected: 200
# Tomcat
curl -s -o /dev/null -w "%{http_code}" http://localhost:8080/
# Expected: 200
# JBoss/WildFly
/opt/jboss/bin/jboss-cli.sh --connect
command=":read-attribute(name=server-state)"
# Expected: "result" => "running"
# MySQL/MariaDB
mysqladmin ping
mysqladmin status
# Expected: mysqld is alive
# PostgreSQL
pg_isready
# Expected: accepting connections
# Oracle
sqlplus -S / as sysdba <<< "SELECT STATUS FROM V\$INSTANCE;"
# Expected: OPEN
# Custom application health endpoints
curl -s http://localhost:<port>/api/health | python3 -m json.tool

```

3.19 Complete Pre-Patch Checklist Table

#	Check	Command	Expected Result	Status	1	OS Version
1	OS Version	cat /etc/redhat-release	RHEL 8.x/9.x	2	Kernel	uname -r
3	Uptime	uptime	Server is up	4	CPU Load	cat /proc/loadavg
5	Memory Available	free -h	> 500MB available	6	Swap Usage	free -h
7	Disk Space	df -h	< 20% used			

80% used | · | 8 | Disk Space /boot | `df -h /boot` | > 100MB free | · | 9 | Disk Space /var | `df -h /var` | > 2GB free | · | 10 | Inode Usage | `df -i` | < 90% used | · | 11 | Read-only FS | `mount \ | grep "ro,"` | No RO filesystems | · | 12 | Services Running | `systemctl list-units` | All services active | · | 13 | Network (IP) | `ip addr show` | IPs configured | · | 14 | Network (Gateway) | `ip route show default` | Gateway reachable | · | 15 | DNS Resolution | `nslookup $(hostname)` | Resolves correctly | · | 16 | NTP Sync | `chronyc sources` | Time synchronized | · | 17 | Repositories | `yum repolist` | All repos enabled | · | 18 | Subscription | `subscription-manager status` | Status: Current | · | 19 | Backup Verified | Backup tool check | Recent backup exists | · | 20 | Snapshot Ready | LVM/VM check | Snapshot space available | · | 21 | Application Health | Health check URL | 200 OK | · | 22 | Cluster Status | Cluster check command | All nodes online | · | 23 | GRUB Config | `grubby --default-kernel` | Valid kernel path | · | 24 | SELinux | `getenforce` | Mode noted (Enforcing) | · | 25 | Firewall | `firewall-cmd --state` | Running | · |

·**EXPERT TIP:** Automate this checklist using Ansible. Run it across all 2000 servers simultaneously and generate a report showing which servers are ready for patching and which need attention first.

- **COMMON MISTAKE:** Engineers often skip the `/boot` space check. If `/boot` is full and you install a kernel update, the transaction will fail midway, potentially leaving the system in an inconsistent state.

Section 4 · Patching Using Red Hat Satellite

·WHERE DO SATELLITE COMMANDS RUN?

```
.
.
.
SATELLITE COMMANDS RUN FROM TWO PLACES:
.
.
.
1. SATELLITE SERVER (e.g., satellite.corp.com)
.
. hammer commands, content management, sync, publish,
promote
. Requires: RHEL 8 (minimum), 4 CPU, 20GB RAM, 500GB+
storage
.
.
2. TARGET HOST (managed client)
.
. subscription-manager, yum/dnf update,
needs-restarting
. These are the 2000 servers you are patching
.
.
```

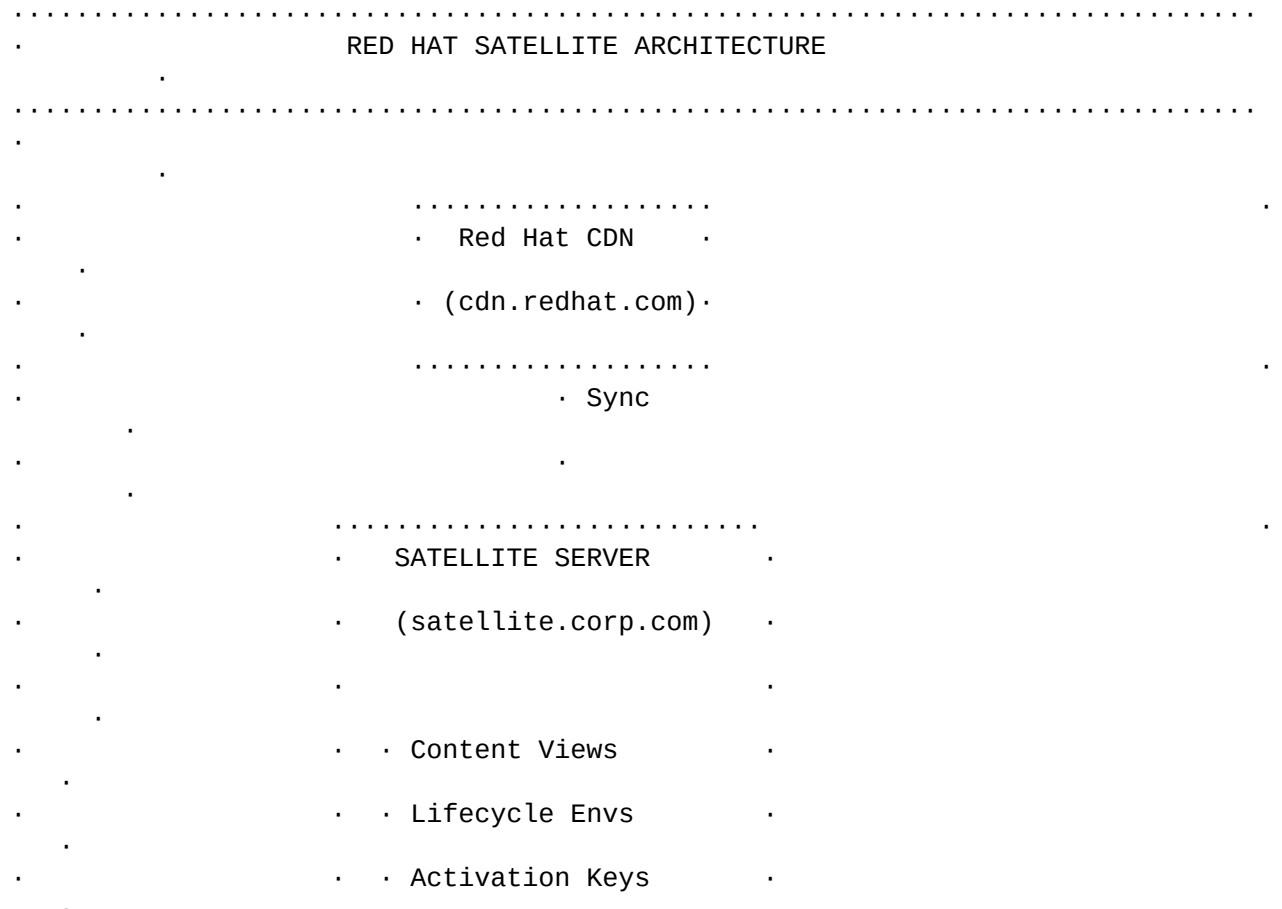
Satellite Server Installation Requirements

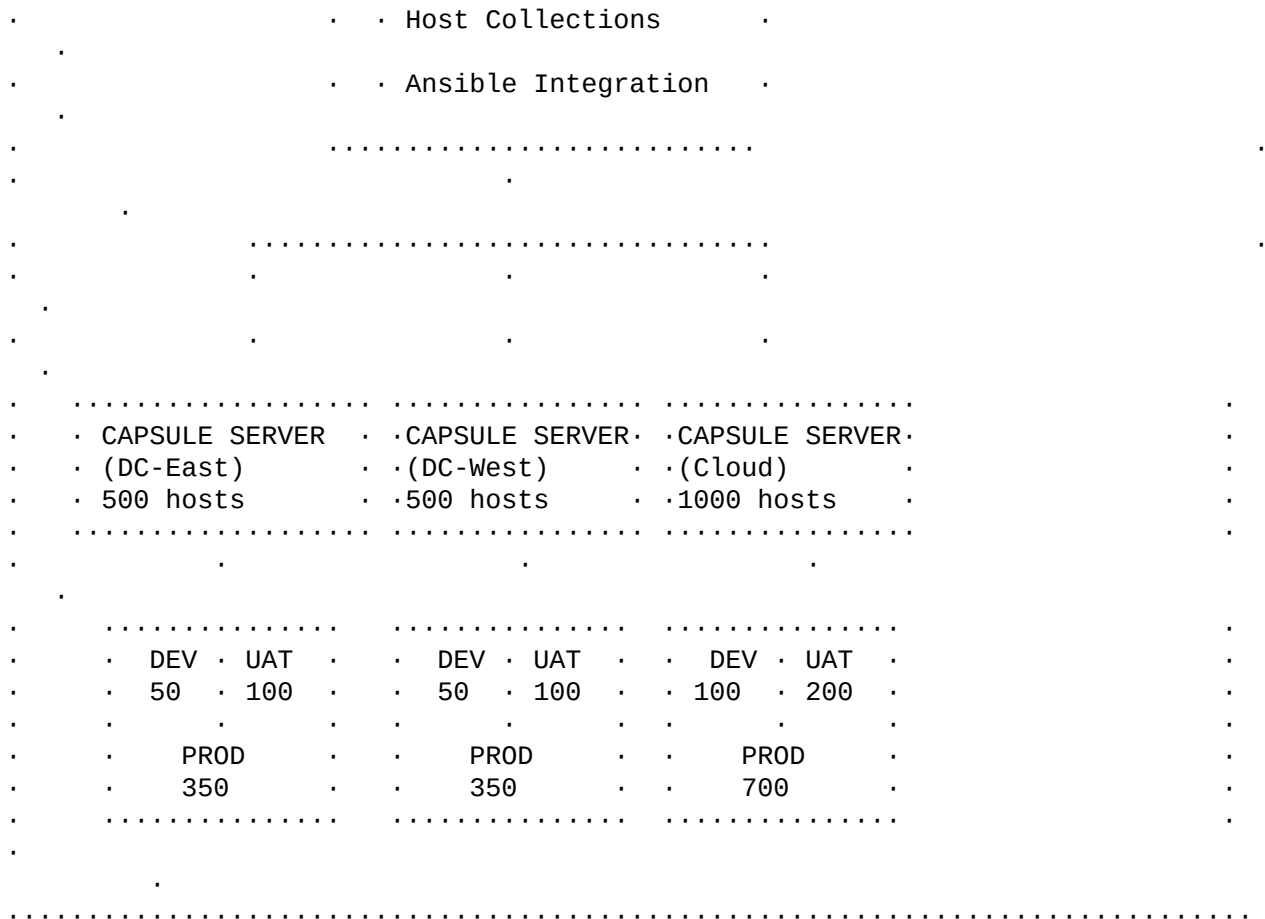
Requirement	Minimum	Recommended (2000 nodes)	OS	RHEL 8.x or 9.x
RHEL 8.x (most tested)	CPU	4 cores 8+ cores	RAM	20 GB 32+ GB
/var/lib/pulp	500 GB	1+ TB (content storage)	/var/lib/pgsql	20 GB 100 GB (database)
Network	10 Mbps	1 Gbps		

[Run on: Satellite Server - e.g., satellite.corp.com]

```
# Install Red Hat Satellite (requires valid Satellite subscription)
# Step 1: Register and enable repos
subscription-manager register
subscription-manager repos --enable=rhel-8-for-x86_64-baseos-rpms \
--enable=rhel-8-for-x86_64-appstream-rpms \
--enable=satellite-6.15-for-rhel-8-x86_64-rpms \
--enable=satellite-maintenance-6.15-for-rhel-8-x86_64-rpms
# Step 2: Install Satellite packages
dnf install satellite -y
# Step 3: Run the installer
satellite-installer --scenario satellite \
--foreman-initial-admin-username admin \
--foreman-initial-admin-password 'SecurePassword123!' \
--foreman-proxy-dns true \
--foreman-proxy-dhcp true
```

4.1 Red Hat Satellite Architecture





[Diagram: Red Hat Satellite 6.x Architecture with Capsule Servers and Content Views]

4.2 Key Satellite Concepts

Content Views

A Content View is a curated set of repositories that defines what packages are available to hosts. This allows you to control exactly which patches reach each environment.

```
# [Run on: Satellite Server - e.g., satellite.corp.com]
# List Content Views
hammer content-view list --organization "My Org"
# Example Output:
# ID | NAME | COMPOSITE | LAST PUBLISHED
# 1 | RHEL8-BaseOS | false | 2026-06-15 10:00:00
# 2 | RHEL8-AppStream | false | 2026-06-15 10:00:00
# 3 | RHEL9-BaseOS | false | 2026-06-15 10:00:00
# 5 | CCV-RHEL8-Production | true | 2026-06-15 12:00:00
# 6 | CCV-RHEL9-Production | true | 2026-06-15 12:00:00
```

Lifecycle Environments

```
# [Run on: Satellite Server - e.g., satellite.corp.com]
```

```
# List Lifecycle Environments
hammer lifecycle-environment list --organization "My Org"
# Example Output:
# ID | NAME          | PRIOR
# 1  | Library       | -
# 2  | Development   | Library
# 3  | UAT           | Development
# 4  | Production    | UAT
# Promotion Path:
# Library · Development · UAT · Production
```

Activation Keys

```
# [Run on: Satellite Server - e.g., satellite.corp.com]
# List Activation Keys
hammer activation-key list --organization "My Org"
# Example Output:
# ID | NAME                                | LIFECYCLE ENVIRONMENT | CONTENT
VIEW
# 1  | ak-rhel8-dev                        | Development           |
CCV-RHEL8-Dev
# 2  | ak-rhel8-uat                       | UAT                   |
CCV-RHEL8-UAT
# 3  | ak-rhel8-prod                      | Production            |
CCV-RHEL8-Production
# 4  | ak-rhel9-dev                       | Development           |
CCV-RHEL9-Dev
# 5  | ak-rhel9-prod                      | Production            |
CCV-RHEL9-Production
```

Host Collections

```
# [Run on: Satellite Server - e.g., satellite.corp.com]
# List Host Collections
hammer host-collection list --organization "My Org"
# Example Output:
# ID | NAME                                | LIMIT | HOSTS
# 1  | hc-webservers-prod                 | None  | 50
# 2  | hc-appservers-prod                 | None  | 80
# 3  | hc-dbservees-prod                   | None  | 40
# 4  | hc-batch1-canary                    | 50    | 50
# 5  | hc-batch2-early                     | 100   | 100
```

4.3 Step-by-Step: Satellite Patching Procedure

Step 1: Synchronize Repositories

```
# [Run on: Satellite Server - e.g., satellite.corp.com]
# Sync all repositories
hammer product synchronize --organization "My Org" --name "Red Hat
Enterprise Linux 8"
# Sync specific repository
```

```

hammer repository synchronize \
  --organization "My Org" \
  --product "Red Hat Enterprise Linux 8 for x86_64" \
  --name "Red Hat Enterprise Linux 8 for x86_64 - BaseOS RPMs 8"
# Verify sync status
hammer sync-plan list --organization "My Org"
hammer repository info --organization "My Org" --product "Red Hat
Enterprise Linux 8 for x86_64" \
  --name "Red Hat Enterprise Linux 8 for x86_64 - BaseOS RPMs 8"

```

[Screenshot: Satellite WebUI showing repository sync status with green checkmarks]

Step 2: Publish Content View

```

# [Run on: Satellite Server - e.g., satellite.corp.com]
# Publish new version of Content View
hammer content-view publish \
  --organization "My Org" \
  --name "RHEL8-BaseOS" \
  --description "June 2026 Security Patches - RHSA-2026:1234"
# Verify publication
hammer content-view version list \
  --organization "My Org" \
  --content-view "RHEL8-BaseOS"
# Example Output:
# ID | VERSION | LIFECYCLE ENVIRONMENTS
# 15 | 15.0    | Library
# 14 | 14.0    | Development, UAT, Production

```

[Screenshot: Content View publish confirmation in Satellite WebUI]

Step 3: Promote to Development

```

# [Run on: Satellite Server - e.g., satellite.corp.com]
# Promote to Development environment
hammer content-view version promote \
  --organization "My Org" \
  --content-view "RHEL8-BaseOS" \
  --version 15.0 \
  --to-lifecycle-environment "Development"
# Verify promotion
hammer content-view version list \
  --organization "My Org" \
  --content-view "RHEL8-BaseOS"
# Expected: Version 15.0 now shows "Library, Development"

```

Step 4: Register Host to Satellite (if not already registered)

```

# [Run on: Target Host (managed client) - the server being
registered]
# On the client host:
# RHEL 8/9 registration
curl -sS --insecure https://satellite.corp.com/register \

```

```

-H 'Authorization: Bearer <token>' | bash
# Or using subscription-manager:
subscription-manager register \
  --org="My_Org" \
  --activationkey="ak-rhel8-prod" \
  --force
# Install Katello agent (if needed)
yum install katello-agent -y
systemctl enable goferd
systemctl start goferd
# Verify registration
subscription-manager identity
subscription-manager status
# Expected Output:
# system identity: 12345678-abcd-1234-5678-abcdef123456
# name: webserver01.corp.com
# org name: My Org
# org ID: My_Org
# Status:
# Overall Status: Current

```

[Screenshot: Host registered in Satellite showing green subscription status]

Step 5: Verify Subscription and Repository

```

# [Run on: Target Host (managed client)]
# Check subscription status
subscription-manager list --consumed
# List enabled repositories
subscription-manager repos --list-enabled
# Expected Output:
# Repo ID:   rhel-8-for-x86_64-baseos-rpms
# Repo Name: Red Hat Enterprise Linux 8 for x86_64 - BaseOS (RPMs)
# Repo URL:  https://satellite.corp.com/pulp/repos/...
# Enabled:   1
# Verify content from Satellite (not CDN)
yum repolist -v 2>/dev/null | grep "Repo-baseurl"
# Should point to satellite.corp.com or capsule.corp.com

```

Step 6: Check Available Errata

```

# [Run on: Satellite Server - e.g., satellite.corp.com]
# From Satellite (hammer):
hammer host errata list --host webserver01.corp.com
# Example Output:
# ID           | TYPE      | TITLE
# RHSA-2026:1234 | security  | Important: kernel security update
# RHSA-2026:1235 | security  | Critical: openssl security update
# RHBA-2026:5678 | bugfix    | systemd bug fix update
# From the host:
yum updateinfo list
# or
dnf updateinfo list

```

```
# Count available security errata
yum updateinfo list security | wc -l
```

Step 7: Apply Patches

```
# [Run on: Satellite Server - for hammer commands; OR Target Host -
for yum commands]
# Apply all available errata (from Satellite):
hammer host errata apply \
  --host webserver01.corp.com \
  --errata-ids "RHSA-2026:1234,RHSA-2026:1235,RHBA-2026:5678"
# Apply to Host Collection (batch):
hammer host-collection errata install \
  --organization "My Org" \
  --name "hc-batch1-canary" \
  --errata-ids "RHSA-2026:1234"
# From the host (all security patches):
yum update --security -y
# or
dnf update --security -y
# Apply specific errata from host:
yum update --advisory=RHSA-2026:1234 -y
# Apply all available updates:
yum update -y
```

[Screenshot: Satellite showing errata installation progress for host collection]

Step 8: Validate Patch Installation

```
# Verify patches applied
yum history info last
# Shows details of last transaction
# Check if specific package was updated
rpm -q kernel
rpm -q openssl
# Verify no errors in yum log
tail -50 /var/log/yum.log # RHEL 7
tail -50 /var/log/dnf.log # RHEL 8/9
# Check for errata still outstanding
yum updateinfo list security
# Expected: Fewer or no entries remaining
```

Step 9: Reboot (if required)

```
# Check if reboot is required
needs-restarting -r
# Exit code 0 = no reboot needed
# Exit code 1 = reboot required
# Or check manually:
if [ $(rpm -q kernel --last | head -1 | awk '{print $1}' | sed
's/kernel-//' ) != $(uname -r) ]; then
  echo "REBOOT REQUIRED - New kernel installed"
else
```

```

    echo "No reboot required"
fi
# Schedule reboot
shutdown -r +5 "System reboot for patching in 5 minutes"
# Or immediate (during maintenance window):
reboot

```

Step 10: Post-Reboot Validation

```

# Verify new kernel booted
uname -r
# Expected: New kernel version
# Verify all services started
systemctl list-units --state=failed
# Expected: 0 loaded units listed
# Run post-patch validation (see Section 11)
# Quick validation:
systemctl is-active sshd
systemctl is-active crond
ip addr show
ping -c 3 <gateway>
curl -s http://localhost/<health_endpoint>

```

[Screenshot: Satellite host detail page showing compliant status after patching]

4.4 Satellite Content View Promotion Workflow

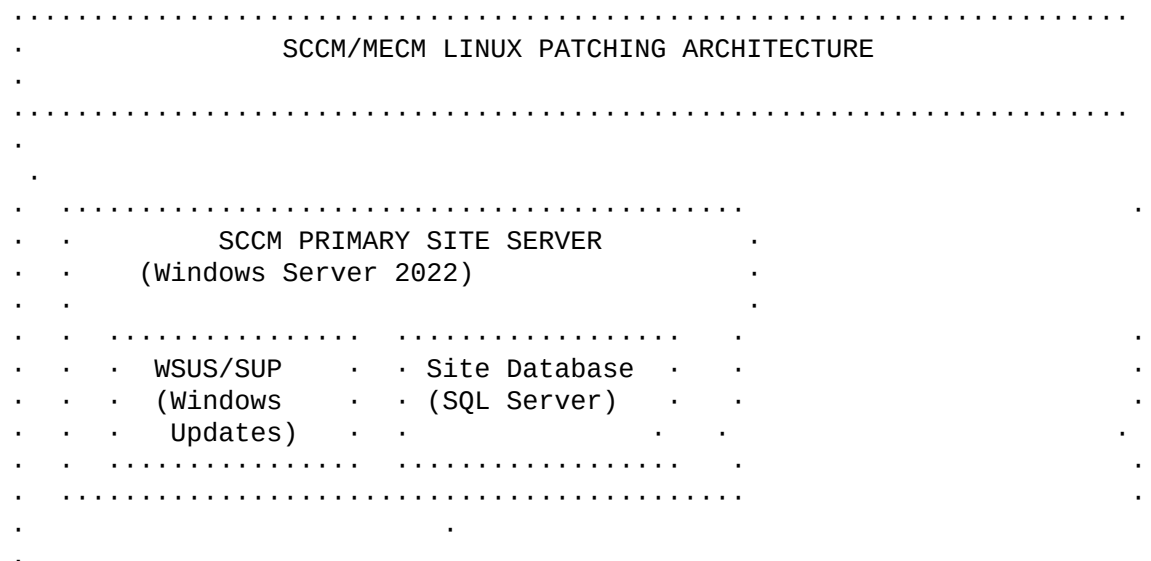
.....			
• PUBLISH	LIBRARY	DEV	UAT PROD
• CV v15 •	• (Automatic) •	• •	• •
.....			
		Wait 2d	Wait 3d Wait 5d
		Test OK?	Test OK? CAB OK?
		•	•
		Yes...	Yes... Promote
		No...STOP	No...STOP

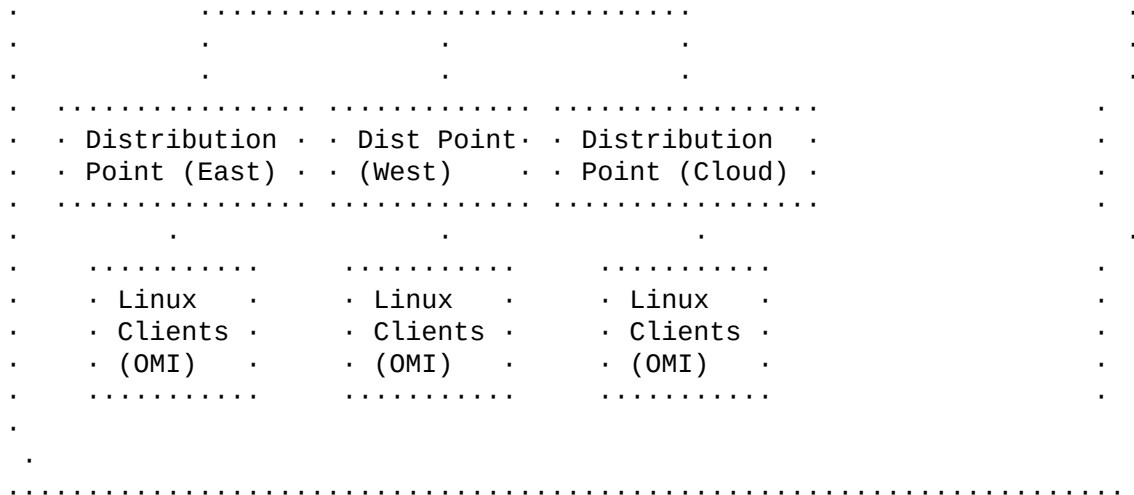
4.5 Satellite CLI Cheat Sheet

```

# Organization management
hammer organization list
hammer organization info --name "My Org"
# Host management
hammer host list --organization "My Org"
hammer host info --name "webserver01.corp.com"
hammer host update --name "webserver01.corp.com" --host-collection
"hc-webservers-prod"
# Errata management
hammer erratum list --organization "My Org" --type security
hammer erratum info --id RHSA-2026:1234
# Content View management
hammer content-view list --organization "My Org"

```





[Diagram: SCCM Site hierarchy with Linux management points]

5.2 SCCM Linux Client Installation

```
# Download SCCM Linux client from Distribution Point
# The client package is typically: configmgr-client-x86_64.tar.gz
# Extract and install
tar xzf configmgr-client-x86_64.tar.gz
cd configmgr-client/
# Install prerequisites
yum install -y omi openssl curl
# Install the SCCM client
./install -mp <management_point_FQDN> \
  -sitecode <site_code> \
  -fsp <fallback_status_point> \
  /UsePKICert
# Verify installation
/opt/microsoft/configmgr/bin/ccmexec -status
# Check client registration
cat /var/opt/microsoft/scxcm.log | grep "Registration"
```

[Screenshot: SCCM console showing Linux devices in device collection]

5.3 Collections and Maintenance Windows

Creating Linux Collections

- In SCCM Console:
1. Navigate to Assets and Compliance · Device Collections
 2. Create Device Collection:
 - Name: "Linux Servers - Production - Batch 1"
 - Limiting Collection: "All Linux Servers"
 - Membership Rule: Query-based

Query Example (WQL):
 SELECT * FROM SMS_R_System

```
WHERE SMS_R_System.OperatingSystemNameandVersion LIKE "%Red Hat%"
AND SMS_R_System.SystemOUName = "OU=Production,OU=Linux"
```

Maintenance Windows Configuration

```
| Collection | Window Name | Schedule | Duration | |-----|-----|-----|-----| | Linux-Prod-Batch1 |
MW-Patch-Sat-22 | Saturday 22:00 | 4 hours | | Linux-Prod-Batch2 | MW-Patch-Sat-23 | Saturday 23:00 | 4 hours | |
Linux-Prod-Batch3 | MW-Patch-Sun-00 | Sunday 00:00 | 4 hours | | Linux-Prod-Batch4 | MW-Patch-Sun-02 |
Sunday 02:00 | 4 hours |
```

5.4 Patch Deployment Procedure

Step 1: Create Software Update Group

In SCCM Console:

1. Software Library · Software Updates · All Software Updates
2. Filter: Product = "Red Hat Enterprise Linux"
3. Select applicable updates
4. Right-click · Create Software Update Group
5. Name: "Linux-Security-June2026"

[Screenshot: SCCM Software Update Group creation wizard]

Step 2: Deploy Updates

1. Right-click Software Update Group · Deploy
2. Select Collection: "Linux Servers - Production - Batch 1"
3. Deployment Settings:
 - Type: Required
 - Deadline: Match maintenance window
 - Restart behavior: Server restart
4. Create Deployment Package
5. Select Distribution Points
6. Download settings: Download from DP

Step 3: Monitor Deployment

In SCCM Console:

Monitoring · Deployments · "Linux-Security-June2026"

Status categories:

- Compliant: Patch installed successfully
- In Progress: Currently installing
- Error: Installation failed
- Unknown: Client not reporting

[Screenshot: SCCM deployment monitoring showing compliance percentage]

5.5 Linux-Side Commands for SCCM-Managed Patching

```
# Trigger SCCM client policy refresh
/opt/microsoft/configmgr/bin/ccmexec -rs policy
# Check available deployments
```

```

/opt/microsoft/configmgr/bin/ccmexec -rs deployment
# Force update scan
/opt/microsoft/configmgr/bin/ccmexec -rs scan
# View client log
tail -f /var/opt/microsoft/scxcm.log
# Check compliance status
cat /var/opt/microsoft/configmgr/inventory/compliance.xml
# Manual patch application (if SCCM deployment method is via
script)
# SCCM typically executes a script on Linux clients:
#!/bin/bash
# SCCM_Patch_Script.sh
yum update --security -y
EXITCODE=$?
if [ $EXITCODE -eq 0 ]; then
    echo "SUCCESS: Patches applied"
    needs-restarting -r
    if [ $? -eq 1 ]; then
        echo "REBOOT_REQUIRED"
    fi
else
    echo "FAILED: Exit code $EXITCODE"
fi
exit $EXITCODE

```

5.6 SCCM Compliance Reporting

```

# Generate compliance report from SCCM (PowerShell on SCCM server):
# Get-CMDeploymentStatus -DeploymentID "DEP001234" |
#   Where-Object {$_.StatusType -eq "Error"} |
#   Select-Object DeviceName, StatusDescription
# From Linux client - report installed patches:
rpm -qa --last | head -20
yum history info last
# Generate patch compliance data for SCCM
cat > /tmp/compliance_report.xml << 'EOF'
<?xml version="1.0" encoding="utf-8"?>
<ComplianceReport>
  <Hostname>$(hostname)</Hostname>
  <KernelVersion>$(uname -r)</KernelVersion>
  <LastPatched>$(rpm -qa --last | head -1 | awk '{print
$2,$3,$4}')

```

5.7 SCCM Rollback Procedure

```

# If patch deployment fails, rollback from Linux host:
# 1. Check last yum transaction
yum history list | head -5

```

```
# 2. Identify the transaction ID for the patch
yum history info <transaction_id>
# 3. Rollback/undo the transaction
yum history undo <transaction_id> -y
# 4. Verify rollback
rpm -q <package_name>
# Should show previous version
# 5. Report rollback status to SCCM
/opt/microsoft/configmgr/bin/ccmexec -rs status
# 6. Update SCCM compliance scan
/opt/microsoft/configmgr/bin/ccmexec -rs scan
```

5.8 SCCM vs Satellite Comparison

Feature	SCCM/MECM	Red Hat Satellite	----- ----- -----	Native Linux Support	Limited
Full	Package Management	Via scripts	Native (yum/dnf)	Content Views	No Yes
No Yes	Errata Management	Basic	Full	Compliance Reporting	Good Excellent
Limited	Native	RBAC	AD-based	Granular	Ansible Integration
RHEL subscription				Scale (Linux)	Good Excellent
				Cost	Per-server license

·**EXPERT TIP:** In mixed Windows/Linux environments, use SCCM for compliance visibility and reporting, but deploy actual patches using Satellite or Ansible for better Linux-native control. Feed results back to SCCM for unified dashboards.

• **LESSON LEARNED:** The SCCM Linux client (OMI-based) has had security vulnerabilities in the past (OMIGOD - CVE-2021-38647). Ensure you keep the SCCM Linux client itself patched and monitor Microsoft security advisories.

Section 6 · Patching Using Ansible

6.1 Ansible Setup & Installation (START HERE)

CRITICAL: WHERE DOES ANSIBLE RUN FROM? ...

```
.
.
.   ANSIBLE RUNS FROM A SINGLE CONTROL NODE (MASTER NODE)
.
.
.   . Ansible is ONLY installed on the CONTROL NODE
.
.   . The control node pushes commands to managed nodes via SSH
.   . Managed nodes (target servers) do NOT need Ansible installed
.   . The control node MUST be a Linux/Unix machine (NOT Windows)
```

- Example Control Node: ansible-master.company.com
 - Example Control Node: 10.0.0.100 (your Ansible server)
 -
 -
 - ALL ansible-playbook and ansible commands below run on this machine!
 -
 -
-

6.1.1 Installing Ansible on the Control Node

IMPORTANT: Install Ansible ONLY on the control node. Do NOT install Ansible on the 2000 managed (target) servers.

[Run on: Ansible Control Node - e.g., ansible-master.company.com]

```
# .....
# RHEL 7 / CentOS 7
# .....
sudo yum install epel-release && sudo yum install ansible
# .....
# RHEL 8
# .....
sudo dnf install ansible-core
# .....
# RHEL 9 / RHEL 10
# .....
sudo dnf install ansible-core
# .....
# Ubuntu 22.04 / Ubuntu 24.04
# .....
sudo apt update && sudo apt install ansible
# .....
# CentOS 7
# .....
sudo yum install epel-release && sudo yum install ansible
```

Verify Installation:

[Run on: Ansible Control Node - e.g., ansible-master.company.com]

```
ansible --version
# Expected Output:
# ansible [core 2.15.x]
#   config file = /etc/ansible/ansible.cfg
#   configured module search path =
#   ansible python module location =
#   /usr/lib/python3.x/site-packages/ansible
#   python version = 3.x.x
```

6.1.2 Ansible Directory Structure & File Paths

BEGINNER NOTE: All these directories and files exist ONLY on the Ansible Control Node.
Create them on your Ansible master server.

[Run on: Ansible Control Node - e.g., ansible-master.company.com]

Ansible Control Node Directory Structure:

```
.....
/etc/ansible/                                · Default Ansible config
directory
... ansible.cfg                             · Main Ansible configuration
file
... hosts                                  · Default inventory file (basic)
... roles/                                · System-wide roles (optional)
/opt/ansible/                                · Enterprise custom directory
(RECOMMENDED)
... ansible.cfg                             · Project-specific config
(overrides /etc/ansible/)
... inventory/                             · All inventory files
.   ... production/
.   .   ... hosts.ini                     · Production static inventory
.   .   ... satellite_inventory.yml        · Dynamic inventory (Satellite)
.   .   ... aws_ec2.yml                   · Dynamic inventory (AWS)
.   ... uat/
.   .   ... hosts.ini                     · UAT inventory
.   ... dev/
.   .   ... hosts.ini                     · Development inventory
.   ... group_vars/                       · Variables applied to groups
.   .   ... all.yml                       · Variables for ALL hosts
.   .   ... prod.yml                      · Production-specific vars
.   .   ... webserver.yml                 · Webserver group vars
.   .   ... dbserver.yml                 · Database group vars
.   .   ... batch1_canary.yml             · Batch 1 specific vars
.   ... host_vars/                        · Variables for specific hosts
.   .   ... webserver01.corp.com.yml
.   .   ... dbserver01.corp.com.yml
... playbooks/                             · All playbook files
.   ... patching/                         · Patching-specific playbooks
.   .   ... patch_single_server.yml
.   .   ... patch_multiple_servers.yml
.   .   ... patch_enterprise_batches.yml
.   .   ... patch_rollback.yml
.   .   ... patch_validate.yml
.   .   ... tasks/                        · Shared task files
.   .   .   ... pre_patch_checks.yml
.   .   .   ... post_patch_checks.yml
.   .   .   ... app_health_check.yml
.   .   .   ... take_snapshot.yml
... roles/                                 · Custom roles
.   ... linux_patching/
.   .   ... defaults/main.yml
```

·	... tasks/main.yml	
·	... handlers/main.yml	
·	... templates/	
·	... vars/	
·	... meta/main.yml	
...	collections/	· Ansible collections
...	vault/	· Encrypted secrets
·	... credentials.yml	
...	logs/	· Ansible execution logs
	... ansible.log	

Exact File Paths Summary Table:

Purpose	Path on Control Node	----- -----	Ansible config	/etc/ansible/ansible.cfg (default) or /opt/ansible/ansible.cfg (custom)
			Main inventory	/etc/ansible/hosts OR /opt/ansible/inventory/
			Playbooks directory	/opt/ansible/playbooks/patching/
			Roles directory	/opt/ansible/roles/
			Group vars	/opt/ansible/inventory/group_vars/
			Host vars	/opt/ansible/inventory/host_vars/
			Vault secrets	/opt/ansible/vault/
			Logs	/opt/ansible/logs/

6.1.3 Create the Directory Structure

[Run on: Ansible Control Node - e.g., ansible-master.company.com]

```
# Create the enterprise Ansible directory structure
sudo mkdir -p

/opt/ansible/{inventory/{production,uat,dev,group_vars,host_vars},playbooks/patching/}
# Set ownership to ansible service account
sudo chown -R ansible_svc:ansible_svc /opt/ansible
sudo chmod -R 750 /opt/ansible
sudo chmod 700 /opt/ansible/vault
# Verify structure
tree /opt/ansible/
# Or if tree not installed:
find /opt/ansible -type d | sort
```

6.1.4 SSH Key Setup (Control Node · Managed Nodes)

·KEY CONCEPT: Ansible connects to managed nodes (your 2000 servers) via SSH. You must set up SSH key-based authentication from the control node to every managed node.

Step 1: Generate SSH Key Pair on Control Node

[Run on: Ansible Control Node - e.g., ansible-master.company.com]

```
# Generate SSH key pair for the ansible service account
sudo -u ansible_svc ssh-keygen -t ed25519 -C "ansible@control-node"
-f /home/ansible_svc/.ssh/id_ed25519 -N ""
# Or RSA if ed25519 not supported on older systems:
sudo -u ansible_svc ssh-keygen -t rsa -b 4096 -C
"ansible@control-node" -f /home/ansible_svc/.ssh/id_rsa -N ""
# View the public key (you'll copy this to managed nodes)
```

```
cat /home/ansible_svc/.ssh/id_ed25519.pub
```

Step 2: Distribute Public Key to All Managed Nodes

[Run on: Ansible Control Node - e.g., ansible-master.company.com]

```
# Method 1: ssh-copy-id (one host at a time - good for initial
setup)
ssh-copy-id -i /home/ansible_svc/.ssh/id_ed25519.pub
ansible_svc@webserver01.corp.com
# Method 2: Using a loop for multiple hosts
for host in webserver0{1..50}.corp.com appserver0{1..80}.corp.com;
do
    ssh-copy-id -i /home/ansible_svc/.ssh/id_ed25519.pub
ansible_svc@${host}
done
# Method 3: Using Satellite/Puppet/Chef to deploy the key
(enterprise method)
# Push the public key content to
/home/ansible_svc/.ssh/authorized_keys on all nodes
# Method 4: Using Ansible itself (if you have password auth
initially)
ansible all -m authorized_key -a "user=ansible_svc key='$(cat
/home/ansible_svc/.ssh/id_ed25519.pub)'" --ask-pass -b
```

Step 3: Configure sudo on Managed Nodes (One-time Setup)

[Run on: Each Managed Node (target server) - or deploy via Satellite/Chef]

```
# Create sudoers entry for ansible service account
echo "ansible_svc ALL=(ALL) NOPASSWD: ALL" | sudo tee
/etc/sudoers.d/ansible_svc
sudo chmod 440 /etc/sudoers.d/ansible_svc
sudo visudo -c # Validate syntax
```

Step 4: Test SSH Connectivity

[Run on: Ansible Control Node - e.g., ansible-master.company.com]

```
# Test single host
ssh -i /home/ansible_svc/.ssh/id_ed25519
ansible_svc@webserver01.corp.com "hostname; uname -r"
# Test with Ansible ping module (tests SSH + Python on remote)
ansible all -m ping
# Expected output:
# webserver01.corp.com | SUCCESS => {
#     "changed": false,
#     "ping": "pong"
# }
# Test with privilege escalation
ansible all -m command -a "whoami" -b
# Expected: root
```

6.1.5 Ansible Configuration File

[Run on: Ansible Control Node - e.g., ansible-master.company.com]

Create /opt/ansible/ansible.cfg:

```
[defaults]
# Inventory
inventory = /opt/ansible/inventory/production/hosts.ini
# SSH settings
remote_user = ansible_svc
private_key_file = /home/ansible_svc/.ssh/id_ed25519
host_key_checking = False
# Performance
forks = 50
timeout = 30
gathering = smart
fact_caching = jsonfile
fact_caching_connection = /tmp/ansible_facts_cache
fact_caching_timeout = 3600
# Logging
log_path = /opt/ansible/logs/ansible.log
# Roles
roles_path = /opt/ansible/roles:/etc/ansible/roles
# Retry files
retry_files_enabled = True
retry_files_save_path = /opt/ansible/logs/
# Output
stdout_callback = yaml
callbacks_enabled = timer, profile_tasks
[privilege_escalation]
become = True
become_method = sudo
become_user = root
become_ask_pass = False
[ssh_connection]
pipelining = True
ssh_args = -o ControlMaster=auto -o ControlPersist=60s -o
StrictHostKeyChecking=no
```

6.1.6 Quick Start: Your First Patch Command

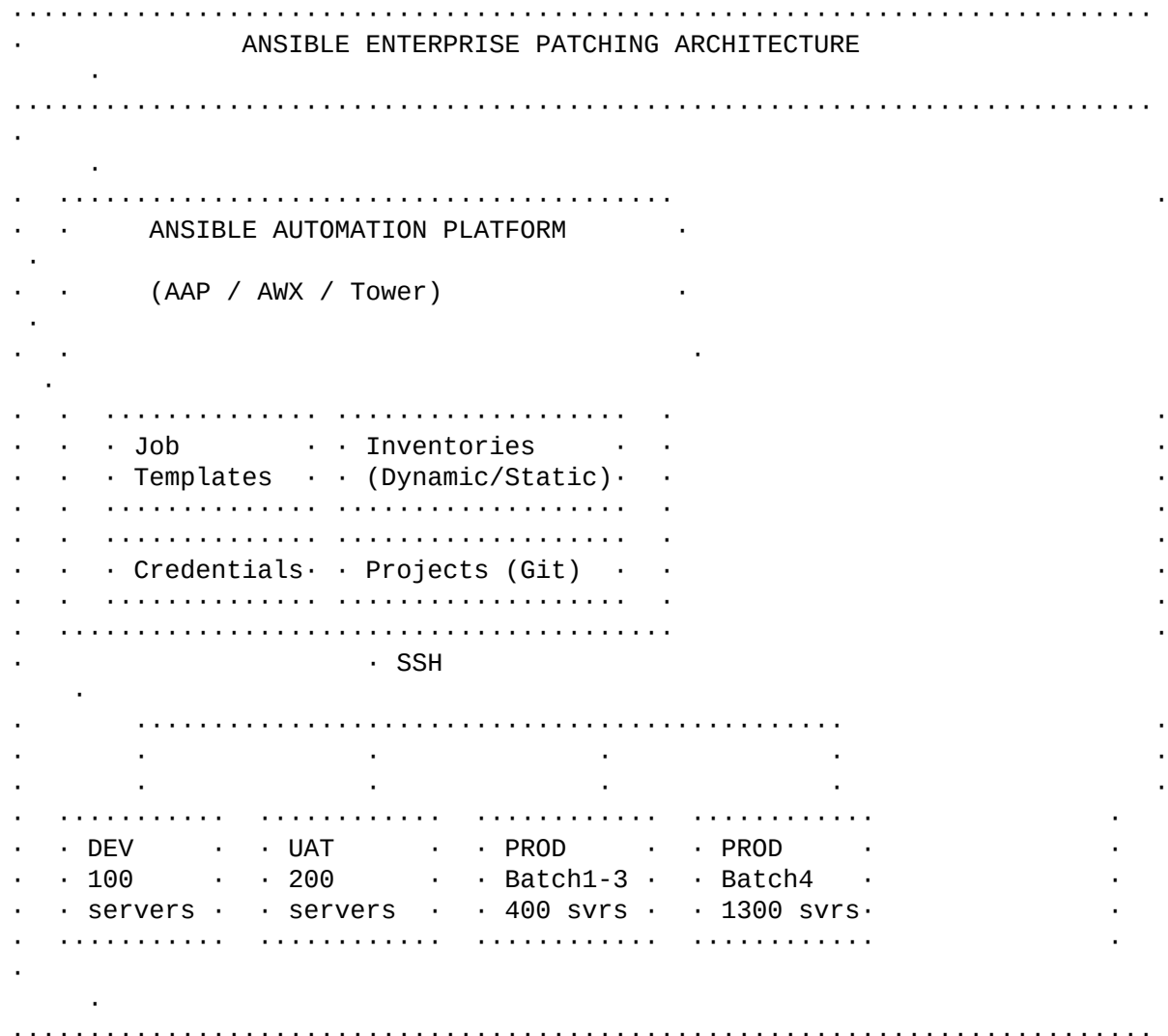
[Run on: Ansible Control Node - e.g., ansible-master.company.com]

```
# 1. Navigate to Ansible directory
cd /opt/ansible
# 2. Test connectivity to all hosts
ansible all -m ping
# 3. Check what updates are available (dry run - no changes)
ansible all -m shell -a "yum check-update 2>/dev/null | tail -20"
-b
# 4. Apply security patches to a SINGLE test server (start small!)
ansible webserver01.corp.com -m yum -a "name=* state=latest
security=yes" -b
# 5. Run a full patching playbook
ansible-playbook
```

```
/opt/ansible/playbooks/patching/patch_single_server.yml \
-e "target_host=webserver01.corp.com" \
-e "ticket=CHG-2026-12345"
```

• **COMMON MISTAKE:** New users try to run `ansible-playbook` on the managed (target) servers. Remember: ALL Ansible commands run ONLY on the control node. Ansible connects to targets over SSH automatically.

6.2 Ansible Architecture for Enterprise Patching



6.3 Inventory Structure

Static Inventory File

```
# inventory/production/hosts.ini
[all:vars]
```

```

ansible_user=ansible_svc
ansible_become=yes
ansible_become_method=sudo
# =====
# ENVIRONMENT GROUPS
# =====
[dev]
dev-web[01:10].corp.com
dev-app[01:20].corp.com
dev-db[01:05].corp.com
[uat]
uat-web[01:20].corp.com
uat-app[01:40].corp.com
uat-db[01:10].corp.com
[prod]
prod-web[01:50].corp.com
prod-app[01:80].corp.com
prod-db[01:40].corp.com
prod-util[01:30].corp.com
# =====
# APPLICATION TIER GROUPS
# =====
[webserver]
prod-web[01:50].corp.com
[appserver]
prod-app[01:80].corp.com
[dbserver]
prod-db[01:40].corp.com
[utility]
prod-util[01:30].corp.com
# =====
# BATCH GROUPS (for 2000 server patching)
# =====
[batch1_canary]
# 50 servers - non-critical, monitoring, utility
prod-util[01:30].corp.com
prod-web[01:10].corp.com
prod-app[01:10].corp.com
[batch2_early]
# 100 servers - low-risk application servers
prod-web[11:30].corp.com
prod-app[11:40].corp.com
prod-db[01:10].corp.com
prod-util[31:50].corp.com
[batch3_mainstream]
# 250 servers - standard servers
prod-web[31:50].corp.com
prod-app[41:80].corp.com
prod-db[11:30].corp.com
prod-util[51:100].corp.com
[batch4_remaining]
# 1600 servers - all remaining
# Defined dynamically or via separate inventory file
# Includes all hosts NOT in batch1, batch2, or batch3

```

```
# =====
# OS VERSION GROUPS
# =====
[rhel7]
legacy-app[01:50].corp.com
[rhel8]
prod-web[01:50].corp.com
prod-app[01:40].corp.com
[rhel9]
prod-app[41:80].corp.com
prod-db[01:40].corp.com
prod-util[01:100].corp.com
```

Dynamic Inventory (Satellite Integration)

```
# inventory/satellite_inventory.yml
plugin: redhat.satellite.foreman
url: https://satellite.corp.com
user: ansible_inventory
password: "{{ vault_satellite_password }}"
validate_certs: yes
groups:
  rhel8_prod: "'RHEL 8' in foreman_operatingsystem and 'Production'
in foreman_lifecycle_environment"
  rhel9_prod: "'RHEL 9' in foreman_operatingsystem and 'Production'
in foreman_lifecycle_environment"
  webserver: "'webserver' in foreman_hostgroup"
  dbserver: "'database' in foreman_hostgroup"
```

6.4 Group Variables

```
# group_vars/all.yml
---
patch_reboot_timeout: 600
patch_reboot_delay: 30
patch_pre_check: true
patch_post_check: true
patch_backup_configs: true
patch_notify_email: "linux-team@corp.com"
patch_change_ticket: "CHG-2026-12345"
ntp_server: "ntp.corp.com"
satellite_server: "satellite.corp.com"
monitoring_server: "nagios.corp.com"

# group_vars/prod.yml
---
patch_security_only: true
patch_exclude_packages:
  - kernel*
  - java*
patch_reboot_required: true
patch_maintenance_window: "Saturday 22:00-06:00 UTC"
```

```

patch_approval_required: true
patch_batch_size: 50

# group_vars/webserver.yml
---
app_service: httpd
app_health_url: "http://localhost/health"
app_expected_response: 200
app_graceful_restart: true
lb_pool_name: "web-prod-pool"
lb_drain_time: 30

```

6.5 Complete Enterprise Playbook · Single Server Patching

```

# playbooks/patch_single_server.yml
---
- name: Patch Single Linux Server
  hosts: "{{ target_host }}"
  become: yes
  gather_facts: yes
  serial: 1
  vars:
    patch_date: "{{ ansible_date_time.date }}"
    pre_patch_dir: "/root/pre_patch_{{ patch_date }}"
    change_ticket: "{{ ticket | default('CHG-UNKNOWN') }}"
  pre_tasks:
    - name: Display patch information
      debug:
        msg: |
          =====
          PATCHING: {{ inventory_hostname }}
          DATE: {{ patch_date }}
          TICKET: {{ change_ticket }}
          OS: {{ ansible_distribution }} {{
ansible_distribution_version }}
          KERNEL: {{ ansible_kernel }}
          =====
    - name: Create pre-patch backup directory
      file:
        path: "{{ pre_patch_dir }}"
        state: directory
        mode: '0700'
    - name: Backup critical configuration files
      copy:
        src: "{{ item }}"
        dest: "{{ pre_patch_dir }}/"
        remote_src: yes
      loop:
        - /etc/fstab
        - /etc/default/grub
        - /etc/hosts
        - /etc/resolv.conf
        - /etc/sysctl.conf

```

```

    ignore_errors: yes
- name: Record pre-patch kernel version
  shell: uname -r
  register: pre_kernel
  changed_when: false
- name: Record pre-patch running services
  shell: systemctl list-units --type=service --state=running
--no-pager | grep ".service" | awk '{print $1}'
  register: pre_services
  changed_when: false
- name: Save pre-patch service list
  copy:
    content: "{{ pre_services.stdout }}"
    dest: "{{ pre_patch_dir }}/running_services.txt"
- name: Check disk space on critical partitions
  shell: df -h / /boot /var | tail -n +2
  register: disk_space
  changed_when: false
- name: Fail if /boot has less than 100MB free
  shell: df /boot --output=avail | tail -1
  register: boot_free
  failed_when: boot_free.stdout | int < 102400
  changed_when: false
- name: Fail if / has less than 2GB free
  shell: df / --output=avail | tail -1
  register: root_free
  failed_when: root_free.stdout | int < 2097152
  changed_when: false
- name: Check repository connectivity
  command: yum repolist enabled -q
  register: repo_check
  changed_when: false
  failed_when: repo_check.rc != 0
- name: Record available updates before patching
  shell: yum check-update 2>/dev/null | grep -v
"^$\\|Loaded\\|repo" || true
  register: available_updates
  changed_when: false
- name: Save available updates list
  copy:
    content: "{{ available_updates.stdout }}"
    dest: "{{ pre_patch_dir }}/available_updates.txt"
tasks:
- name: Apply security patches only
  yum:
    name: '*'
    state: latest
    security: yes
    update_cache: yes
  register: patch_result
  when: patch_security_only | default(true)
- name: Apply all available patches
  yum:
    name: '*'

```

```

        state: latest
        update_cache: yes
        exclude: "{{ patch_exclude_packages | default([]) |
join(',') }}"
        register: patch_result_all
        when: not (patch_security_only | default(true))
    - name: Display patch results
      debug:
        msg: "Patches applied: {{ patch_result.results |
default(patch_result_all.results) | default([]) | length }}
packages updated"
    - name: Check if reboot is required
      command: needs-restarting -r
      register: reboot_needed
      failed_when: false
      changed_when: reboot_needed.rc == 1
    - name: Reboot server if required
      reboot:
        reboot_timeout: "{{ patch_reboot_timeout | default(600) }}"
        pre_reboot_delay: "{{ patch_reboot_delay | default(30) }}"
        post_reboot_delay: 60
        msg: "Reboot for patching - {{ change_ticket }}"
        when: reboot_needed.rc == 1
  post_tasks:
    - name: Verify new kernel (if updated)
      shell: uname -r
      register: post_kernel
      changed_when: false
    - name: Display kernel change
      debug:
        msg: "Kernel: {{ pre_kernel.stdout }} · {{ post_kernel.stdout
    }}"
    when: pre_kernel.stdout != post_kernel.stdout
    - name: Check for failed services
      shell: systemctl list-units --state=failed --no-pager
      register: failed_services
      changed_when: false
    - name: Fail if services are in failed state
      fail:
        msg: "ALERT: Failed services detected: {{
failed_services.stdout }}"
        when: failed_services.stdout_lines | length > 1
    - name: Verify all pre-patch services are running
      shell: |
        for svc in $(cat {{ pre_patch_dir }}/running_services.txt);
do
            if ! systemctl is-active --quiet $svc 2>/dev/null; then
                echo "NOT RUNNING: $svc"
            fi
        done
      register: service_check
      changed_when: false
    - name: Alert if services not running
      debug:

```

```

        msg: "WARNING: Some services not running after patch: {{
service_check.stdout }}"
    when: service_check.stdout | length > 0
    - name: Verify network connectivity
      wait_for:
        host: "{{ item }}"
        port: 443
        timeout: 10
      loop:
        - "{{ satellite_server }}"
        - "{{ monitoring_server }}"
      ignore_errors: yes
    - name: Generate patch report
      copy:
        content: |
          =====
          PATCH REPORT: {{ inventory_hostname }}
          DATE: {{ patch_date }}
          TICKET: {{ change_ticket }}
          =====
          Pre-Patch Kernel: {{ pre_kernel.stdout }}
          Post-Patch Kernel: {{ post_kernel.stdout }}
          Reboot Performed: {{ 'Yes' if reboot_needed.rc == 1 else
'No' }}
          Failed Services: {{ failed_services.stdout_lines | length
- 1 }}
          Status: {{ 'SUCCESS' if (failed_services.stdout_lines |
length <= 1) else 'NEEDS ATTENTION' }}
          =====
          dest: "{{ pre_patch_dir }}/patch_report.txt"

```

Usage:

[Run on: Ansible Control Node - e.g., ansible-master.company.com]

```

# Patch a single server
ansible-playbook playbooks/patch_single_server.yml \
-e "target_host=webserver01.corp.com" \
-e "ticket=CHG-2026-12345" \
-e "patch_security_only=true"

```

6.6 Complete Enterprise Playbook · Multiple Servers

```

# playbooks/patch_multiple_servers.yml
---
- name: Patch Multiple Linux Servers (Serial)
  hosts: "{{ target_group }}"
  become: yes
  gather_facts: yes
  serial: "{{ batch_size | default(10) }}"
  max_fail_percentage: 10
  vars:
    patch_date: "{{ ansible_date_time.date }}"
    change_ticket: "{{ ticket | default('CHG-UNKNOWN') }}"

```

```

pre_tasks:
  - name: Pre-patch health check
    include_tasks: tasks/pre_patch_checks.yml
  - name: Disable monitoring alerts
    uri:
      url: "http://{{ monitoring_server }}/api/v1/silence"
      method: POST
      body_format: json
      body:
        matchers:
          - name: instance
            value: "{{ inventory_hostname }}"
            startsAt: "{{ ansible_date_time.iso8601 }}"
            endsAt: "{{ (ansible_date_time.epoch | int + 7200) |
strftime('%Y-%m-%dT%H:%M:%S') }}"
            comment: "Patching - {{ change_ticket }}"
        delegate_to: localhost
        ignore_errors: yes
    tasks:
      - name: Apply patches
        yum:
          name: '*'
          state: latest
          security: "{{ patch_security_only | default(true) }}"
          exclude: "{{ patch_exclude_packages | default([]) |
join(',') }}"
          register: patch_result
      - name: Check if reboot required
        command: needs-restarting -r
        register: reboot_check
        failed_when: false
        changed_when: false
      - name: Reboot if needed
        reboot:
          reboot_timeout: 600
          pre_reboot_delay: 10
          post_reboot_delay: 60
          when: reboot_check.rc == 1 and (patch_reboot_required |
default(true))
    post_tasks:
      - name: Post-patch validation
        include_tasks: tasks/post_patch_checks.yml
      - name: Re-enable monitoring
        uri:
          url: "http://{{ monitoring_server }}/api/v1/silence/{{
inventory_hostname }}"
          method: DELETE
          delegate_to: localhost
          ignore_errors: yes
    handlers:
      - name: restart application
        service:
          name: "{{ app_service }}"
          state: restarted

```

when: app_service is defined

Usage:

[Run on: Ansible Control Node - e.g., ansible-master.company.com]

```
# Patch all webserver (10 at a time)
ansible-playbook playbooks/patch_multiple_servers.yml \
  -e "target_group=webserver" \
  -e "batch_size=10" \
  -e "ticket=CHG-2026-12345"
# Patch UAT environment (20 at a time)
ansible-playbook playbooks/patch_multiple_servers.yml \
  -e "target_group=uat" \
  -e "batch_size=20" \
  -e "ticket=CHG-2026-12345"
```

6.7 Enterprise Playbook · 2000 Servers in Batches

```
# playbooks/patch_enterprise_batches.yml
---
# =====
# ENTERPRISE BATCH PATCHING - 2000 SERVERS
# Batch 1: 50 servers (Canary)
# Batch 2: 100 servers (Early Adopters)
# Batch 3: 250 servers (Mainstream)
# Batch 4: 1600 servers (Remaining - parallel 50 at a time)
# =====
# ---- BATCH 1: CANARY (50 servers) ----
- name: "BATCH 1 - Canary Group (50 servers)"
  hosts: batch1_canary
  become: yes
  gather_facts: yes
  serial: 10
  max_fail_percentage: 0
  vars:
    batch_name: "Batch1-Canary"
    change_ticket: "{{ ticket }}"
    patch_security_only: true

  pre_tasks:
    - name: "{{ batch_name }}" - Pre-patch validation
      include_tasks: tasks/pre_patch_checks.yml
    - name: "{{ batch_name }}" - Take VM snapshot
      include_tasks: tasks/take_snapshot.yml
      when: is_virtual | default(true)

  tasks:
    - name: "{{ batch_name }}" - Apply security patches
      yum:
        name: '*'
        state: latest
        security: yes
        register: patch_result
    - name: "{{ batch_name }}" - Check reboot requirement"
```

```

    command: needs-restarting -r
    register: reboot_check
    failed_when: false
    changed_when: false
- name: "{{ batch_name }}" - Reboot server"
  reboot:
    reboot_timeout: 600
    pre_reboot_delay: 10
    post_reboot_delay: 60
  when: reboot_check.rc == 1
post_tasks:
- name: "{{ batch_name }}" - Post-patch validation"
  include_tasks: tasks/post_patch_checks.yml
- name: "{{ batch_name }}" - Application health check"
  include_tasks: tasks/app_health_check.yml
- name: "BATCH 1 - Wait and Validate (2 hours monitoring)"
  hosts: localhost
  gather_facts: no
  tasks:
    - name: Pause for monitoring observation (2 hours)
      pause:
        prompt: |
          .....
          .  BATCH 1 COMPLETE - 50 servers patched  .
          .
          .
          .  ACTION REQUIRED:
          .
          .  1. Check monitoring dashboards for anomalies
          .
          .  2. Verify application health checks
          .
          .  3. Review logs for errors
          .
          .  4. Confirm with application teams
          .
          .
          .  Press ENTER to proceed to Batch 2
          .
          .  Press Ctrl+C then 'A' to abort
          .....
      when: manual_approval | default(true)
    - name: Automated wait (if no manual approval)
      wait_for:
        timeout: 7200
      when: not (manual_approval | default(true))
# ---- BATCH 2: EARLY ADOPTERS (100 servers) ----
- name: "BATCH 2 - Early Adopters (100 servers)"
  hosts: batch2_early
  become: yes
  gather_facts: yes
  serial: 20
  max_fail_percentage: 5

```

```

vars:
  batch_name: "Batch2-EarlyAdopters"
  change_ticket: "{{ ticket }}"
  patch_security_only: true
pre_tasks:
  - name: "{{ batch_name }}" - Pre-patch validation"
    include_tasks: tasks/pre_patch_checks.yml
tasks:
  - name: "{{ batch_name }}" - Apply security patches"
    yum:
      name: '*'
      state: latest
      security: yes
      register: patch_result
  - name: "{{ batch_name }}" - Reboot if required"
    reboot:
      reboot_timeout: 600
    when: patch_result.changed
post_tasks:
  - name: "{{ batch_name }}" - Post-patch validation"
    include_tasks: tasks/post_patch_checks.yml
- name: "BATCH 2 - Wait and Validate (2 hours)"
  hosts: localhost
  gather_facts: no
  tasks:
    - name: Pause for Batch 2 validation
      pause:
        prompt: |
          .....
          .  BATCH 2 COMPLETE - 100 servers patched  .
          .  Press ENTER to proceed to Batch 3      .
          .  Press Ctrl+C then 'A' to abort          .
          .....

      when: manual_approval | default(true)
# ---- BATCH 3: MAINSTREAM (250 servers) ----
- name: "BATCH 3 - Mainstream (250 servers)"
  hosts: batch3_mainstream
  become: yes
  gather_facts: yes
  serial: 50
  max_fail_percentage: 5
  vars:
    batch_name: "Batch3-Mainstream"
    change_ticket: "{{ ticket }}"
    patch_security_only: true
  pre_tasks:
    - name: "{{ batch_name }}" - Pre-patch validation"
      include_tasks: tasks/pre_patch_checks.yml
  tasks:
    - name: "{{ batch_name }}" - Apply security patches"
      yum:
        name: '*'
        state: latest
        security: yes

```

```

        register: patch_result
    - name: "{{ batch_name }}" - Reboot if required"
      reboot:
        reboot_timeout: 600
      when: patch_result.changed
  post_tasks:
    - name: "{{ batch_name }}" - Post-patch validation"
      include_tasks: tasks/post_patch_checks.yml
- name: "BATCH 3 - Wait and Validate (1 hour)"
  hosts: localhost
  gather_facts: no
  tasks:
    - name: Pause for Batch 3 validation
      pause:
        prompt: |
          .....
          .   BATCH 3 COMPLETE - 250 servers patched   .
          .   Press ENTER to proceed to Batch 4 (FINAL) .
          .   Press Ctrl+C then 'A' to abort           .
          .....

      when: manual_approval | default(true)
# ---- BATCH 4: REMAINING (1600 servers) ----
- name: "BATCH 4 - Remaining Fleet (1600 servers)"
  hosts: batch4_remaining
  become: yes
  gather_facts: yes
  serial: 50
  max_fail_percentage: 10
  vars:
    batch_name: "Batch4-Remaining"
    change_ticket: "{{ ticket }}"
    patch_security_only: true
  pre_tasks:
    - name: "{{ batch_name }}" - Pre-patch validation"
      include_tasks: tasks/pre_patch_checks.yml
  tasks:
    - name: "{{ batch_name }}" - Apply security patches"
      yum:
        name: '*'
        state: latest
        security: yes
        register: patch_result
    - name: "{{ batch_name }}" - Reboot if required"
      reboot:
        reboot_timeout: 600
      when: patch_result.changed
  post_tasks:
    - name: "{{ batch_name }}" - Post-patch validation"
      include_tasks: tasks/post_patch_checks.yml
# ---- FINAL REPORT ----
- name: "Generate Final Patch Report"
  hosts: all
  become: yes
  gather_facts: yes

```

```

tasks:
  - name: Collect final kernel version
    command: uname -r
    register: final_kernel
    changed_when: false
  - name: Generate server report line
    set_fact:
      report_line: "{{ inventory_hostname }},{{
final_kernel.stdout }},{{ ansible_distribution_version }},SUCCESS"
  - name: Write consolidated report
    lineinfile:
      path: "/tmp/patch_report_{{ ansible_date_time.date }}.csv"
      line: "{{ report_line }}"
      create: yes
      delegate_to: localhost

```

Usage:

[Run on: Ansible Control Node - e.g., ansible-master.company.com]

```

# Execute full batch patching with manual approval between batches
ansible-playbook playbooks/patch_enterprise_batches.yml \
  -e "ticket=CHG-2026-12345" \
  -e "manual_approval=true"
# Execute with automated waits (no manual intervention)
ansible-playbook playbooks/patch_enterprise_batches.yml \
  -e "ticket=CHG-2026-12345" \
  -e "manual_approval=false"

```

6.8 Shared Task Files

tasks/pre_patch_checks.yml

```

---
- name: Check disk space on /
  shell: df / --output=pcent | tail -1 | tr -d ' %'
  register: root_usage
  changed_when: false
  failed_when: root_usage.stdout | int > 85
- name: Check disk space on /boot
  shell: df /boot --output=avail | tail -1
  register: boot_avail
  changed_when: false
  failed_when: boot_avail.stdout | int < 102400
- name: Check disk space on /var
  shell: df /var --output=avail | tail -1
  register: var_avail
  changed_when: false
  failed_when: var_avail.stdout | int < 2097152
- name: Check inode usage
  shell: df -i / --output=ipcent | tail -1 | tr -d ' %'
  register: inode_usage
  changed_when: false

```

```

    failed_when: inode_usage.stdout | int > 90
- name: Check repository connectivity
  command: yum repolist enabled -q
  register: repo_status
  changed_when: false
  failed_when: repo_status.rc != 0
- name: Verify subscription status
  command: subscription-manager status
  register: sub_status
  changed_when: false
  failed_when: "'Overall Status: Current' not in sub_status.stdout"
  ignore_errors: yes
- name: Check for read-only filesystems
  shell: mount | grep -w "ro," | grep -v "iso9660\squashfs" ||
true
  register: ro_check
  changed_when: false
  failed_when: ro_check.stdout | length > 0
- name: Record pre-patch state
  shell: |
    echo "HOSTNAME: $(hostname)"
    echo "KERNEL: $(uname -r)"
    echo "UPTIME: $(uptime)"
    echo "SERVICES_RUNNING: $(systemctl list-units --state=running
--type=service --no-pager | grep -c '.service')"
    echo "DISK_ROOT: $(df -h / | tail -1)"
  register: pre_state
  changed_when: false

```

tasks/post_patch_checks.yml

```

---
- name: Check for failed services
  shell: systemctl list-units --state=failed --no-pager | grep -c
"failed" || echo "0"
  register: failed_count
  changed_when: false
- name: Alert on failed services
  debug:
    msg: "WARNING: {{ failed_count.stdout }} failed services on {{
inventory_hostname }}"
  when: failed_count.stdout | int > 0
- name: Verify SSH is running
  wait_for:
    port: 22
    host: "{{ inventory_hostname }}"
    timeout: 30
  delegate_to: localhost
- name: Verify network connectivity
  command: ping -c 2 {{ ansible_default_ipv4.gateway }}
  changed_when: false
  ignore_errors: yes
- name: Check system log for critical errors

```

```

    shell: journalctl -p crit --since "10 minutes ago" --no-pager |
head -20
    register: crit_logs
    changed_when: false
- name: Verify kernel booted successfully
  command: uname -r
  register: current_kernel
  changed_when: false
- name: Check uptime (should be low if rebooted)
  command: uptime
  register: uptime_check
  changed_when: false

```

tasks/app_health_check.yml

```

---
- name: Check application health endpoint
  uri:
    url: "{{ app_health_url }}"
    method: GET
    status_code: "{{ app_expected_response | default(200) }}"
    timeout: 30
  register: health_result
  when: app_health_url is defined
  retries: 3
  delay: 10
  ignore_errors: yes
- name: Check application service status
  service_facts:
- name: Verify application service is running
  assert:
    that: "ansible_facts.services['{{ app_service
}}.service'].state == 'running'"
    fail_msg: "Service {{ app_service }} is NOT running!"
    success_msg: "Service {{ app_service }} is running"
  when: app_service is defined
  ignore_errors: yes

```

6.9 Rollback Playbook

```

# playbooks/patch_rollback.yml
---
- name: Rollback Patches on Linux Servers
  hosts: "{{ target_host | default(target_group) }}"
  become: yes
  gather_facts: yes
  serial: 1
  vars:
    change_ticket: "{{ ticket | default('CHG-UNKNOWN') }}"
  tasks:
    - name: Display rollback information
      debug:

```

```

msg: |
.....
.   ROLLBACK INITIATED
.   Host: {{ inventory_hostname }}
.   Ticket: {{ change_ticket }}
.   Time: {{ ansible_date_time.iso8601 }}
.....
- name: Get last yum transaction ID
  shell: yum history list | awk 'NR==4{print $1}'
  register: last_transaction
  changed_when: false
- name: Display transaction to rollback
  shell: yum history info {{ last_transaction.stdout }}
  register: transaction_info
  changed_when: false
- name: Show transaction details
  debug:
    var: transaction_info.stdout_lines
- name: Confirm rollback (manual mode)
  pause:
    prompt: "About to rollback transaction {{
last_transaction.stdout }}. Press ENTER to continue or Ctrl+C to
abort."
    when: manual_confirm | default(true)
- name: Perform yum history undo
  command: yum history undo {{ last_transaction.stdout }} -y
  register: rollback_result
- name: Display rollback result
  debug:
    var: rollback_result.stdout_lines
- name: Check if reboot needed after rollback
  command: needs-restarting -r
  register: reboot_after_rollback
  failed_when: false
  changed_when: false
- name: Reboot after rollback (if kernel was rolled back)
  reboot:
    reboot_timeout: 600
    pre_reboot_delay: 10
    post_reboot_delay: 60
    msg: "Rollback reboot - {{ change_ticket }}"
  when: reboot_after_rollback.rc == 1
- name: Post-rollback validation
  include_tasks: tasks/post_patch_checks.yml
- name: Verify rolled-back kernel
  command: uname -r
  register: rollback_kernel
  changed_when: false
- name: Generate rollback report
  copy:
    content: |
      ROLLBACK REPORT
      =====
      Host: {{ inventory_hostname }}

```

```

Ticket: {{ change_ticket }}
Transaction Rolled Back: {{ last_transaction.stdout }}
Current Kernel: {{ rollback_kernel.stdout }}
Status: ROLLBACK COMPLETE
Time: {{ ansible_date_time.iso8601 }}
dest: "/root/rollback_report_{{ ansible_date_time.date
}}.txt"

```

Usage:

[Run on: Ansible Control Node - e.g., ansible-master.company.com]

```

# Rollback single server
ansible-playbook playbooks/patch_rollback.yml \
  -e "target_host=webserver01.corp.com" \
  -e "ticket=CHG-2026-12345"
# Rollback a group with manual confirmation
ansible-playbook playbooks/patch_rollback.yml \
  -e "target_group=batch1_canary" \
  -e "ticket=CHG-2026-12345" \
  -e "manual_confirm=true"
# Rollback specific transaction ID
ansible-playbook playbooks/patch_rollback.yml \
  -e "target_host=webserver01.corp.com" \
  -e "transaction_id=45"

```

6.10 Validation Playbook

```

# playbooks/patch_validate.yml
---
- name: Validate Patch Status Across Fleet
  hosts: "{{ target_group | default('all') }}"
  become: yes
  gather_facts: yes
  serial: 100
  tasks:
    - name: Get current kernel version
      command: uname -r
      register: kernel_version
      changed_when: false
    - name: Get OS version
      command: cat /etc/redhat-release
      register: os_version
      changed_when: false
    - name: Check for outstanding security patches
      shell: yum updateinfo list security 2>/dev/null | grep -c
"RHSA" || echo "0"
      register: pending_security
      changed_when: false
    - name: Check uptime
      shell: uptime -s
      register: last_boot
      changed_when: false
    - name: Check failed services

```

```

    shell: systemctl list-units --state=failed --no-pager | grep
-c "failed" || echo "0"
    register: failed_svcs
    changed_when: false
  - name: Check subscription status
    shell: subscription-manager status 2>/dev/null | grep
"Overall Status" | awk -F': ' '{print $2}'
    register: sub_status
    changed_when: false
    ignore_errors: yes
  - name: Build validation report
    set_fact:
      validation_line: "{{ inventory_hostname }},{{
kernel_version.stdout }},{{ os_version.stdout }},{{
pending_security.stdout }},{{ last_boot.stdout }},{{
failed_svcs.stdout }},{{ sub_status.stdout | default('N/A') }}"
  - name: Write to consolidated report
    lineinfile:
      path: "/tmp/patch_validation_{{ ansible_date_time.date
}}.csv"
      line: "{{ validation_line }}"
      create: yes
      delegate_to: localhost
  - name: Add CSV header
    lineinfile:
      path: "/tmp/patch_validation_{{ ansible_date_time.date
}}.csv"
      line: "Hostname,Kernel,OS Version,Pending Security,Last
Boot,Failed Services,Subscription"
      insertbefore: BOF
      delegate_to: localhost
      run_once: yes
  - name: Flag non-compliant hosts
    debug:
      msg: ".. NON-COMPLIANT: {{ inventory_hostname }} has {{
pending_security.stdout }} outstanding security patches"
      when: pending_security.stdout | int > 0
  - name: Flag hosts with failed services
    debug:
      msg: ".. ALERT: {{ inventory_hostname }} has {{
failed_svcs.stdout }} failed services"
      when: failed_svcs.stdout | int > 0

```

Usage:

[Run on: Ansible Control Node - e.g., ansible-master.company.com]

```

# Validate all production servers
ansible-playbook playbooks/patch_validate.yml \
  -e "target_group=prod"
# Validate specific batch
ansible-playbook playbooks/patch_validate.yml \
  -e "target_group=batch1_canary"
# Report output at: /tmp/patch_validation_YYYY-MM-DD.csv

```

6.11 Ansible Roles Structure

```
roles/  
... linux_patching/  
.  ... defaults/  
.  .  ... main.yml  
.  ... tasks/  
.  .  ... main.yml  
.  .  ... pre_checks.yml  
.  .  ... patch_rhel7.yml  
.  .  ... patch_rhel8.yml  
.  .  ... patch_rhel9.yml  
.  .  ... reboot.yml  
.  .  ... post_checks.yml  
.  .  ... rollback.yml  
.  ... handlers/  
.  .  ... main.yml  
.  ... templates/  
.  .  ... patch_report.j2  
.  .  ... email_notification.j2  
.  ... vars/  
.  .  ... RedHat-7.yml  
.  .  ... RedHat-8.yml  
.  .  ... RedHat-9.yml  
.  ... meta/  
.  ... main.yml
```

roles/linux_patching/defaults/main.yml

```
---  
patch_security_only: true  
patch_reboot_enabled: true  
patch_reboot_timeout: 600  
patch_exclude_packages: []  
patch_pre_check_enabled: true  
patch_post_check_enabled: true  
patch_snapshot_enabled: true  
patch_notification_enabled: true  
patch_rollback_on_failure: true
```

roles/linux_patching/tasks/main.yml

```
---  
- name: Include OS-specific variables  
  include_vars: "{{ ansible_distribution }}-{{  
ansible_distribution_major_version }}.yaml"  
- name: Run pre-patch checks  
  include_tasks: pre_checks.yml  
  when: patch_pre_check_enabled  
- name: Apply patches (RHEL 7)  
  include_tasks: patch_rhel7.yml  
  when: ansible_distribution_major_version == "7"  
- name: Apply patches (RHEL 8)
```

```

    include_tasks: patch_rhel8.yml
    when: ansible_distribution_major_version == "8"
- name: Apply patches (RHEL 9)
    include_tasks: patch_rhel9.yml
    when: ansible_distribution_major_version == "9"
- name: Handle reboot
    include_tasks: reboot.yml
    when: patch_reboot_enabled
- name: Run post-patch checks
    include_tasks: post_checks.yml
    when: patch_post_check_enabled

```

·**EXPERT TIP:** Use Ansible Tower/AAP workflow templates to chain the batch playbooks together with approval gates. This allows authorized managers to approve progression from one batch to the next via the web interface.

·**COMMON MISTAKE:** Using `serial: 1` for 2000 servers means patching one at a time, which could take days. Use appropriate serial values (10-50) based on your validation needs. Start conservative and increase as confidence grows.

·**BEGINNER NOTE:** Always test your playbooks on a single DEV server first with `--check` (dry run) mode before running against production.

Section 7 · Patching Using Chef

·WHERE DO CHEF COMMANDS RUN FROM?

```

.....
.
.
.
.   CHEF HAS THREE COMPONENTS - KNOW WHERE EACH COMMAND
RUNS:
.
.
.
.   1. CHEF SERVER (e.g., chef-server.company.com)
.
.   . Stores cookbooks, roles, environments, node data
.
.   . You typically do NOT run commands directly here
.
.   . Install: chef-server-core package
.
.
.
.   2. CHEF WORKSTATION (e.g., your admin laptop/jump
host)
.
.   . Where you run knife commands from
.
.   . Where you develop and upload cookbooks
.
.   . Install: chef-workstation package
.

```

- 3. CHEF CLIENT (managed nodes - your 2000 servers)
 - Runs chef-client to pull and apply cookbooks
 - Install: chef-client package on EVERY managed node
-

Chef Installation & Setup

Chef Server Installation:

[Run on: Chef Server - e.g., chef-server.company.com]

```
# Requirements: RHEL 7/8/9, 4+ CPU, 8+ GB RAM, 50+ GB disk
# Download and install Chef Server
sudo rpm -Uvh chef-server-core-<version>.el8.x86_64.rpm
# Configure Chef Server
sudo chef-server-ctl reconfigure
# Create admin user and organization
sudo chef-server-ctl user-create admin Admin User admin@company.com
'password' --filename /etc/chef/admin.pem
sudo chef-server-ctl org-create myorg "My Organization"
--association_user admin --filename /etc/chef/myorg-validator.pem
# Verify
sudo chef-server-ctl status
```

Chef Workstation Installation:

[Run on: Chef Workstation - e.g., your admin machine/jump host]

```
# RHEL/CentOS:
sudo rpm -Uvh chef-workstation-<version>.el8.x86_64.rpm
# Ubuntu:
sudo dpkg -i chef-workstation_<version>_amd64.deb
# Verify
chef --version
knife --version
```

Chef Client Installation (on managed nodes):

[Run on: Each Managed Node (target server)]

```
# Bootstrap from Workstation (installs client automatically):
knife bootstrap <node_fqdn> -U root -P 'password' --node-name
<node_name>
# Or install manually on the node:
curl -L https://omnitruck.chef.io/install.sh | sudo bash -s -- -v
<version>
```

Chef File Paths Reference

Purpose	Path	Located On
Chef Server config	/etc/opscode/chef-server.rb	Chef Server
Server data	/var/opt/opscode/	Chef Server
Workstation config	~/.chef/config.rb (or ~/.chef/knife.rb)	Chef Workstation
Workstation credentials	~/.chef/*.pem	Chef Workstation
Cookbooks (development)	~/chef-repo/cookbooks/	Chef Workstation
Roles (development)	~/chef-repo/roles/	Chef Workstation
Data bags	~/chef-repo/data_bags/	Chef Workstation
Environments	~/chef-repo/environments/	Chef Workstation
Client config	/etc/chef/client.rb	Managed Nodes
Client key	/etc/chef/client.pem	Managed Nodes
Validation key	/etc/chef/validation.pem	Managed Nodes
Ohai plugins	/etc/chef/ohai/plugins/	Managed Nodes
Chef cache	/var/chef/cache/	Managed Nodes
Chef backup	/var/chef/backup/	Managed Nodes

Chef Workstation Directory Structure

[Run on: Chef Workstation]

~/chef-repo/ workstation)	• Your Chef repository (on workstation)
... .chef/	
• ... config.rb	• Knife configuration
• ... admin.pem	• Admin private key
• ... myorg-validator.pem	• Org validator key
... cookbooks/	
• ... linux_patching/	• Patching cookbook
• metadata.rb	
• attributes/default.rb	
• recipes/	
• default.rb	
• pre_check.rb	
• patch_security.rb	
• reboot.rb	
• post_check.rb	
• rollback.rb	
• templates/	
• files/	
• spec/	
• ... baseline/	
• ... hardening/	
... roles/	
• ... webserver.json	
• ... dbserver.json	
• ... patching.json	
... environments/	
• ... development.json	
• ... uat.json	
• ... production.json	
... data_bags/	
• ... credentials/	
• ... patch_config/	
... Policyfile.rb	

7.1 Chef Architecture for Patching


```

.    ... patch_report.erb
.    ... pre_patch_script.sh.erb
... libraries/
.    ... patch_helpers.rb
... files/
.    ... post_patch_validate.sh
... spec/
    ... unit/
        ... recipes/
            ... default_spec.rb

```

7.3 Cookbook Code

metadata.rb

```

name 'linux_patching'
maintainer 'Linux Infrastructure Team'
maintainer_email 'linux-team@corp.com'
license 'Internal Use Only'
description 'Enterprise Linux Patching Cookbook'
version '3.0.0'
supports 'redhat', '>= 7.0'
supports 'centos', '>= 7.0'
supports 'oracle', '>= 7.0'
depends 'yum', '~> 7.0'

```

attributes/default.rb

```

# Patching configuration
default['linux_patching']['enabled'] = true
default['linux_patching']['security_only'] = true
default['linux_patching']['reboot_enabled'] = true
default['linux_patching']['reboot_delay'] = 60
default['linux_patching']['exclude_packages'] = []
default['linux_patching']['pre_check'] = true
default['linux_patching']['post_check'] = true
default['linux_patching']['backup_configs'] = true
default['linux_patching']['change_ticket'] = 'CHG-UNKNOWN'
default['linux_patching']['max_boot_usage'] = 80
default['linux_patching']['max_root_usage'] = 85
default['linux_patching']['min_var_free_mb'] = 2048
default['linux_patching']['report_path'] = '/root/patch_reports'

```

recipes/default.rb

```

#
# Cookbook:: linux_patching
# Recipe:: default
#
# Enterprise Linux Patching - Main Recipe
return unless node['linux_patching']['enabled']
# Create report directory

```

```

directory node['linux_patching']['report_path'] do
  owner 'root'
  group 'root'
  mode '0700'
  action :create
end
# Run pre-checks
include_recipe 'linux_patching::pre_check' if
node['linux_patching']['pre_check']
# Apply patches based on configuration
if node['linux_patching']['security_only']
  include_recipe 'linux_patching::patch_security'
else
  include_recipe 'linux_patching::patch_all'
end
# Handle reboot
include_recipe 'linux_patching::reboot' if
node['linux_patching']['reboot_enabled']
# Run post-checks
include_recipe 'linux_patching::post_check' if
node['linux_patching']['post_check']

```

recipes/pre_check.rb

```

#
# Recipe:: pre_check
# Pre-patching validation checks
log 'Starting pre-patch checks' do
  level :info
end
# Check disk space
ruby_block 'check_disk_space' do
  block do
    # Check /boot
    boot_usage = shell_out("df /boot --output=pcent | tail -1 | tr
-d ' %'").stdout.strip.to_i
    if boot_usage > node['linux_patching']['max_boot_usage']
      raise "/boot is #{boot_usage}% full. Must be below
#{node['linux_patching']['max_boot_usage']}%"
    end
    # Check /
    root_usage = shell_out("df / --output=pcent | tail -1 | tr -d '
%'").stdout.strip.to_i
    if root_usage > node['linux_patching']['max_root_usage']
      raise "/" is #{root_usage}% full. Must be below
#{node['linux_patching']['max_root_usage']}%"
    end
    # Check /var free space
    var_free = shell_out("df /var --output=avail | tail
-1").stdout.strip.to_i / 1024
    if var_free < node['linux_patching']['min_var_free_mb']
      raise "/var has only #{var_free}MB free. Need
#{node['linux_patching']['min_var_free_mb']}MB"
    end
  end
end

```

```

        end
        Chef::Log.info("Disk space checks passed: /boot=#{boot_usage}%,
/=#{root_usage}%, /var=#{var_free}MB free")
    end
end
# Backup critical files
if node['linux_patching']['backup_configs']
    backup_dir =

    "#{node['linux_patching']['report_path']}/backup_#{Time.now.strftime('%Y%m%d')}}"

    directory backup_dir do
        owner 'root'
        mode '0700'
    end
    %w[/etc/fstab /etc/default/grub /etc/hosts /etc/resolv.conf].each
do |config_file|
    execute "backup_#{File.basename(config_file)}" do
        command "cp -a #{config_file} #{backup_dir}/ 2>/dev/null ||
true"
        not_if { !File.exist?(config_file) }
    end
end
end
# Record pre-patch state
execute 'record_pre_patch_state' do
    command <<-EOH
        echo "=== PRE-PATCH STATE ===" >
        #{node['linux_patching']['report_path']}/pre_state.txt
        echo "Date: $(date)" >>
        #{node['linux_patching']['report_path']}/pre_state.txt
        echo "Hostname: $(hostname)" >>
        #{node['linux_patching']['report_path']}/pre_state.txt
        echo "Kernel: $(uname -r)" >>
        #{node['linux_patching']['report_path']}/pre_state.txt
        echo "Uptime: $(uptime)" >>
        #{node['linux_patching']['report_path']}/pre_state.txt
        systemctl list-units --state=running --type=service --no-pager
> #{node['linux_patching']['report_path']}/pre_services.txt
        rpm -qa --last | head -20 >
        #{node['linux_patching']['report_path']}/pre_packages.txt
    EOH
end
end

```

recipes/patch_security.rb

```

#
# Recipe:: patch_security
# Apply security patches only
log 'Applying security patches' do
    level :info
end
# Build exclude list

```

```

exclude_string = node['linux_patching']['exclude_packages'].map {
|p| "--exclude=#{p}" }.join(' ')
execute 'apply_security_patches' do
  command "yum update --security -y #{exclude_string}"
  action :run
  live_stream true
  timeout 3600
  returns [0, 100] # 100 = no updates available
end
# Record what was installed
execute 'record_installed_patches' do
  command "yum history info last >
#{node['linux_patching']['report_path']}/patch_transaction.txt
2>&1"
end

```

recipes/patch_all.rb

```

#
# Recipe:: patch_all
# Apply all available patches
log 'Applying all available patches' do
  level :info
end
exclude_string = node['linux_patching']['exclude_packages'].map {
|p| "--exclude=#{p}" }.join(' ')
execute 'apply_all_patches' do
  command "yum update -y #{exclude_string}"
  action :run
  live_stream true
  timeout 3600
  returns [0, 100]
end
execute 'record_installed_patches' do
  command "yum history info last >
#{node['linux_patching']['report_path']}/patch_transaction.txt
2>&1"
end

```

recipes/reboot.rb

```

#
# Recipe:: reboot
# Handle reboot if required
reboot_required = shell_out('needs-restarting -r')
if reboot_required.exitstatus == 1
  log 'Reboot is required - scheduling reboot' do
    level :warn
  end
  file
  "#{node['linux_patching']['report_path']}/reboot_scheduled.flag" do
    content "Reboot scheduled at #{Time.now}\nReason: Kernel or
core library update\n"
  end
end

```

```

        owner 'root'
        mode '0644'
    end
    reboot 'patching_reboot' do
        delay_mins node['linux_patching']['reboot_delay'] / 60
        reason "Patching reboot -
#{node['linux_patching']['change_ticket']}"
        action :reboot_now
    end
else
    log 'No reboot required' do
        level :info
    end
end
end

```

recipes/post_check.rb

```

#
# Recipe:: post_check
# Post-patching validation
log 'Running post-patch checks' do
    level :info
end
ruby_block 'post_patch_validation' do
    block do
        # Check for failed services
        failed = shell_out('systemctl list-units --state=failed
--no-pager').stdout
        unless failed.include?('0 loaded units listed')
            Chef::Log.warn("FAILED SERVICES DETECTED:\n#{failed}")
        end
        # Verify kernel
        current_kernel = shell_out('uname -r').stdout.strip
        Chef::Log.info("Current kernel: #{current_kernel}")
        # Check network
        gateway = shell_out("ip route show default | awk '{print
$3}'").stdout.strip
        ping_result = shell_out("ping -c 2 #{gateway}")
        unless ping_result.exitstatus == 0
            Chef::Log.error("Cannot ping default gateway #{gateway}!")
        end
    end
end
end
# Generate report
template

"#{node['linux_patching']['report_path']}/patch_report_#{Time.now.strftime('%Y%m%d')}"
source 'patch_report.erb'
owner 'root'
mode '0644'
variables(
    hostname: node['hostname'],
    kernel: node['kernel']['release'],

```

```

        platform_version: node['platform_version'],
        change_ticket: node['linux_patching']['change_ticket']
    )
end

```

recipes/rollback.rb

```

#
# Recipe:: rollback
# Rollback last patch transaction
log 'INITIATING PATCH ROLLBACK' do
  level :warn
end
# Get last transaction ID
ruby_block 'get_transaction_id' do
  block do
    result = shell_out("yum history list | awk 'NR==4{print $1}'")
    node.run_state['rollback_transaction'] = result.stdout.strip
    Chef::Log.warn("Rolling back transaction:
#{node.run_state['rollback_transaction']}")
  end
end
end
execute 'yum_history_undo' do
  command lazy { "yum history undo
#{node.run_state['rollback_transaction']} -y" }
  action :run
  live_stream true
  timeout 1800
end
# Reboot after rollback if kernel was changed
reboot 'rollback_reboot' do
  delay_mins 1
  reason "Rollback reboot -
#{node['linux_patching']['change_ticket']}"
  action :reboot_now
  only_if 'needs-restarting -r; test $? -eq 1'
end

```

7.4 Chef Execution Workflow

```

# [Run on: Chef Workstation - e.g., your admin machine]
# Upload cookbook to Chef Server
knife cookbook upload linux_patching
# Set environment attributes
knife environment edit production
# Add: "linux_patching": { "security_only": true, "change_ticket":
"CHG-2026-12345" }
# Run on single node
knife ssh "name:webserver01.corp.com" "sudo chef-client -o
'recipe[linux_patching]'"
# Run on nodes by role
knife ssh "role:webserver AND chef_environment:production" \

```

```

    "sudo chef-client -o 'recipe[linux_patching]'" \
    --concurrency 10
# Run on nodes by environment
knife ssh "chef_environment:development" \
    "sudo chef-client -o 'recipe[linux_patching]'"
# Rollback
knife ssh "name:webserver01.corp.com" \
    "sudo chef-client -o 'recipe[linux_patching::rollback]'"

```

7.5 Chef vs Ansible Comparison for Patching

Feature	Chef	Ansible	Architecture	Agent-based (pull)	Agentless (push)	Language
Ruby DSL	YAML	Learning Curve	Steeper	Easier	Idempotency	Built-in
Batch Control	Limited (requires orchestration)	Native (serial)	Rollback	Manual recipe	yum history undo	Reporting
Chef Automate	Tower/AAP	Speed (2000 nodes)	Parallel (agent pull)	Serial/batched (push)	Best For	Configuration drift
Ad-hoc patching						

·EXPERT TIP: Chef's agent-based model means all 2000 servers can pull patches simultaneously during their chef-client run. However, this makes batch control harder. Use Chef environments and version pinning to control rollout timing.

Section 8 · Production Patching Procedure

8.1 Before Patching (T-minus Checklist)

T-5 Days: Planning Phase

#	Action	Responsible	Status
1	Submit Change Request to CAB	Linux Team Lead	
2	Identify servers for patching window	Linux Engineer	
3	Review patch content (CVEs, packages)	Security Team	
4	Verify DEV/UAT testing completed	QA Team	
5	Notify application owners	Linux Team Lead	
6	Notify DBA team	Linux Team Lead	
7	Confirm backup schedule	Backup Team	
8	Prepare rollback plan	Linux Engineer	

T-2 Days: Preparation Phase

#	Action	Responsible	Status
1	Obtain CAB approval	Change Manager	
2	Obtain Business approval (critical systems)	Business Owner	
3	Send downtime notification to users	Service Desk	
4	Confirm backup completion for all servers	Backup Team	
5	Verify VM snapshots can be taken	VMware Team	
6	Pre-stage patches on Satellite/Capsules	Linux Engineer	
7	Verify Ansible playbooks tested	Linux Engineer	
8	Prepare monitoring suppression	NOC Team	

T-1 Day: Final Preparation

#	Action	Responsible	Status
1	Run pre-patch checks on all target servers	Linux Engineer	
2	Resolve any pre-check failures	Linux Engineer	
3	Confirm no conflicting maintenance activities	Change Manager	
4	Ensure on-call team is aware and available	Team Lead	
5	Verify console/IPMI/iLO access works	Linux Engineer	
6	Brief bridge call participants	Team Lead	
7	Prepare status update template	Linux Engineer	

T-0 (Execution Window Start): Go/No-Go

```

.....
.                                GO / NO-GO CHECKLIST                                .
.....
.
.
.   . CAB Approval obtained (CHG-XXXX approved)
.   . Business approval obtained (if required)
.   . Backup verified for all target servers
.   . VM snapshots taken (where applicable)
.   . Monitoring alerts suppressed
.   . Bridge call established
.   . All team members available
.   . No active P1/P2 incidents
.   . No other conflicting changes in progress
.   . Rollback plan confirmed and tested
.   . Console access verified for critical servers
.
.
.  DECISION:   . GO      . NO-GO
.  AUTHORIZED BY: _____  TIME: _____
.
.....

```

8.2 During Patching

Sequential Patching (Recommended for Critical Systems)

Execution Order for HA Applications:

- ```

.....
1. Patch PASSIVE/STANDBY node first
2. Validate standby node completely
3. Perform controlled failover (optional)
4. Patch remaining node(s)
5. Verify cluster status

```

Timeline:

```

... 22:00 - Bridge call start, Go/No-Go
... 22:15 - Start Batch 1 (50 servers)
... 23:15 - Batch 1 validation complete
... 23:30 - Start Batch 2 (100 servers)
... 01:00 - Batch 2 validation complete
... 01:15 - Start Batch 3 (250 servers)
... 03:00 - Batch 3 validation complete
... 03:15 - Start Batch 4 (1600 servers, parallel)
... 05:30 - Batch 4 complete
... 05:30 - Final validation
... 06:00 - Window close, bridge call end

```

### Parallel Patching (For Non-Critical Systems)

```

Ansible parallel execution
ansible-playbook patch_enterprise_batches.yml \
 -e "target_group=batch4_remaining" \
 -f 50 # 50 forks (parallel connections)

```

## HA Cluster Patching Procedure

```
.....
. HA CLUSTER PATCHING PROCEDURE .
.....
.
. Step 1: Verify cluster health .
. $ pcs status .
. $ pcs resource status .
.
. Step 2: Put Node 2 (standby) in maintenance .
. $ pcs node standby node2.corp.com .
.
. Step 3: Patch Node 2 .
. $ yum update --security -y .
. $ reboot .
.
. Step 4: Bring Node 2 back online .
. $ pcs node unstandby node2.corp.com .
. $ pcs status (verify resources) .
.
. Step 5: Move resources to Node 2 .
. $ pcs resource move <resource> node2.corp.com .
.
. Step 6: Put Node 1 in maintenance .
. $ pcs node standby node1.corp.com .
.
. Step 7: Patch Node 1 .
. $ yum update --security -y .
. $ reboot .
.
. Step 8: Bring Node 1 back online .
. $ pcs node unstandby node1.corp.com .
. $ pcs status (verify all resources) .
.
. Step 9: Clear resource constraints .
. $ pcs resource clear <resource> .
.
.....
```

## Load Balancer Considerations

```
Before patching a server behind a load balancer:
1. Drain connections from the server
F5 BIG-IP:
tmsh modify /ltm node /Common/webserver01.corp.com session
user-disabled
tmsh modify /ltm node /Common/webserver01.corp.com state user-down
HAProxy:
echo "disable server web-backend/webserver01" | socat stdio
/var/run/haproxy/admin.sock
Nginx upstream:
```

```

Mark server as down in upstream config or use dynamic module
2. Wait for active connections to drain
sleep 30
Verify no active connections:
ss -tn | grep <server_ip>:<app_port> | wc -l
3. Patch the server
yum update --security -y
reboot # if needed
4. Verify application health
curl -s http://webserver01:80/health
Expected: 200 OK
5. Re-enable in load balancer
F5:
tmsh modify /ltm node /Common/webserver01.corp.com session
user-enabled
tmsh modify /ltm node /Common/webserver01.corp.com state user-up
HAProxy:
echo "enable server web-backend/webserver01" | socat stdio
/var/run/haproxy/admin.sock

```

## 8.3 After Patching

### Application Validation Checklist

```

|#| Validation | Command/Method | Expected Result | ---|-----|-----|-----| 1 | Web
application responds | curl -s -o /dev/null -w "%{http_code}" http://app/health | 200 || 2
| API endpoints functional | curl -X GET http://api/v1/status | JSON success || 3 | Database
connections | mysql -e "SELECT 1" or psql -c "SELECT 1" | Returns 1 || 4 | Message queues | Check
RabbitMQ/Kafka status | All queues active || 5 | Batch jobs | Verify cron/scheduler | Jobs executing || 6 | Log files
generating | tail -1 /var/log/app/application.log | Recent entries || 7 | SSL certificates | openssl
s_client -connect host:443 | Cert valid || 8 | Authentication | Login test | Successful login || 9 | File
transfers | SFTP/SCP test | Transfer success || 10 | Monitoring reporting | Check Nagios/Zabbix | All checks green |

```

### Service Validation Commands

```

Verify all expected services are running
systemctl list-units --state=failed
Expected: 0 loaded units listed
Compare with pre-patch service list
diff <(systemctl list-units --type=service --state=running
--no-pager | awk '{print $1}' | sort) \
 <(cat /root/pre_patch_backup/running_services.txt | sort)
Application-specific checks
Apache/Nginx:
curl -I http://localhost/
systemctl status httpd nginx
Tomcat:
curl http://localhost:8080/
systemctl status tomcat
Database:
systemctl status mysqld postgresql oracle-xe
Check application logs for errors after reboot

```

```
journalctl --since "10 minutes ago" -p err --no-pager
tail -50 /var/log/app/error.log
```

## Business Sign-off Process

```
.....
· POST-PATCHING SIGN-OFF ·
.....
·
· All servers patched successfully: · Yes · No ·
· All services running: · Yes · No ·
· Application health verified: · Yes · No ·
· No rollbacks required: · Yes · No ·
· Monitoring shows normal: · Yes · No ·
· Business users confirmed: · Yes · No ·
·
· SIGN-OFF:
· Linux Team Lead: _____ Date: _____ ·
· Application Owner: _____ Date: _____ ·
· Business Owner: _____ Date: _____ ·
·
· Notes/Exceptions:
· _____ ·
· _____ ·
·
.....
```

**LESSON LEARNED:** In a 2000-server environment, always have at least 2 engineers on the bridge call. One executes the patching, the other monitors dashboards and validates results. Never have a single point of failure in your execution team.

---

## Section 9 · Zero Downtime Patching

### 9.1 Zero Downtime Patching Concepts

```
.....
· ZERO DOWNTIME PATCHING STRATEGY ·
.....
·
·
· Key Principle: Never patch ALL nodes serving the same
· application simultaneously. Always maintain service availability.
·
·
· Methods:
·
· 1. Rolling updates behind load balancer
```

- 
- 2. Active-Passive cluster failover
- 
- 3. Active-Active cluster rolling
- 
- 4. Blue-Green deployment
- 
- 5. Canary deployment
- 
- 
- 
- .....

## 9.2 Active-Passive Cluster Patching

BEFORE PATCHING:

```
.....
· NODE 1 · · CLIENTS ·
· (ACTIVE) · · ·
.....
· NODE 2 · (Standby - not serving traffic)
· (PASSIVE)·
.....
```

STEP 1: Patch passive node

```
.....
· NODE 1 · · CLIENTS · (Still serving)
· (ACTIVE) · · ·
.....
· NODE 2 · · PATCHING + REBOOT
· (PASSIVE)·
.....
```

STEP 2: Failover to patched node

```
.....
· NODE 1 · (Now standby)
· (PASSIVE)·
.....
· NODE 2 · · CLIENTS · (Serving on patched node)
· (ACTIVE) · · ·
.....
```

STEP 3: Patch former active node

```
.....
· NODE 1 · · PATCHING + REBOOT
· (PASSIVE)·
.....
· NODE 2 · · CLIENTS ·
· (ACTIVE) · · ·
.....
```

FINAL: Both nodes patched, service never interrupted

## Pacemaker/Corosync Cluster Commands

```
Check cluster status
pcs status
pcs status --full
Put standby node in maintenance
pcs node standby node2.corp.com
Expected: node2 is now in standby mode
Verify resources moved
pcs resource status
All resources should be on node1
Patch the standby node
ssh node2.corp.com "yum update --security -y && reboot"
Wait for node2 to come back
(wait for reboot to complete)
Bring node2 back online
pcs node unstandby node2.corp.com
Verify node2 is healthy
pcs status
node2 should be Online
Move resources to node2 (controlled failover)
pcs resource move myapp-vip node2.corp.com
pcs resource move myapp-service node2.corp.com
Now patch node1
pcs node standby node1.corp.com
ssh node1.corp.com "yum update --security -y && reboot"
Bring node1 back
pcs node unstandby node1.corp.com
Clear location constraints
pcs resource clear myapp-vip
pcs resource clear myapp-service
Final verification
pcs status
```

## 9.3 Active-Active Cluster Patching

```
For Active-Active clusters (e.g., Galera MySQL, multi-node web)
Patch one node at a time while others handle traffic
Example: 4-node Active-Active web cluster behind LB
Nodes: web01, web02, web03, web04
Step 1: Remove web01 from load balancer
echo "disable server web-backend/web01" | socat stdio
/var/run/haproxy/admin.sock
Step 2: Wait for connections to drain
sleep 60
ss -tn state established | grep web01 | wc -l # Should be 0
Step 3: Patch web01
ssh web01 "yum update --security -y"
ssh web01 "reboot" # if needed
Step 4: Wait for web01 to come back, verify health
until curl -sf http://web01/health; do sleep 5; done
Step 5: Re-enable web01 in load balancer
echo "enable server web-backend/web01" | socat stdio
```

```
/var/run/haproxy/admin.sock
Step 6: Wait, verify traffic flowing
sleep 30
Repeat for web02, web03, web04...
```

## 9.4 Application-Specific Zero Downtime Procedures

### Apache HTTP Server

```
Apache behind load balancer - graceful restart (no connection
drop)
1. Remove from LB
2. Patch
yum update httpd -y
3. Graceful restart (finishes active requests)
apachectl graceful
Or:
systemctl reload httpd
4. Verify
curl -I http://localhost/
systemctl status httpd
Check access log for new requests
5. Add back to LB
```

### Nginx

```
Nginx zero-downtime reload
1. Remove from upstream LB pool
2. Patch
yum update nginx -y
3. Test configuration
nginx -t
Expected: syntax is ok, test is successful
4. Graceful reload (zero-downtime)
systemctl reload nginx
Or:
nginx -s reload
5. Verify
curl -I http://localhost/
Check: nginx worker processes reloaded
6. Add back to LB pool
```

### Tomcat

```
Tomcat zero-downtime with multiple instances behind LB
1. Remove instance from LB
2. Drain active sessions (wait for session timeout or use session
replication)
3. Stop Tomcat gracefully
/opt/tomcat/bin/shutdown.sh
Wait up to 30 seconds for clean shutdown
sleep 30
```

```

4. Apply OS patches
yum update --security -y
5. Start Tomcat
/opt/tomcat/bin/startup.sh
6. Wait for application to be ready
until curl -sf http://localhost:8080/health; do
 echo "Waiting for Tomcat..."
 sleep 5
done
7. Verify application health
curl http://localhost:8080/api/v1/health
Expected: {"status": "UP"}
8. Add back to LB pool
Reboot if kernel update (schedule during next node's drain time):
reboot

```

## JBoss/WildFly

```

JBoss EAP in domain mode - rolling restart
1. Check domain status
/opt/jboss/bin/jboss-cli.sh --connect
controller=domain-controller:9990 \
 --command="/host=host1:read-attribute(name=host-state)"
Expected: "running"
2. Stop one server in the server group
/opt/jboss/bin/jboss-cli.sh --connect \
 --command="/host=host1/server-config=server-one:stop"
3. Patch OS
yum update --security -y
4. Restart the server
/opt/jboss/bin/jboss-cli.sh --connect \
 --command="/host=host1/server-config=server-one:start"
5. Verify deployment status
/opt/jboss/bin/jboss-cli.sh --connect \

 --command="/host=host1/server=server-one/deployment=myapp.war:read-attribute(name=status)"
Expected: "OK"
Repeat for other hosts in domain

```

## MySQL/MariaDB (Galera Cluster)

```

Galera Cluster - 3 node minimum
Nodes: db01, db02, db03
1. Check cluster status
mysql -e "SHOW STATUS LIKE 'wsrep_cluster_size';"
Expected: 3
mysql -e "SHOW STATUS LIKE 'wsrep_cluster_status';"
Expected: Primary
2. Set node to desync (stop receiving new writes temporarily)
mysql -e "SET GLOBAL wsrep_desync=ON;"
3. Patch the node
yum update --security -y

```

```

4. Restart MySQL (if needed)
systemctl restart mysqld
5. Re-enable sync
mysql -e "SET GLOBAL wsrep_desync=OFF;"
6. Verify cluster rejoined
mysql -e "SHOW STATUS LIKE 'wsrep_local_state_comment';"
Expected: Synced
mysql -e "SHOW STATUS LIKE 'wsrep_cluster_size';"
Expected: 3
Repeat for next node (wait for full sync between nodes)

```

## PostgreSQL (Streaming Replication)

```

PostgreSQL Primary-Standby Replication
1. Check replication status (on Primary)
psql -c "SELECT client_addr, state, sync_state FROM
pg_stat_replication;"
Expected: standby showing "streaming"
2. Patch STANDBY first
ssh pg-standby "yum update --security -y"
ssh pg-standby "systemctl restart postgresql"
3. Verify replication resumed
psql -c "SELECT client_addr, state FROM pg_stat_replication;"
Expected: standby showing "streaming" again
4. Promote standby (if you want to patch primary)
ssh pg-standby "pg_ctl promote -D /var/lib/pgsql/data"
Or:
ssh pg-standby "psql -c 'SELECT pg_promote();'"
5. Patch former primary
yum update --security -y
reboot # if needed
6. Reconfigure as new standby
Edit recovery.conf / postgresql.conf for standby mode
Restart PostgreSQL
7. Verify new topology
psql -c "SELECT pg_is_in_recovery();"
Primary: false, Standby: true

```

## Oracle Database (Data Guard)

```

Oracle Data Guard - Patch Standby First
1. Check Data Guard status
dgmgrl sys/password@primary "show configuration"
Expected: Configuration Status: SUCCESS
2. On STANDBY: Stop apply
dgmgrl sys/password@standby "edit database standby set
state='APPLY-OFF'"
3. Patch standby OS
yum update --security -y
reboot # if needed
4. Resume apply on standby
dgmgrl sys/password@standby "edit database standby set
state='APPLY-ON'"

```

```
5. Verify standby is caught up
dgmgrl sys/password@primary "show database standby"
Check: Transport Lag: 0 seconds, Apply Lag: 0 seconds
6. Perform switchover (if patching primary)
dgmgrl sys/password@primary "switchover to standby"
7. Patch former primary (now standby)
yum update --security -y
reboot
8. Verify configuration
dgmgrl sys/password@new_primary "show configuration"
```

## 9.5 Zero Downtime Decision Matrix

| Application Type | Method    | Downtime | Complexity              | Stateless web (behind LB) | Stateful web (sessions) | Active-Passive DB       | Active-Active DB (Galera) | Streaming Replication DB | Message Queue cluster | Standalone application |
|------------------|-----------|----------|-------------------------|---------------------------|-------------------------|-------------------------|---------------------------|--------------------------|-----------------------|------------------------|
| Rolling restart  | 0 seconds | Low      | Session drain + rolling | 0 seconds                 | Failover + patch        | < 30 seconds (failover) | Promote standby           | < 60 seconds             | High                  | Full restart time      |
| Low              | Low       | Low      | Low                     | Low                       | Low                     | Low                     | Low                       | Low                      | Low                   | Low                    |

**EXPERT TIP:** True zero-downtime is only possible with redundant infrastructure. If you have single points of failure, budget for brief outages during kernel patches that require reboot. Consider implementing kpatch/livepatch for critical single-node systems.

## Section 10 · Reboot Management

### 10.1 When is Reboot Required?

| Update Type                   | Reboot Required? | Reason                         | Kernel update            | glibc update             | openssl/gnutls           | dbus update              | NetworkManager           | Application packages     |
|-------------------------------|------------------|--------------------------------|--------------------------|--------------------------|--------------------------|--------------------------|--------------------------|--------------------------|
| New kernel only loads at boot | <b>YES</b>       | All processes link to glibc    | <b>YES</b> (recommended) | <b>YES</b> (recommended) | <b>YES</b> (recommended) | <b>YES</b> (recommended) | <b>YES</b> (recommended) | <b>YES</b> (recommended) |
| systemd update                | <b>YES</b>       | PID 1 cannot be restarted      | <b>YES</b>               | <b>YES</b>               | <b>YES</b>               | <b>YES</b>               | <b>YES</b>               | <b>YES</b>               |
| Service restart minimum       | <b>YES</b>       | Libraries reloaded per-process | <b>YES</b>               | <b>YES</b>               | <b>YES</b>               | <b>YES</b>               | <b>YES</b>               | <b>YES</b>               |
| Core IPC mechanism            | <b>YES</b>       | Linux firmware                 | <b>YES</b>               | <b>YES</b>               | <b>YES</b>               | <b>YES</b>               | <b>YES</b>               | <b>YES</b>               |
| Firmware loads at boot        | <b>YES</b>       | Firmware loads at boot         | <b>YES</b>               | <b>YES</b>               | <b>YES</b>               | <b>YES</b>               | <b>YES</b>               | <b>YES</b>               |
| Service restart               | <b>YES</b>       | Can restart without reboot     | <b>YES</b>               | <b>YES</b>               | <b>YES</b>               | <b>YES</b>               | <b>YES</b>               | <b>YES</b>               |
| httpd/nginx                   | <b>YES</b>       | Service restart                | <b>YES</b>               | <b>YES</b>               | <b>YES</b>               | <b>YES</b>               | <b>YES</b>               | <b>YES</b>               |
| Standalone service            | <b>YES</b>       | Restart the application        | <b>YES</b>               | <b>YES</b>               | <b>YES</b>               | <b>YES</b>               | <b>YES</b>               | <b>YES</b>               |

### 10.2 How to Identify if Reboot is Required

```
Method 1: needs-restarting command (recommended)
needs-restarting -r
Exit code 0 = No reboot needed
Exit code 1 = Reboot needed
Output example when reboot needed:
Core libraries or services have been updated since boot-up:
* kernel
* glibc
* linux-firmware
Reboot is required to fully utilize these updates.
Method 2: Compare running kernel vs installed kernel
RUNNING=$(uname -r)
INSTALLED=$(rpm -q kernel --last | head -1 | awk '{print $1}' | sed
```

```

's/kernel-//')
if ["$RUNNING" != "$INSTALLED"]; then
 echo "REBOOT REQUIRED: Running $RUNNING, Installed $INSTALLED"
else
 echo "No kernel reboot needed"
fi
Method 3: Check for processes using deleted libraries
needs-restarting -s
Lists services that need restart (without full reboot)
Example output:
httpd.service
sshd.service
crond.service
Method 4: Check /var/run/reboot-required (Debian/Ubuntu)
Not applicable for RHEL, but mentioning for completeness
RHEL 6 (no needs-restarting):
Compare running kernel manually:
rpm -q kernel | sort -V | tail -1 | sed 's/kernel-//'
uname -r
If different · reboot needed

```

### 10.3 Services That Need Restart (Without Full Reboot)

```

List services needing restart after library updates
needs-restarting -s
Restart only affected services (avoid full reboot when possible)
for svc in $(needs-restarting -s 2>/dev/null); do
 echo "Restarting: $svc"
 systemctl restart "$svc"
done
Verify after service restarts:
needs-restarting -s
Expected: Empty output (no services need restart)
If still shows services · full reboot may be needed
needs-restarting -r

```

### 10.4 Reboot Best Practices

#### Pre-Reboot Checklist

```

1. Verify GRUB is configured correctly
grubby --default-kernel
Expected: /boot/vmlinuz-<new_kernel_version>
2. Verify fstab is correct (critical!)
cat /etc/fstab
findmnt --verify
Expected: No errors
3. Check for pending filesystem journals
sync
4. Notify users/applications
wall "System reboot in 5 minutes for patching. Please save your
work."

```

```
5. Stop critical applications gracefully
systemctl stop tomcat
systemctl stop httpd
6. Verify console access is available (in case boot fails)
Check iLO/iDRAC/IPMI connectivity from management network
ipmitool -H <bmc_ip> -U admin -P password power status
```

## Reboot Commands

```
Standard reboot (immediate)
reboot
Or:
systemctl reboot
Or:
shutdown -r now
Scheduled reboot (5 minutes)
shutdown -r +5 "Patching reboot in 5 minutes"
Cancel scheduled reboot
shutdown -c
Force reboot (DANGEROUS - use only if system is hung)
echo b > /proc/sysrq-trigger
Or:
reboot -f
Reboot with specific kernel (if new kernel has issues)
grubby --set-default /boot/vmlinuz-4.18.0-513.18.1.el8_9.x86_64
reboot
```

## Post-Reboot Validation

```
1. Verify correct kernel booted
uname -r
Expected: New kernel version
2. Check uptime (should be very low)
uptime
Expected: up 0 min or up X min (where X is small)
3. Check for failed services
systemctl list-units --state=failed
Expected: 0 loaded units listed
4. Verify network
ip addr show
ping -c 3 <gateway>
ping -c 3 <dns_server>
5. Verify filesystems
df -h
mount | grep -w "ro," # Check for read-only mounts
6. Check application services
systemctl status httpd tomcat mysqld # whatever applies
7. Check system logs for boot errors
journalctl -b -p err --no-pager | head -30
dmesg | grep -i error | head -10
```

## 10.5 Reboot Scheduling for Large Environments

```

2000 Server Reboot Schedule (8-hour window):
.....
22:00-22:30 Batch 1 (50 servers) - 10 parallel reboots
22:30-23:30 Validation + monitoring observation
23:30-00:00 Batch 2 (100 servers) - 20 parallel reboots
00:00-01:00 Validation + monitoring observation
01:00-02:00 Batch 3 (250 servers) - 50 parallel reboots
02:00-02:30 Validation
02:30-05:30 Batch 4 (1600 servers) - 100 parallel reboots
05:30-06:00 Final validation and close

```

·**COMMON MISTAKE:** Never reboot a server without verifying /etc/fstab is correct. If fstab references a non-existent device or has syntax errors, the server will hang at boot and require console intervention. Run `findmnt --verify` before rebooting.

---

## Section 11 · Post Patch Validation

### 11.1 Kernel Validation

```

Verify new kernel is running
uname -r
Expected: New kernel version (e.g., 5.14.0-362.24.1.el9_3.x86_64)
Verify kernel modules loaded
lsmod | wc -l
Expected: Similar count to pre-patch (typically 50-100+)
Check for kernel errors
dmesg | grep -i "error\|warn\|fail" | grep -v "ACPI" | head -20
Verify kernel parameters
sysctl -a 2>/dev/null | grep -E "net.ipv4.ip_forward|vm.swappiness"
| head -5
Compare with pre-patch values
Check SELinux status
getenforce
Expected: Enforcing (or whatever was set before)

```

### 11.2 Service Validation

```

Check for failed services (most critical check)
systemctl list-units --state=failed --no-pager
Expected: 0 loaded units listed. Pass.
Verify critical infrastructure services
for svc in sshd crond rsyslog chronyd NetworkManager firewalld
auditd; do
 STATUS=$(systemctl is-active $svc 2>/dev/null)
 echo "$svc: $STATUS"
done
Expected: All should show "active"
Compare running services with pre-patch list
diff <(systemctl list-units --type=service --state=running

```

```
--no-pager | \
 grep ".service" | awk '{print $1}' | sort) \
 /root/pre_patch_backup/running_services.txt
Expected: Minimal differences (maybe new service versions)
Check service start times (should show recent boot time)
systemctl show sshd --property=ActiveEnterTimestamp
```

### 11.3 Application Validation

```
Web Server
curl -s -o /dev/null -w "HTTP Code: %{http_code}\nTime:
%{time_total}s\n" http://localhost/health
Expected: HTTP Code: 200, Time: < 1s
Tomcat / Java Application
curl -s http://localhost:8080/api/health | python3 -m json.tool
Expected: {"status": "UP", "components": {...}}
JBoss/WildFly
/opt/jboss/bin/jboss-cli.sh --connect
command=":read-attribute(name=server-state)"
Expected: "running"
Check application-specific logs
tail -20 /var/log/tomcat/catalina.out | grep -i
"error\|exception\|started"
tail -20 /var/log/httpd/error_log
tail -20 /var/log/nginx/error.log
```

### 11.4 Network Validation

```
Verify IP addresses
ip addr show | grep "inet "
Compare with pre-patch output
Verify default gateway
ip route show default
Expected: default via <gateway_ip> dev <interface>
Test connectivity
ping -c 3 $(ip route show default | awk '{print $3}')
Expected: 3 packets received, 0% loss
Verify DNS resolution
nslookup $(hostname -f)
dig google.com +short # If external DNS allowed
Verify listening ports
ss -tuln | sort
Compare with pre-patch output
Check bonding/teaming (if applicable)
cat /proc/net/bonding/bond0 2>/dev/null | grep -E
"Status|Interface"
Expected: MII Status: up for all interfaces
```

### 11.5 Storage Validation

```
Filesystem check
```

```

df -hT
Compare with pre-patch values, ensure no filesystem is 100%
Mount points
mount | column -t
Verify all expected mounts are present
LVM status
lvs
vgs
pvs
All should show normal status
Multipath (SAN storage)
multipath -ll
Expected: All paths active [ready]
NFS mounts
showmount -e <nfs_server> 2>/dev/null
mount | grep nfs
Verify NFS mounts accessible
ls /mnt/nfs_share/ >/dev/null 2>&1 && echo "NFS OK" || echo "NFS
FAIL"

```

## 11.6 Database Validation

```

MySQL/MariaDB
mysqladmin ping
mysql -e "SELECT VERSION();"
mysql -e "SHOW SLAVE STATUS\G" | grep -E
"Slave_IO|Slave_SQL|Seconds_Behind"
Expected: Both Running: Yes, Seconds_Behind_Master: 0
PostgreSQL
pg_isready
psql -c "SELECT version();"
psql -c "SELECT * FROM pg_stat_replication;" 2>/dev/null
Expected: Replication streaming
Oracle
su - oracle -c "sqlplus -S / as sysdba <<< 'SELECT STATUS FROM
V\$INSTANCE;'"
Expected: OPEN

```

## 11.7 Monitoring Validation

```

Verify monitoring agent is running
systemctl status zabbix-agent nrpe nagios-nrpe node_exporter
2>/dev/null | grep "Active:"
Expected: active (running)
Test monitoring connectivity
Zabbix:
zabbix_agentd -t system.uptime
Expected: Returns uptime value
Nagios/NRPE:
/usr/lib64/nagios/plugins/check_load -w 5,4,3 -c 10,8,6
Expected: OK - load average: x.xx, x.xx, x.xx
Prometheus Node Exporter:

```

```
curl -s http://localhost:9100/metrics | head -5
Expected: Metrics output
```

## 11.8 Log Validation

```
Check for critical errors after boot
journalctl -b -p crit --no-pager
Expected: No critical errors (or known/acceptable ones)
Check for error-level messages
journalctl -b -p err --no-pager | tail -20
Review for any patching-related errors
Application log check
tail -50 /var/log/messages | grep -i "error\|fail\|panic"
tail -50 /var/log/secure | grep -i "error\|fail"
Check audit log
ausearch -m AVC --start recent 2>/dev/null | head -10
Check for SELinux denials after patching
```

## 11.9 Complete Post-Patch Validation Script

```
#!/bin/bash
post_patch_validate.sh
Run after patching to validate server health
REPORT="/root/post_patch_report_$(date +%Y%m%d_%H%M%S).txt"
PASS=0
FAIL=0
check() {
 local desc="$1"
 local result="$2"
 if [$? -eq 0] && [-n "$result"]; then
 echo ". PASS: $desc" | tee -a $REPORT
 ((PASS++))
 else
 echo ". FAIL: $desc" | tee -a $REPORT
 ((FAIL++))
 fi
}
}
echo "=== POST-PATCH VALIDATION: $(hostname) ===" | tee $REPORT
echo "Date: $(date)" | tee -a $REPORT
echo "Kernel: $(uname -r)" | tee -a $REPORT
echo "===== " | tee -a $REPORT
Kernel
check "Kernel loaded" "$(uname -r)"
Services
FAILED=$(systemctl list-units --state=failed --no-pager | grep -c "failed")
["$FAILED" -eq 0] && check "No failed services" "OK" || { echo ". FAIL: $FAILED failed services" | tee -a $REPORT; ((FAIL++)); }
Network
check "Default gateway reachable" "$(ping -c 1 -W 3 $(ip route show default | awk '{print $3}')) 2>/dev/null && echo OK"
```

```

DNS
check "DNS resolution" "$(nslookup $(hostname) 2>/dev/null && echo
OK)"
Disk
ROOT_USE=$(df / --output=pcent | tail -1 | tr -d ' %')
["$ROOT_USE" -lt 90] && check "Root filesystem OK (${ROOT_USE}%)"
"OK" || { echo ". FAIL: Root at ${ROOT_USE}%" | tee -a $REPORT;
((FAIL++)); }
SSH
check "SSH running" "$(systemctl is-active sshd)"
Summary
echo "===== " | tee -a
$REPORT
echo "RESULTS: $PASS passed, $FAIL failed" | tee -a $REPORT
echo "STATUS: $([$FAIL -eq 0] && echo 'ALL CHECKS PASSED' || echo
'ATTENTION NEEDED') " | tee -a $REPORT

```

·**EXPERT TIP:** Automate post-patch validation by running this script via Ansible immediately after patching. Collect results centrally and generate a consolidated dashboard showing pass/fail status for all 2000 servers.

---

## Section 12 · Troubleshooting Guide

·**BEGINNER NOTE:** This section covers 100+ real-world scenarios. Use Ctrl+F to search for your specific issue. Each scenario includes symptoms, root cause, resolution commands, and validation steps.

### 12.1 Satellite Sync & Subscription Issues (Scenarios 1-15)

#### Scenario 1: Satellite Repository Sync Fails

|       |         |       |                   |                                                                                  |                 |                                                          |
|-------|---------|-------|-------------------|----------------------------------------------------------------------------------|-----------------|----------------------------------------------------------|
| Field | Details | ----- | <b>Issue</b>      | Satellite cannot sync content from Red Hat CDN                                   | <b>Symptoms</b> | Sync task fails with "Connection refused" or "SSL error" |
|       |         |       | <b>Root Cause</b> | Network/firewall blocking CDN access, expired manifest, or SSL certificate issue | <b>Commands</b> |                                                          |

```

Check Satellite sync status
hammer sync-plan list --organization "My Org"
foreman-rake katello:sync_plans:list
Check CDN connectivity from Satellite
curl -v https://cdn.redhat.com 2>&1 | grep -E "Connected|SSL|error"
Check firewall rules
firewall-cmd --list-all
iptables -L -n | grep 443
Check manifest validity
hammer subscription list --organization "My Org"
Resolution:
1. Verify firewall allows outbound 443 to cdn.redhat.com
firewall-cmd --add-port=443/tcp --permanent
firewall-cmd --reload
2. Update SSL certificates
katello-certs-check

```

```
3. Re-upload manifest if expired
hammer subscription upload --organization "My Org" --file
/tmp/manifest.zip
4. Retry sync
hammer repository synchronize --organization "My Org" --product
"Red Hat Enterprise Linux 8 for x86_64" --name "Red Hat Enterprise
Linux 8 for x86_64 - BaseOS RPMs 8"
```

| **Validation** | hammer repository info shows "Last Sync" as current |

## Scenario 2: Host Not Receiving Patches from Satellite

| Field | Details | |-----|-----| | **Issue** | Client host shows no available updates despite Satellite having them | | **Symptoms** | yum check-update returns nothing; Satellite shows errata available | | **Root Cause** | Content View not promoted, wrong Activation Key, or host in wrong lifecycle | | **Commands** | |

```
On the host - check which repos are enabled
subscription-manager repos --list-enabled
yum repolist -v
Check Content View version on host
subscription-manager identity
cat /etc/rhsm/rhsm.conf | grep baseurl
On Satellite - verify host details
hammer host info --name $(hostname -f)
hammer host errata list --host $(hostname -f)
Resolution:
1. Check Content View promotion
hammer content-view version list --content-view "RHEL8-BaseOS"
--organization "My Org"
2. Promote if needed
hammer content-view version promote --content-view "RHEL8-BaseOS"
--version 15 --to-lifecycle-environment "Production" --organization
"My Org"
3. Refresh subscription on client
subscription-manager refresh
yum clean all
yum check-update
```

| **Validation** | yum check-update now shows available patches |

## Scenario 3: Subscription-Manager Registration Fails

| Field | Details | |-----|-----| | **Issue** | Cannot register host to Satellite | | **Symptoms** | "Unable to find available subscriptions" or "Connection refused" | | **Root Cause** | Wrong org/activation key, network issue, or Satellite service down | | **Commands** | |

```
Check Satellite services
hammer ping
On client - test connectivity
curl -k https://satellite.corp.com/katello/api/v2/ping
Clean old registration
subscription-manager unregister
subscription-manager clean
Re-register
subscription-manager register --org="My_Org"
```

```
--activationkey="ak-rhel8-prod" --force
If SSL issue:
yum install -y
http://satellite.corp.com/pub/katello-ca-consumer-latest.noarch.rpm
```

| **Validation** | subscription-manager status shows "Overall Status: Current" |

## Scenario 4: Satellite Content View Publish Hangs

| Field | Details | |-----|-----| | **Issue** | Content View publish task remains in "pending" state | | **Symptoms** | Task stuck for hours, other tasks queued behind it | | **Root Cause** | Pulp worker crashed, disk full on Satellite, or deadlock | | **Commands** |

```
Check Satellite disk space
df -h /var/lib/pulp
Pulp data can be very large (500GB+)
Check Pulp worker status
systemctl status pulpcore-worker@*
Restart Pulp workers
systemctl restart pulpcore-worker@1
systemctl restart pulpcore-worker@2
Cancel stuck task
foreman-rake foreman_tasks:cleanup AFTER=0 TASK_SEARCH='state =
running'
Or via hammer:
hammer task list --search "state = running"
hammer task cancel --id <task_id>
Restart all Satellite services
satellite-maintain service restart
```

| **Validation** | New publish task completes successfully |

## Scenario 5: "Package does not match intended download" Error

| Field | Details | |-----|-----| | **Issue** | Yum/DNF fails with checksum mismatch error | | **Symptoms** | "Downloading Packages: error: package does not match intended download" | | **Root Cause** | Corrupted package in cache, incomplete sync, or CDN mirror issue | | **Commands** |

```
Clean all caches
yum clean all
rm -rf /var/cache/yum/*
Or for dnf:
dnf clean all
Rebuild cache
yum makecache
If issue persists, re-sync repository on Satellite
hammer repository synchronize --id <repo_id>
Verify package integrity
rpm --rebuilddb
yum check
```

| **Validation** | yum update downloads and installs without errors |

## Scenario 6: Subscription Expired / Invalid

| Field | Details | |-----|-----| | **Issue** | "This system is not registered with an entitlement server" | | **Symptoms** | Cannot install packages, repos disabled | | **Root Cause** | Subscription expired, manifest not refreshed, or entitlement revoked | | **Commands** |

```
Check subscription details
subscription-manager list --consumed
subscription-manager status
Check expiration
subscription-manager list --consumed | grep -E "Ends|Status"
Refresh from Satellite
subscription-manager refresh
If expired - on Satellite:
hammer subscription refresh-manifest --organization "My Org"
Re-attach subscription
subscription-manager attach --auto
```

| **Validation** | subscription-manager status shows Current |

## Scenario 7: Capsule Server Not Syncing

| Field | Details | |-----|-----| | **Issue** | Capsule does not receive content from Satellite | | **Symptoms** | Hosts behind Capsule cannot get updates | | **Root Cause** | Sync plan not assigned, network issue between Satellite and Capsule | | **Commands** |

```
Check Capsule sync status
hammer capsule content lifecycle-environments --id <capsule_id>
hammer capsule content synchronize --id <capsule_id>
Verify Capsule services
satellite-maintain service status # On Capsule
Check connectivity Satellite . Capsule
curl -k https://capsule.corp.com:9090/features
Force content sync
hammer capsule content synchronize --id <capsule_id>
--lifecycle-environment-id <env_id>
```

| **Validation** | hammer capsule content info shows recent sync |

## Scenario 8: "Cannot retrieve metalink for repository"

| Field | Details | |-----|-----| | **Issue** | Yum fails with metalink/mirrorlist errors | | **Symptoms** | "Cannot retrieve metalink for repository. Please verify its path" | | **Root Cause** | DNS failure, network issue, or wrong repo configuration | | **Commands** |

```
Check DNS
cat /etc/resolv.conf
nslookup satellite.corp.com
Check network
ping satellite.corp.com
curl -k https://satellite.corp.com/pulp/repos/
Check repo configuration
cat /etc/yum.repos.d/redhat.repo | head -20
Fix: ensure pointed to Satellite not CDN
subscription-manager config
--rhsm.baseurl=https://satellite.corp.com/pulp/repos
Clean and retry
```

```
yum clean all
yum repolist
```

| **Validation** | yum repolist shows repositories successfully |

## 12.2 YUM/DNF Package Management Issues (Scenarios 9-25)

### Scenario 9: "Multilib version problems"

| Field | Details | |-----|-----| | **Issue** | YUM fails with multilib version mismatch | | **Symptoms** | "Protected multilib versions: package-x.x.x-1.el8.x86\_64 != package-x.x.x-2.el8.i686" | | **Root Cause** | Mixed versions of 32-bit and 64-bit packages | | **Commands** | |

```
Identify the conflicting packages
yum list installed | grep <package_name>
Fix by updating both architectures
yum update <package_name>.x86_64 <package_name>.i686
Or remove the 32-bit package if not needed
yum remove <package_name>.i686
Force if needed
yum update --setopt=protected_multilib=false
```

| **Validation** | yum update completes without multilib errors |

### Scenario 10: "Transaction check error" - File Conflicts

| Field | Details | |-----|-----| | **Issue** | Package installation fails due to file conflicts | | **Symptoms** | "file /usr/lib64/libfoo.so from install of pkg-A conflicts with file from pkg-B" | | **Root Cause** | Two packages claim ownership of same file | | **Commands** | |

```
Identify which packages own the file
rpm -qf /usr/lib64/libfoo.so
Check package details
rpm -qi <package_name>
Resolution options:
Option 1: Remove conflicting package
yum remove <conflicting_package>
yum update
Option 2: Force (use with caution)
rpm -ivh --force <package>.rpm
Option 3: Use --allowerase (DNF)
dnf update --allowerase
```

| **Validation** | rpm -V <package> shows no verification errors |

### Scenario 11: "Depsolve Error" - Dependency Resolution Failure

| Field | Details | |-----|-----| | **Issue** | Cannot resolve package dependencies | | **Symptoms** | "Error: Package: pkg-1.0 requires libfoo >= 2.0, but libfoo-1.5 is installed" | | **Root Cause** | Missing repository, version lock, or broken dependency chain | | **Commands** | |

```
Show detailed dependency tree
yum deplist <package_name>
Or:
dnf repoquery --requires <package_name>
```

```
Check if required package is in any repo
yum provides "libfoo >= 2.0"
dnf provides "libfoo >= 2.0"
Check for version locks
cat /etc/yum/pluginconf.d/versionlock.list
yum versionlock list
Resolution:
1. Enable missing repository
subscription-manager repos
--enable=rhel-8-for-x86_64-appstream-rpms
2. Remove version lock if applicable
yum versionlock delete <package_name>
3. Clean metadata and retry
yum clean metadata
yum update
```

| **Validation** | yum update resolves dependencies and installs |

## Scenario 12: YUM/DNF Transaction Fails Midway

| Field | Details | |-----|-----| | **Issue** | Patch installation interrupted (power loss, disk full, kill) | | **Symptoms** | Partial install, RPM database inconsistent, "rpmdb: damaged" | | **Root Cause** | Interrupted transaction left system in inconsistent state | | **Commands** |

```
Check for incomplete transactions
yum history list | head -5
yum-complete-transaction # RHEL 7
dnf history list | head -5
Check RPM database
rpm --rebuilddb
Verify package integrity
rpm -Va | grep -v "^..5" | head -20
Complete or rollback the transaction
yum-complete-transaction --cleanup-only # RHEL 7
DNF (RHEL 8/9):
dnf history undo last
If RPM DB is severely corrupted:
rm -f /var/lib/rpm/__db*
rpm --rebuilddb
yum clean all
yum makecache
```

| **Validation** | rpm -qa lists packages correctly, yum check passes |

## Scenario 13: "Disk space required on /boot exceeds available"

| Field | Details | |-----|-----| | **Issue** | Kernel update fails due to full /boot partition | | **Symptoms** | "Disk Requirements: At least XXXMB more space needed on the /boot filesystem" | | **Root Cause** | Old kernels filling /boot (typically 500MB-1GB partition) | | **Commands** |

```
Check /boot usage
df -h /boot
ls -la /boot/vmlinuz-* | wc -l
List installed kernels
rpm -qa kernel | sort -V
```

```
Remove old kernels (keep last 2)
RHEL 7:
package-cleanup --oldkernels --count=2
RHEL 8/9:
dnf remove --oldinstallonly --setopt installonly_limit=2 kernel
Manual removal (if above fails):
Identify current running kernel
uname -r
Remove specific old kernel (NOT the running one!)
yum remove kernel-4.18.0-240.el8.x86_64
Also clean rescue images if present
ls -la /boot/vmlinuz-0-rescue-*
rm -f /boot/vmlinuz-0-rescue-* /boot/initramfs-0-rescue-*
Rebuild GRUB
grub2-mkconfig -o /boot/grub2/grub.cfg
```

| **Validation** | df -h /boot shows sufficient free space, kernel installs |

#### Scenario 14: "GPG key retrieval failed"

| Field | Details | |-----|-----| | **Issue** | Package installation fails GPG verification | | **Symptoms** | "GPG key retrieval failed: [Errno 14] HTTP Error 404" | | **Root Cause** | GPG key not imported, wrong key URL, or gpgcheck misconfigured | | **Commands** |

```
Import Red Hat GPG key
rpm --import /etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-release
Or from Satellite:
rpm --import

https://satellite.corp.com/katello/api/v2/repositories/gpg_key_content
List imported keys
rpm -qa gpg-pubkey*
Temporary workaround (not recommended for production):
yum update --nogpgcheck
Fix repo configuration
cat /etc/yum.repos.d/redhat.repo | grep gpgkey
Ensure GPG key path is correct
```

| **Validation** | yum update installs packages without GPG errors |

#### Scenario 15: "Another app is currently holding the yum lock"

| Field | Details | |-----|-----| | **Issue** | Cannot run yum/dnf because lock file exists | | **Symptoms** | "Another app is currently holding the yum lock; waiting for it to exit..." | | **Root Cause** | Previous yum process crashed or still running | | **Commands** |

```
Check for running yum/dnf processes
ps aux | grep -E "yum|dnf" | grep -v grep
If legitimate process running, wait for it
If stale lock:
kill $(cat /var/run/yum.pid 2>/dev/null)
Remove lock file
rm -f /var/run/yum.pid
rm -f /var/cache/dnf/metadata_lock.pid
RHEL 8/9 (DNF):
```

```
rm -f /var/cache/dnf/*.lock
Verify
yum repolist
```

| **Validation** | yum update runs without lock errors |

## Scenario 16: Package Rollback Fails

| Field | Details | |-----|-----| | **Issue** | yum history undo fails to rollback a transaction | | **Symptoms** | "Cannot rollback - some packages were not found" | | **Root Cause** | Old package version no longer in repository | | **Commands** | |

```
Check transaction details
yum history info <transaction_id>
Check if old version still available
yum list --showduplicates <package_name> | sort -V
If old version not in repo, download from Satellite:
yumdownloader <package_name>-<old_version>
rpm -Uvh --oldpackage <package_name>-<old_version>.rpm
Or use yum downgrade:
yum downgrade <package_name>-<old_version>
```

| **Validation** | rpm -q <package> shows old version |

## Scenario 17: DNF Module Stream Conflicts

| Field | Details | |-----|-----| | **Issue** | Module stream conflicts prevent package update | | **Symptoms** | "Modular dependency problems" | | **Root Cause** | Conflicting module streams enabled (e.g., PHP 7.4 vs 8.0) | | **Commands** | |

```
List module streams
dnf module list
dnf module list --enabled
Check current stream
dnf module info <module_name>
Reset module
dnf module reset <module_name>
Switch stream
dnf module switch-to <module_name>:<stream>
Example: Switch PHP from 7.4 to 8.0
dnf module reset php
dnf module enable php:8.0
dnf update
```

| **Validation** | dnf module list --enabled shows correct streams |

## Scenario 18: "Error unpacking rpm package"

| Field | Details | |-----|-----| | **Issue** | RPM package extraction fails during installation | | **Symptoms** | "error: unpacking of archive failed on file /path: cpio: write" | | **Root Cause** | Disk full, corrupted download, or filesystem error | | **Commands** | |

```
Check disk space
df -h
Clean yum cache and re-download
yum clean packages
```

```

yum update
Check filesystem for errors (unmount first if possible)
xfs_repair /dev/mapper/vg-root # XFS
fsck -y /dev/mapper/vg-root # ext4 (requires unmount)
If disk full - free space
yum clean all
rm -rf /var/cache/yum/*
journalctl --vacuum-size=100M

```

| **Validation** | Packages install without unpack errors |

## Scenario 19: Yum Update Extremely Slow

| Field | Details | |-----|-----| | **Issue** | Yum/DNF takes very long to check/download updates | | **Symptoms** | Metadata download hangs, progress bar stuck | | **Root Cause** | Slow network to repo, too many repos, or large metadata | | **Commands** |

```

Check network speed to Satellite
curl -o /dev/null -w "Speed: %{speed_download} bytes/sec\n"

https://satellite.corp.com/pub/katello-ca-consumer-latest.noarch.rpm
Reduce metadata download time
yum makecache fast
Or use closest Capsule server
Disable unnecessary repos
yum repolist enabled
subscription-manager repos --disable=<unused_repo>
Set timeout in yum.conf
echo "timeout=30" >> /etc/yum.conf
echo "retries=3" >> /etc/yum.conf
Use fastest mirror plugin
yum install yum-plugin-fastestmirror

```

| **Validation** | yum check-update completes in reasonable time |

## Scenario 20: "Protected packages" Cannot Be Updated

| Field | Details | |-----|-----| | **Issue** | Critical packages marked as protected cannot be updated | | **Symptoms** | "Trying to remove protected package" | | **Root Cause** | Package listed in protected\_packages or installonlypkgs | | **Commands** |

```

Check protected packages
cat /etc/yum.conf | grep -E "protected|exclude"
cat /etc/dnf/dnf.conf | grep -E "protected|exclude"
cat /etc/yum/pluginconf.d/versionlock.list 2>/dev/null
Remove from protected list if intentional
Edit /etc/yum.conf and remove from exclude= line
Or override temporarily
yum update --disableexcludes=all
For versionlock:
yum versionlock delete <package_name>
yum update <package_name>
yum versionlock add <package_name> # Re-lock if needed

```

| **Validation** | Package updates successfully |

## Scenario 21: RHEL 6 - Yum Update with TLS Issues

| Field | Details | |-----|-----| | **Issue** | RHEL 6 cannot connect to repositories (TLS 1.0/1.1 deprecated) | |  
| **Symptoms** | "SSL connection error" or "NSS error" | | **Root Cause** | RHEL 6 uses old TLS versions, repos require TLS 1.2+ | | **Commands** | |

```
Check NSS/SSL version
rpm -q nss openssl
Update NSS and curl packages (if available)
yum update nss curl
Set minimum TLS version
Add to /etc/yum.conf:
sslverify=1
sslcacert=/etc/pki/tls/certs/ca-bundle.crt
If connecting to internal Satellite with modern TLS:
Ensure Satellite allows TLS 1.0 for RHEL 6 clients (not
recommended)
Better: Upgrade RHEL 6 to 7/8/9
```

| **Validation** | yum repolist connects successfully |

## Scenario 22: "Rpmdb checksum is invalid"

| Field | Details | |-----|-----| | **Issue** | RPM database corruption | | **Symptoms** | "rpmdb: BDB0113 Thread/process failed: Thread died in Berkeley DB library" | | **Root Cause** | System crash during package transaction, disk errors | |  
**Commands** | |

```
Backup existing RPM database
cp -a /var/lib/rpm /var/lib/rpm.backup
Remove stale lock files
rm -f /var/lib/rpm/__db*
Rebuild database
rpm --rebuilddb
Verify
rpm -qa | wc -l
yum check
If still failing:
rm -f /var/lib/rpm/__db*
db_verify /var/lib/rpm/Packages
rpm --rebuilddb
```

| **Validation** | rpm -qa and yum check both succeed |

## Scenario 23: Exclude Packages Not Working

| Field | Details | |-----|-----| | **Issue** | Packages being updated despite exclude configuration | | **Symptoms** | Excluded packages updated during yum update | | **Root Cause** | Exclude syntax wrong, wrong config file, or --disableexcludes used | | **Commands** | |

```
Check exclude configuration
grep exclude /etc/yum.conf
grep exclude /etc/dnf/dnf.conf
Correct syntax:
In /etc/yum.conf or /etc/dnf/dnf.conf under [main]:
exclude=kernel* java* postgresql*
Verify excludes are active:
```

```

yum update --assumeno 2>&1 | grep -i "exclude"
dnf update --assumeno 2>&1 | grep -i "excluded"
Per-repo exclude (in repo file):
[rhel-8-baseos]
exclude=kernel*

```

| **Validation** | yum update --assumeno does not list excluded packages |

## Scenario 24: "Package not found" but exists in Satellite

| Field | Details | |-----|-----| | **Issue** | Package available in Satellite but not visible on host | | **Symptoms** | "No match for argument: package-name" | | **Root Cause** | Package not in Content View, not promoted, or repo not enabled | | **Commands** |

```

Check on host which repos are enabled
yum repolist enabled -v
Search for package
yum search <package_name>
dnf provides <package_name>
Check on Satellite
hammer package list --organization "My Org" --search "name ~
<package>"
Resolution:
1. Add repository to Content View
hammer content-view add-repository --name "RHEL8-CV"
--repository-id <id> --organization "My Org"
2. Republish Content View
hammer content-view publish --name "RHEL8-CV" --organization "My
Org"
3. Promote to environment
hammer content-view version promote --content-view "RHEL8-CV"
--version <ver> --to-lifecycle-environment "Production"
--organization "My Org"
4. On host:
yum clean all
yum makecache
yum install <package_name>

```

| **Validation** | yum install <package> succeeds |

## Scenario 25: YUM runs out of memory (OOM killed)

| Field | Details | |-----|-----| | **Issue** | Yum/DNF process killed by OOM during large update | | **Symptoms** | "Killed" message, dmesg shows OOM | | **Root Cause** | Large metadata processing on low-memory system | | **Commands** |

```

Check for OOM events
dmesg | grep -i "oom\|killed"
journalctl | grep "Out of memory"
Reduce memory usage:
1. Disable unused repos to reduce metadata
subscription-manager repos --disable=<unused_repo>
2. Update in smaller batches
yum update --security -y # Fewer packages
yum update <specific_packages>

```

```
3. Add swap temporarily
dd if=/dev/zero of=/tmp/swapfile bs=1M count=2048
mkswap /tmp/swapfile
swapon /tmp/swapfile
4. After successful update:
swapoff /tmp/swapfile
rm -f /tmp/swapfile
```

| **Validation** | yum update completes without OOM |

## 12.3 Kernel & Boot Issues (Scenarios 26-40)

### Scenario 26: Kernel Panic After Patch

| Field | Details | |-----|-----| | **Issue** | Server kernel panics immediately after booting new kernel | | **Symptoms** | Console shows "Kernel panic - not syncing" | | **Root Cause** | Incompatible driver, corrupted initramfs, or hardware incompatibility | | **Commands** |

```
Boot into previous kernel:
1. At GRUB menu, select previous kernel entry
2. Or interrupt boot with 'e', change kernel line
Once booted on old kernel:
uname -r # Confirm old kernel
Check initramfs for new kernel
lsinitrd /boot/initramfs-<new_kernel>.img | grep -i "driver_name"
Rebuild initramfs
dracut -f /boot/initramfs-<new_kernel>.img <new_kernel_version>
If driver issue, blacklist problematic module
echo "blacklist <module_name>" >
/etc/modprobe.d/blacklist-patch.conf
dracut -f
Set old kernel as default (safety)
grubby --set-default /boot/vmlinuz-<old_kernel_version>
Try booting new kernel again
reboot
```

| **Validation** | System boots without panic on new kernel |

### Scenario 27: Server Stuck at GRUB Prompt

| Field | Details | |-----|-----| | **Issue** | Server drops to grub> or grub rescue> prompt | | **Symptoms** | No OS menu, just grub prompt | | **Root Cause** | GRUB configuration corrupted, missing grub.cfg | | **Commands** |

```
At grub rescue prompt:
grub rescue> set prefix=(hd0,msdos1)/grub2
grub rescue> set root=(hd0,msdos1)
grub rescue> insmod normal
grub rescue> normal
Once booted (or from rescue mode):
Reinstall GRUB (BIOS):
grub2-install /dev/sda
grub2-mkconfig -o /boot/grub2/grub.cfg
Reinstall GRUB (UEFI):
grub2-mkconfig -o /boot/efi/EFI/redhat/grub.cfg
```

```
dnf reinstall grub2-efi-x64 shim-x64
Verify
grubby --info=ALL
grubby --default-kernel
```

| **Validation** | Server boots normally from GRUB |

## Scenario 28: initramfs Missing or Corrupted

| Field | Details | |-----|-----| | **Issue** | Boot fails with "Cannot find root device" or dracut emergency shell | | **Symptoms** | Drops to dracut emergency shell, cannot mount root | | **Root Cause** | initramfs not generated for new kernel, or corrupted | | **Commands** | |

```
From dracut emergency shell or rescue boot:
Check if initramfs exists
ls -la /boot/initramfs-*
Regenerate initramfs for specific kernel
dracut -f /boot/initramfs-$(uname -r).img $(uname -r)
Or regenerate for all kernels
dracut -f --regenerate-all
If root device issue, check dracut modules
dracut --list-modules
lsinitrd /boot/initramfs-$(uname -r).img | grep -E "lvm|dm|sd"
Ensure LVM/dm modules are included
dracut -f --add "lvm dm" /boot/initramfs-$(uname -r).img $(uname -r)
```

| **Validation** | System boots without dropping to emergency shell |

## Scenario 29: Wrong Default Kernel After Update

| Field | Details | |-----|-----| | **Issue** | System boots into wrong kernel version | | **Symptoms** | `uname -r` shows old kernel after reboot | | **Root Cause** | GRUB default not updated, or `GRUB_DEFAULT=saved` not set | | **Commands** | |

```
Check current default
grubby --default-kernel
grub2-editenv list
List all available kernels
grubby --info=ALL | grep -E "^index|^kernel"
Set correct default
grubby --set-default /boot/vmlinuz-<desired_kernel>
Or set by index (0 = first entry)
grub2-set-default 0
Verify
grub2-editenv list
Expected: saved_entry=<correct_kernel>
Ensure /etc/default/grub has:
GRUB_DEFAULT=saved
grep GRUB_DEFAULT /etc/default/grub
```

| **Validation** | After reboot, `uname -r` shows expected kernel |

## Scenario 30: Boot Hangs at "Starting Switch Root"

| Field | Details | |-----|-----| | **Issue** | Boot process hangs after loading initramfs | | **Symptoms** | Last message on console: "Starting Switch Root..." then nothing | | **Root Cause** | Root filesystem cannot be found/mounted, LVM issue | | **Commands** | |

```
Boot with debug:
At GRUB, add to kernel line: rd.break rd.shell
In dracut shell:
dracut# lvm vgscan
dracut# lvm vgchange -ay
dracut# mount /dev/mapper/vg-root /sysroot
If successful, exit and boot continues
If not, check fstab:
dracut# cat /sysroot/etc/fstab
Ensure root device matches actual LVM/device names
Fix if UUID changed:
dracut# blkid
Update fstab with correct UUID
```

| **Validation** | System completes boot process fully |

## Scenario 31: Kernel Module Not Loading After Update

| Field | Details | |-----|-----| | **Issue** | Third-party kernel module (GPU, storage) fails to load | | **Symptoms** | Hardware not detected, dmesg shows "module not found" | | **Root Cause** | Module compiled for old kernel, not rebuilt for new kernel | | **Commands** | |

```
Check if module exists for new kernel
find /lib/modules/$(uname -r) -name "<module_name>.ko*"
Check dkms status (if using dkms)
dkms status
Rebuild module via dkms
dkms autoinstall -k $(uname -r)
Or:
dkms install <module_name>/<version> -k $(uname -r)
If no dkms, rebuild from source
cd /usr/src/<module_source>
make clean
make KDIR=/lib/modules/$(uname -r)/build
make install
Load module
modprobe <module_name>
lsmod | grep <module_name>
```

| **Validation** | `lsmod` shows module loaded, hardware functional |

## Scenario 32: SELinux Denials After Kernel Update

| Field | Details | |-----|-----| | **Issue** | Applications fail with permission denied after kernel/policy update | | **Symptoms** | AVC denied messages in audit.log, services fail to start | | **Root Cause** | New SELinux policy or context changes from update | | **Commands** | |

```
Check for denials
ausearch -m AVC --start recent
sealert -a /var/log/audit/audit.log | head -50
Quick fix - set to permissive (temporary)
```

```

setenforce 0
Test if issue resolves
Proper fix - generate and apply policy module
audit2allow -a -M mypatch_fix
semodule -i mypatch_fix.pp
Restore file contexts
restorecon -Rv /affected/path
Relabel entire filesystem (if many issues)
touch /.autorelabel
reboot
WARNING: Relabel can take 30+ minutes on large filesystems

```

| **Validation** | ausearch -m AVC --start recent shows no new denials |

### Scenario 33: Server Runs Out of Memory During Boot After Patch

| Field | Details | |-----|-----| | **Issue** | New services or drivers consume too much memory at boot | | **Symptoms** | OOM killer activates during boot, services killed | | **Root Cause** | New kernel memory overhead, or services started that shouldn't be | | **Commands** | |

```

Boot with limited services:
Add to kernel line: systemd.unit=multi-user.target (no GUI)
Check what's consuming memory
ps aux --sort=-%mem | head -20
Disable unnecessary services
systemctl disable <unnecessary_service>
Check for memory-hungry new services
systemctl list-units --type=service --state=running | grep -v -E
"sshd|crond|rsyslog"
Increase swap if needed
fallocate -l 4G /swapfile
chmod 600 /swapfile
mkswap /swapfile
swapon /swapfile
echo '/swapfile swap swap defaults 0 0' >> /etc/fstab

```

| **Validation** | System boots without OOM events |

### Scenario 34: Firmware Update Causes Hardware Issues

| Field | Details | |-----|-----| | **Issue** | NIC, HBA, or RAID controller malfunction after firmware update | | **Symptoms** | Network interfaces down, storage not visible, hardware errors | | **Root Cause** | Firmware incompatibility or failed firmware flash | | **Commands** | |

```

Check hardware status
lspci -v | grep -i "controller\|ethernet"
dmesg | grep -i "firmware\|error"
Check NIC firmware
ethtool -i eth0 | grep firmware
For Dell servers:
racadm getversion
For HP/HPE:
hponcfg -g
Rollback firmware (vendor-specific):
Dell: Use iDRAC/Lifecycle Controller to rollback

```

```
HP: Use iLO/SPP to rollback
If NIC firmware issue:
Use alternative NIC or console access
Download previous firmware from vendor
Flash with vendor tool (e.g., ethtool, fwupd)
fwupdmgr get-history
fwupdmgr downgrade <device_id>
```

| **Validation** | Hardware functional, no errors in dmesg |

## Scenario 35: kpatch/Livepatch Fails to Apply

| Field | Details | |-----|-----| | **Issue** | Kernel live patch cannot be applied | | **Symptoms** | "kpatch: module verification failed" or "patch conflicts" | | **Root Cause** | Kernel version mismatch, or conflicting patches | | **Commands** |

```
Check kpatch status
kpatch list
Check kernel version compatibility
uname -r
rpm -qa kpatch-patch*
Remove conflicting patch
kpatch unload <patch_module>
Install correct patch
yum install kpatch-patch-$(uname -r)
Apply manually
kpatch load /var/lib/kpatch/$(uname -r)/<patch>.ko
If livepatch not available, schedule traditional kernel update
```

| **Validation** | kpatch list shows patch loaded and enabled |

## 12.4 Network & DNS Issues (Scenarios 36-50)

### Scenario 36: Network Interface Down After Reboot

| Field | Details | |-----|-----| | **Issue** | Network interface does not come up after patching reboot | | **Symptoms** | No IP address, cannot SSH, server unreachable | | **Root Cause** | NetworkManager update changed config, interface renamed, or driver issue | | **Commands** |

```
From console/iLO/iDRAC:
Check interface status
ip link show
nmcli device status
Check if driver loaded
dmesg | grep -i "eth\|ens\|eno"
lspci -v | grep -i ethernet
Check NetworkManager
systemctl status NetworkManager
nmcli connection show
Bring interface up
nmcli connection up <connection_name>
Or:
ifup <interface> # RHEL 7
If interface renamed (e.g., eth0 -> ens192):
```

```
Check new name
ip link show
Update connection to use new name
nmcli connection modify <conn> connection.interface-name <new_name>
nmcli connection up <conn>
If driver issue:
modprobe <driver_name>
e.g., modprobe e1000e, modprobe ixgbe
```

| **Validation** | ip addr show displays correct IP, ping gateway succeeds |

## Scenario 37: DNS Resolution Broken After Patch

| Field | Details | |-----|-----| | **Issue** | Cannot resolve hostnames after patching | | **Symptoms** | nslookup/dig fail, applications cannot connect by hostname | | **Root Cause** | resolv.conf overwritten, NetworkManager changed DNS settings | | **Commands** |

```
Check resolv.conf
cat /etc/resolv.conf
If empty or wrong nameservers:
Check NetworkManager DNS config
nmcli connection show <conn> | grep dns
Restore DNS settings
nmcli connection modify <conn> ipv4.dns "10.0.0.53 10.0.0.54"
nmcli connection modify <conn> ipv4.dns-search "corp.com"
nmcli connection up <conn>
Or manually fix resolv.conf (temporary):
echo "nameserver 10.0.0.53" > /etc/resolv.conf
echo "nameserver 10.0.0.54" >> /etc/resolv.conf
Prevent NM from overwriting resolv.conf:
In /etc/NetworkManager/NetworkManager.conf under [main]:
dns=none
systemctl restart NetworkManager
```

| **Validation** | nslookup \$(hostname) resolves correctly |

## Scenario 38: Static Routes Lost After Reboot

| Field | Details | |-----|-----| | **Issue** | Custom static routes missing after reboot | | **Symptoms** | Cannot reach certain subnets, routing table incomplete | | **Root Cause** | Route configuration not persistent, or NM update reset routes | | **Commands** |

```
Check current routes
ip route show
RHEL 7 - Check route files
cat /etc/sysconfig/network-scripts/route-<interface>
RHEL 8/9 - Check NetworkManager routes
nmcli connection show <conn> | grep route
Add persistent routes (RHEL 8/9):
nmcli connection modify <conn> +ipv4.routes "172.16.0.0/16
10.0.0.1"
nmcli connection modify <conn> +ipv4.routes "192.168.0.0/16
10.0.0.1"
nmcli connection up <conn>
Verify
```

```
ip route show | grep "172.16\|192.168"
```

| **Validation** | ip route show displays all required routes after reboot |

### Scenario 39: Firewall Rules Reset After Update

| Field | Details | |-----|-----| | **Issue** | Custom firewall rules lost after firewalld update | | **Symptoms** | Applications inaccessible, ports blocked | | **Root Cause** | firewalld update reset to default zone configuration | | **Commands** | |

```
Check current firewall status
firewall-cmd --state
firewall-cmd --list-all
Check if custom rules preserved
firewall-cmd --permanent --list-all
If rules lost, re-apply from backup:
Re-add ports
firewall-cmd --permanent --add-port=8080/tcp
firewall-cmd --permanent --add-port=443/tcp
firewall-cmd --permanent --add-service=http
Reload
firewall-cmd --reload
firewall-cmd --list-all
Verify services can connect
ss -tuln | grep <port>
```

| **Validation** | firewall-cmd --list-all shows expected rules |

### Scenario 40: Bond/Team Interface Degraded After Patch

| Field | Details | |-----|-----| | **Issue** | Network bond/team loses one or more slave interfaces | | **Symptoms** | Degraded bond, single path only, possible performance issues | | **Root Cause** | NIC driver update, interface rename, or config mismatch | | **Commands** | |

```
Check bond status
cat /proc/net/bonding/bond0
Look for: "MII Status: up" for all slaves
Or for team:
teamctl team0 state
If slave is down:
Check physical link
ethtool <slave_interface> | grep "Link detected"
Bring slave back into bond
ifenslave bond0 <slave_interface>
Or:
nmcli connection up <slave_connection>
If driver issue after update:
modprobe -r <driver>
modprobe <driver>
Verify bond is healthy
cat /proc/net/bonding/bond0 | grep -E "Status|Interface"
```

| **Validation** | Bond shows all slaves "up", full bandwidth available |

### Scenario 41: NTP/Chrony Time Sync Lost

| Field | Details | |-----|-----| | **Issue** | System time drifts after chrony/NTP update | | **Symptoms** | Time offset, Kerberos auth fails, log timestamps wrong | | **Root Cause** | Chrony config changed, NTP servers unreachable after update | | **Commands** |

```
Check chrony status
chronyc sources -v
chronyc tracking
If not synced:
systemctl restart chronyd
chronyc makestep
Check configuration
cat /etc/chrony.conf | grep "^server\|^pool"
Verify NTP server reachable
chronyc sources
'*' = current sync source, '+' = acceptable, '?' = not reachable
Force sync
chronyc -a makestep
```

| **Validation** | chronyc tracking shows "Leap status: Normal" |

## Scenario 42: SSH Connection Refused After Patch

| Field | Details | |-----|-----| | **Issue** | Cannot SSH to server after patching | | **Symptoms** | "Connection refused" on port 22 | | **Root Cause** | sshd failed to start, port changed, or firewall blocking | | **Commands** |

```
From console:
Check sshd status
systemctl status sshd
journalctl -u sshd --since "5 minutes ago"
Common causes:
1. Config syntax error
sshd -t
Fix any config errors in /etc/ssh/sshd_config
2. Host keys regenerated
ls -la /etc/ssh/ssh_host_*
3. Port changed
grep "^Port" /etc/ssh/sshd_config
4. Firewall blocking
firewall-cmd --list-all | grep ssh
Restart sshd
systemctl restart sshd
systemctl enable sshd
```

| **Validation** | SSH connection succeeds from remote host |

## Scenario 43: IPv6 Issues After Network Update

| Field | Details | |-----|-----| | **Issue** | IPv6 connectivity broken after NetworkManager update | | **Symptoms** | IPv6 addresses missing, AAAA resolution fails | | **Root Cause** | IPv6 method changed to "disabled" or SLAAC broken | | **Commands** |

```
Check IPv6 status
ip -6 addr show
cat /proc/sys/net/ipv6/conf/all/disable_ipv6
If IPv6 disabled unintentionally:
```

```

sysctl -w net.ipv6.conf.all.disable_ipv6=0
Check NetworkManager IPv6 settings
nmcli connection show <conn> | grep ipv6
Re-enable IPv6
nmcli connection modify <conn> ipv6.method auto
nmcli connection up <conn>

```

| **Validation** | ip -6 addr show displays expected IPv6 addresses |

## Scenario 44: MTU Changed After Network Update

| Field | Details | |-----|-----| | **Issue** | Network performance degraded, packet fragmentation | | **Symptoms** | Slow transfers, jumbo frames not working | | **Root Cause** | MTU reset to default 1500 after NM update | | **Commands** | |

```

Check current MTU
ip link show | grep mtu
Set MTU back to expected value
nmcli connection modify <conn> 802-3-ethernet.mtu 9000
nmcli connection up <conn>
Verify
ip link show <interface> | grep mtu
Test with ping
ping -M do -s 8972 <remote_host> # 8972 + 28 header = 9000

```

| **Validation** | ip link show confirms correct MTU |

## Scenario 45: VLAN Interfaces Missing After Reboot

| Field | Details | |-----|-----| | **Issue** | VLAN tagged interfaces not created after reboot | | **Symptoms** | VLAN IPs missing, cannot reach VLAN networks | | **Root Cause** | 8021q module not loading, or NM VLAN config lost | | **Commands** | |

```

Check if 8021q module loaded
lsmod | grep 8021q
Load module
modprobe 8021q
echo "8021q" > /etc/modules-load.d/8021q.conf
Check VLAN config
nmcli connection show | grep vlan
Recreate VLAN if needed
nmcli connection add type vlan con-name vlan100 dev eth0 id 100
nmcli connection modify vlan100 ipv4.addresses 10.100.0.50/24
nmcli connection modify vlan100 ipv4.method manual
nmcli connection up vlan100

```

| **Validation** | ip addr show displays VLAN interface with correct IP |

## Scenario 46: Proxy Settings Lost After Update

| Field | Details | |-----|-----| | **Issue** | Server cannot reach internet/repos through proxy after update | | **Symptoms** | Connection timeouts to external repos | | **Root Cause** | yum.conf or environment proxy settings cleared | | **Commands** | |

```

Check yum proxy settings
grep proxy /etc/yum.conf

```

```

grep proxy /etc/dnf/dnf.conf
Add proxy back
echo "proxy=http://proxy.corp.com:8080" >> /etc/yum.conf
Check environment proxy
echo $http_proxy $https_proxy
Set system-wide proxy
cat > /etc/profile.d/proxy.sh << 'EOF'
export http_proxy="http://proxy.corp.com:8080"
export https_proxy="http://proxy.corp.com:8080"
export no_proxy="localhost,127.0.0.1,.corp.com"
EOF
source /etc/profile.d/proxy.sh

```

| **Validation** | yum repolist connects through proxy successfully |

## Scenario 47: Network Bridge Broken After Update

| Field | Details | |-----|-----| | **Issue** | Bridge interface (br0) not functioning for VMs | | **Symptoms** | VMs lose network, bridge shows no slave ports | | **Root Cause** | Bridge configuration cleared by NM update | | **Commands** | |

```

Check bridge status
brctl show
Or:
bridge link show
Check NM bridge config
nmcli connection show | grep bridge
Recreate bridge if needed
nmcli connection add type bridge con-name br0 ifname br0
nmcli connection add type bridge-slave con-name br0-slave ifname
eth0 master br0
nmcli connection modify br0 ipv4.addresses 10.0.0.50/24
nmcli connection modify br0 ipv4.gateway 10.0.0.1
nmcli connection modify br0 ipv4.method manual
nmcli connection up br0

```

| **Validation** | brctl show shows bridge with slave interfaces |

## Scenario 48: Multipath Storage Paths Down

| Field | Details | |-----|-----| | **Issue** | SAN multipath shows dead paths after reboot | | **Symptoms** | I/O errors, degraded multipath, dm devices missing | | **Root Cause** | multipathd service not started, HBA driver issue | | **Commands** | |

```

Check multipath status
multipath -ll
Look for [failed][faulty] paths
Check multipathd service
systemctl status multipathd
Restart multipathd
systemctl restart multipathd
multipath -r # Reconfigure
Check HBA driver
lsmod | grep -E "qla2xxx|lpfc"
dmesg | grep -i "scsi\|fibre\|link"
Rescan SCSI bus

```

```
echo "- - -" > /sys/class/scsi_host/host0/scan
echo "- - -" > /sys/class/scsi_host/host1/scan
Verify paths recovered
multipath -ll
All paths should show [active][ready]
```

| **Validation** | multipath -ll shows all paths active/ready |

## Scenario 49: Network Performance Degraded After Patch

| Field | Details | |-----|-----| | **Issue** | Network throughput significantly lower after patching | | **Symptoms** | Slow file transfers, high latency, packet drops | | **Root Cause** | NIC offload settings changed, ring buffer reset, or driver regression | | **Commands** | |

```
Check NIC settings
ethtool <interface>
ethtool -k <interface> # Offload settings
ethtool -g <interface> # Ring buffer
Restore offload settings
ethtool -K <interface> rx on tx on gso on gro on tso on
Restore ring buffers
ethtool -G <interface> rx 4096 tx 4096
Check for packet errors
ethtool -S <interface> | grep -i "error\|drop\|miss"
Check driver version
ethtool -i <interface>
Compare with pre-patch version
```

| **Validation** | Network throughput matches pre-patch baseline |

## Scenario 50: Hostname Changed After NetworkManager Update

| Field | Details | |-----|-----| | **Issue** | Server hostname reverted to DHCP-provided name or "localhost" | | **Symptoms** | hostname shows wrong name, applications fail identity checks | | **Root Cause** | NetworkManager overrode static hostname from DHCP | | **Commands** | |

```
Check current hostname
hostname
hostnamectl status
Set static hostname (persistent)
hostnamectl set-hostname webserver01.corp.com
Prevent NM from changing hostname
In /etc/NetworkManager/NetworkManager.conf:
[main]
hostname-mode=none
Or per-connection:
nmcli connection modify <conn> ipv4.dhcp-hostname ""
nmcli connection modify <conn> ipv4.dhcp-send-hostname no
systemctl restart NetworkManager
hostname # Verify
```

| **Validation** | hostname -f shows correct FQDN after reboot |

## 12.5 Filesystem, Storage & LVM Issues (Scenarios 51-65)

## Scenario 51: Filesystem Mounted Read-Only After Reboot

| Field | Details | |-----|-----| | **Issue** | Filesystem remounted read-only after patching | | **Symptoms** | "Read-only file system" errors, cannot write files | | **Root Cause** | Disk errors detected, fsck required, or fstab error | | **Commands** | |

```
Check mount status
mount | grep " ro,"
Check dmesg for filesystem errors
dmesg | grep -i "ext4\|xfs\|error\|readonly"
Remount read-write (temporary fix)
mount -o remount,rw /
For XFS - check filesystem
xfs_repair -n /dev/mapper/vg-root # Check only (no fix)
For ext4:
Must unmount first (use rescue mode for root)
fsck -y /dev/mapper/vg-root
Check fstab for errors
cat /etc/fstab
findmnt --verify
Fix fstab if needed (wrong UUID, etc.)
blkid # Get correct UUIDs
vi /etc/fstab # Update
```

| **Validation** | mount shows filesystems as "rw" |

## Scenario 52: LVM Volume Group Not Activating

| Field | Details | |-----|-----| | **Issue** | LVM VG not activated after boot | | **Symptoms** | Logical volumes missing, mount points fail | | **Root Cause** | LVM metadata corrupted, or lvm2 package update issue | | **Commands** | |

```
Scan for VGs
vgscan
pvscan
Activate volume groups
vgchange -ay
Check LV status
lvs -a
lvdisplay
If VG not found:
Check for backup metadata
ls /etc/lvm/backup/
vgcfgrestore <vg_name>
Rebuild LVM cache
lvmconfig --type diff
pvscan --cache
If metadata corrupted:
vgcfgrestore --list <vg_name>
vgcfgrestore -f /etc/lvm/archive/<vg_name>_XXXXX.vg <vg_name>
vgchange -ay <vg_name>
```

| **Validation** | lvs shows all volumes active |

## Scenario 53: /var/log Full - Cannot Complete Patching

| Field | Details | |-----|-----| | **Issue** | /var or /var/log partition full, patching fails | | **Symptoms** | "No space left on device", yum transaction fails | | **Root Cause** | Logs not rotated, large log files, or audit log overflow | | **Commands** |

```
Find large files
du -sh /var/log/* | sort -rh | head -10
find /var/log -size +100M -ls
Truncate large log files (don't delete - may break logging)
> /var/log/messages.old
> /var/log/audit/audit.log.1
cat /dev/null > /var/log/large_app_log.log
Force log rotation
logrotate -f /etc/logrotate.conf
Clean old journal logs
journalctl --vacuum-size=500M
journalctl --vacuum-time=7d
Clean yum cache
yum clean all
rm -rf /var/cache/yum/*
Verify space freed
df -h /var
```

| **Validation** | df -h /var shows adequate free space |

## Scenario 54: Inode Exhaustion on Filesystem

| Field | Details | |-----|-----| | **Issue** | Cannot create new files despite having disk space | | **Symptoms** | "No space left on device" but df -h shows free space | | **Root Cause** | All inodes consumed by many small files | | **Commands** |

```
Check inode usage
df -i
Find directory with most files
find / -xdev -printf '%h\n' | sort | uniq -c | sort -rn | head -10
Common culprits:
ls /tmp/ | wc -l
ls /var/spool/postfix/maildrop/ | wc -l
ls /var/spool/at/ | wc -l
Clean up
find /tmp -type f -mtime +7 -delete
find /var/spool/postfix/maildrop -type f -delete
find /var/cache -type f -mtime +30 -delete
Verify
df -i
```

| **Validation** | df -i shows inode usage below 80% |

## Scenario 55: NFS Mount Hangs After Reboot

| Field | Details | |-----|-----| | **Issue** | Server hangs during boot waiting for NFS mount | | **Symptoms** | Boot stuck at "Mounting NFS filesystems" or process hangs accessing NFS | | **Root Cause** | NFS server unreachable, or mount options missing nofail/bg | | **Commands** |

```
Check NFS mounts in fstab
grep nfs /etc/fstab
```

```
Fix fstab to prevent boot hang:
Add options: bg,soft,timeo=5,retrans=3,nofail
Example:
nfsserver:/share /mnt/share nfs bg,soft,timeo=5,retrans=3,nofail
0 0
Check NFS connectivity
showmount -e <nfs_server>
ping <nfs_server>
Mount manually
mount -t nfs <nfs_server>:/share /mnt/share
If mount is hung:
umount -f /mnt/share # Force unmount
umount -l /mnt/share # Lazy unmount (last resort)
Check for stale NFS handles
dmesg | grep -i "stale\nnfs"
```

| **Validation** | NFS mounts accessible, boot completes without hanging |

## Scenario 56: Disk I/O Performance Degraded After Update

| Field | Details | |-----|-----| | **Issue** | Disk I/O significantly slower after kernel update | | **Symptoms** | High iowait, slow application response, throughput drop | | **Root Cause** | I/O scheduler changed, block device settings reset | | **Commands** | |

```
Check current I/O scheduler
cat /sys/block/sda/queue/scheduler
Expected: [mq-deadline] or [none] for NVMe
Check disk stats
iostat -x 1 5
Restore scheduler (if changed)
echo "mq-deadline" > /sys/block/sda/queue/scheduler
Make persistent via kernel parameter:
Add to /etc/default/grub GRUB_CMDLINE_LINUX:
elevator=mq-deadline
grub2-mkconfig -o /boot/grub2/grub.cfg
Check readahead
blockdev --getra /dev/sda
Set if needed:
blockdev --setra 8192 /dev/sda
```

| **Validation** | iostat shows normal I/O metrics |

## Scenario 57: Swap Partition Not Activating

| Field | Details | |-----|-----| | **Issue** | Swap space not active after reboot | | **Symptoms** | free -h shows 0 swap | | **Root Cause** | UUID changed, fstab entry wrong, or swap partition corrupted | | **Commands** | |

```
Check swap status
swapon -s
free -h
Check fstab
grep swap /etc/fstab
blkid | grep swap
If UUID mismatch, update fstab
blkid /dev/dm-1 # Get current UUID
```

```
vi /etc/fstab # Update UUID
Activate swap
swapon -a
If swap corrupted, recreate:
swapoff /dev/dm-1
mkswap /dev/dm-1
swapon /dev/dm-1
```

| **Validation** | swapon -s shows swap active |

## 12.6 Service & Application Issues (Scenarios 58-75)

### Scenario 58: Apache httpd Fails to Start After Update

| Field | Details | |-----|-----| | **Issue** | Apache won't start after httpd package update | | **Symptoms** | "Job for httpd.service failed", configuration errors | | **Root Cause** | Config syntax changed in new version, deprecated directives | | **Commands** |

```
Check error details
systemctl status httpd -l
journalctl -u httpd --since "5 minutes ago"
Test configuration
apachectl configtest
httpd -t
Common issues after update:
1. Module path changed
find /etc/httpd/modules -name "*.so" | sort
Compare with LoadModule directives
2. Deprecated directives
grep -rn "NameVirtualHost\|Order\|Allow from\|Deny from"
/etc/httpd/conf.d/
Fix deprecated (Apache 2.4+):
Replace: Order allow,deny / Allow from all
With: Require all granted
3. Config file replaced (.rpmnew created)
find /etc/httpd/ -name "*.rpmnew" -o -name "*.rpmsave"
Merge changes from .rpmnew files
Restart after fixing
systemctl start httpd
```

| **Validation** | systemctl status httpd shows active/running |

### Scenario 59: Tomcat Application Not Starting

| Field | Details | |-----|-----| | **Issue** | Tomcat fails to start after Java/system library update | | **Symptoms** | Java exceptions in catalina.out, port binding errors | | **Root Cause** | Java version compatibility, library changes | | **Commands** |

```
Check Tomcat logs
tail -100 /opt/tomcat/logs/catalina.out | grep -i
"error\|exception\|fatal"
Check Java version
java -version
alternatives --list | grep java
```

```
If Java updated to incompatible version:
alternatives --config java
Select compatible version
Check port conflicts
ss -tuln | grep 8080
Kill conflicting process if needed
Check file permissions (sometimes changed by updates)
ls -la /opt/tomcat/conf/
chown -R tomcat:tomcat /opt/tomcat/
Restart Tomcat
systemctl restart tomcat
Or:
/opt/tomcat/bin/startup.sh
```

| **Validation** | Tomcat health endpoint responds HTTP 200 |

## Scenario 60: JBoss/WildFly Fails After Update

| Field | Details | |-----|-----| | **Issue** | JBoss cannot start in domain/standalone mode | | **Symptoms** | "Failed to connect to controller" or deployment errors | | **Root Cause** | Java version change, library conflicts, or config incompatibility | | **Commands** | |

```
Check JBoss logs
tail -100 /opt/jboss/standalone/log/server.log
Verify Java
echo $JAVA_HOME
$JAVA_HOME/bin/java -version
Check for required libraries
ldd /opt/jboss/bin/jboss-modules.jar 2>/dev/null
Restart in standalone mode for testing
/opt/jboss/bin/standalone.sh &
Check deployment status
/opt/jboss/bin/jboss-cli.sh --connect \
 --command="deployment-info"
If config corrupted, restore from backup
cp /root/pre_patch_backup/standalone.xml
/opt/jboss/standalone/configuration/
```

| **Validation** | JBoss CLI shows server-state "running" |

## Scenario 61: MySQL/MariaDB Won't Start After Update

| Field | Details | |-----|-----| | **Issue** | Database service fails to start after package update | | **Symptoms** | "mysql.service: Failed", InnoDB recovery errors | | **Root Cause** | Data file incompatibility, config changes, or upgrade needed | | **Commands** | |

```
Check error log
tail -50 /var/log/mysqld.log
Or:
tail -50 /var/log/mariadb/mariadb.log
Common issues:
1. innodb_log_file_size changed
Remove old log files (MySQL will recreate):
CAUTION: Only if MySQL cleanly shut down before update
rm /var/lib/mysql/ib_logfile0 /var/lib/mysql/ib_logfile1
```

```
2. Run mysql_upgrade
mysql_upgrade --force
3. Check permissions
ls -la /var/lib/mysql/
chown -R mysql:mysql /var/lib/mysql
4. Start with skip-grant-tables for troubleshooting
mysqld_safe --skip-grant-tables &
Start normally
systemctl start mysqld
```

| **Validation** | mysqladmin ping returns "mysqld is alive" |

## Scenario 62: PostgreSQL Fails After Library Update

| Field | Details | |-----|-----| | **Issue** | PostgreSQL won't start after glibc or shared library update | | **Symptoms** | "could not load library" errors | | **Root Cause** | Shared library versions changed, extensions broken | | **Commands** | |

```
Check logs
tail -50 /var/lib/pgsql/data/log/postgresql-*.log
Check shared libraries
ldd $(which postgres) | grep "not found"
If extensions broken:
Rebuild extensions
pg_config --pkglibdir
ls /usr/pgsql-*/lib/
Reinstall PostgreSQL contrib
yum reinstall postgresql-contrib
Check data directory ownership
ls -la /var/lib/pgsql/data/
chown -R postgres:postgres /var/lib/pgsql/
Start PostgreSQL
systemctl start postgresql
pg_isready
```

| **Validation** | pg\_isready returns "accepting connections" |

## Scenario 63: Systemd Service Unit File Changed (.rpmnew)

| Field | Details | |-----|-----| | **Issue** | Service behaves differently after package update | | **Symptoms** | Service options changed, environment variables lost | | **Root Cause** | Package update replaced unit file, custom overrides lost | | **Commands** | |

```
Check for .rpmnew files
find /etc/systemd/ -name "*.rpmnew" -o -name "*.rpmsave"
find /usr/lib/systemd/ -name "*.rpmnew"
Check if custom overrides exist
systemctl cat <service_name>
Shows combined unit file with overrides
Create proper override (won't be replaced on updates)
systemctl edit <service_name>
This creates
/etc/systemd/system/<service>.service.d/override.conf
Reload systemd after any unit file changes
systemctl daemon-reload
systemctl restart <service_name>
```

| **Validation** | `systemctl status <service>` shows expected configuration |

## Scenario 64: Cron Jobs Not Running After Update

| Field | Details | |-----|-----| | **Issue** | Scheduled cron jobs stopped executing | | **Symptoms** | Expected outputs not generated, `cron.log` shows no executions | | **Root Cause** | `crond` service not started, `crontab` cleared, or `PATH` changed | | **Commands** |

```
Check crond status
systemctl status crond
Check crontab
crontab -l
cat /etc/crontab
ls -la /etc/cron.d/
Check cron logs
grep CRON /var/log/cron
journalctl -u crond --since "1 hour ago"
Restart crond
systemctl restart crond
Common issue: PATH not set in crontab
Add at top of crontab:
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
crontab -e
Verify cron is accepting jobs
crontab -l
```

| **Validation** | `grep CRON /var/log/cron` shows job executions |

## Scenario 65: rsyslog/journald Not Logging After Update

| Field | Details | |-----|-----| | **Issue** | System logs empty or not updating | | **Symptoms** | `/var/log/messages` not growing, `journalctl` shows old entries only | | **Root Cause** | `rsyslog` config error after update, or journal corrupted | | **Commands** |

```
Check rsyslog
systemctl status rsyslog
rsyslogd -N1 # Test config
If config error:
find /etc/rsyslog.d/ -name "*.rpmnew"
Merge config changes
Restart rsyslog
systemctl restart rsyslog
Test logging
logger "test message from $(hostname)"
grep "test message" /var/log/messages
For journald issues:
journalctl --verify
If corrupted:
rm -rf /var/log/journal/*
systemctl restart systemd-journald
```

| **Validation** | `logger "test"` appears in `/var/log/messages` |

## Scenario 66: Ansible Connection Fails After Patch

| Field | Details | |-----|-----| | **Issue** | Ansible cannot connect to patched hosts | | **Symptoms** | "UNREACHABLE" errors in Ansible output | | **Root Cause** | SSH host key changed, Python version changed, or sshd down | | **Commands** | |

```
From Ansible control node:
Check SSH connectivity
ssh -v ansible_svc@<host>
If host key changed (after OS reinstall/major update):
ssh-keygen -R <host>
ssh-keyscan <host> >> ~/.ssh/known_hosts
Or in ansible.cfg:
host_key_checking = False
If Python issue:
ansible <host> -m ping -e
"ansible_python_interpreter=/usr/bin/python3"
On the target host:
which python3
rpm -qa | grep python3
```

| **Validation** | ansible <host> -m ping returns SUCCESS |

## Scenario 67: Chef Client Fails After Patch

| Field | Details | |-----|-----| | **Issue** | Chef-client cannot run after system update | | **Symptoms** | "chef-client: command not found" or SSL errors | | **Root Cause** | Ruby/openssl update broke Chef dependencies | | **Commands** | |

```
Check chef-client
which chef-client
chef-client --version
If not found:
export PATH=$PATH:/opt/chef/bin:/opt/chef/embedded/bin
If SSL error:
knife ssl fetch
chef-client -l debug
Reinstall chef-client if corrupted
rpm -e chef
curl -L https://omnitruck.chef.io/install.sh | bash -s -- -v
<version>
Verify
chef-client --once
```

| **Validation** | chef-client --once completes successfully |

## Scenario 68: Monitoring Agent Stopped After Patch

| Field | Details | |-----|-----| | **Issue** | Zabbix/Nagios/Prometheus agent not running | | **Symptoms** | Host shows as "down" in monitoring, no metrics | | **Root Cause** | Agent service not enabled, or config file replaced | | **Commands** | |

```
Check agent status
systemctl status zabbix-agent2
systemctl status nrpe
systemctl status node_exporter
Enable and start
systemctl enable zabbix-agent2
```

```

systemctl start zabbix-agent2
Check if config was replaced
find /etc/zabbix/ -name "*.rpmnew"
Merge config if needed
Test agent locally
zabbix_agent2 -t system.uptime
Or for NRPE:
/usr/lib64/nagios/plugins/check_load -w 5,4,3 -c 10,8,6
Verify connectivity to monitoring server
zabbix_sender -z <zabbix_server> -s $(hostname) -k test -o 1

```

| **Validation** | Host appears "up" in monitoring dashboard |

## Scenario 69: Cluster Resource Fails to Start After Patch

| Field | Details | |-----|-----| | **Issue** | Pacemaker cluster resource won't start on patched node | | **Symptoms** | Resource in "Stopped" or "Failed" state | | **Root Cause** | Resource agent dependency changed, or fencing issue | | **Commands** | |

```

Check cluster status
pcs status
pcs resource status
Check resource failure reason
pcs resource failcount show <resource_name>
pcs resource debug-start <resource_name>
Clear failure count
pcs resource cleanup <resource_name>
Check resource agent
pcs resource describe <resource_type>
If fencing issue:
pcs stonith status
pcs stonith show
Restart resource
pcs resource restart <resource_name>
If node issue:
pcs node unstandby $(hostname)

```

| **Validation** | pcs status shows all resources Started |

## Scenario 70: HAProxy/Nginx Load Balancer Backend Failures

| Field | Details | |-----|-----| | **Issue** | Backend servers failing health checks after their patching | | **Symptoms** | HAProxy shows backends as DOWN, 503 errors | | **Root Cause** | Application not fully started, health endpoint changed | | **Commands** | |

```

Check HAProxy status
echo "show stat" | socat stdio /var/run/haproxy/admin.sock | grep
-i "down\|maint"
Check backend server health
curl -v http://<backend_server>:<port>/health
Wait for application warm-up
sleep 60
curl http://<backend_server>:<port>/health
Force backend UP in HAProxy
echo "set server web-backend/<server> state ready" | socat stdio

```

```

/var/run/haproxy/admin.sock
For Nginx:
Check upstream status
curl http://localhost/upstream_status
Reload Nginx if upstream config changed
nginx -t && systemctl reload nginx

```

| **Validation** | All backends show "UP" in load balancer status |

## Scenario 71: SSL/TLS Certificate Issues After OpenSSL Update

| Field | Details | |-----|-----| | **Issue** | SSL connections failing after openssl update | | **Symptoms** | "SSL handshake failed", "certificate verify failed" | | **Root Cause** | CA bundle updated, cipher suites changed, or TLS version mismatch | | **Commands** |

```

Test SSL connection
openssl s_client -connect localhost:443 -tls1_2
Check certificate
openssl x509 -in /etc/pki/tls/certs/server.crt -noout -dates
Check CA bundle
openssl verify -CAfile /etc/pki/tls/certs/ca-bundle.crt
/path/to/cert.pem
Update CA certificates
update-ca-trust force-enable
update-ca-trust extract
If custom CA needed:
cp custom-ca.crt /etc/pki/ca-trust/source/anchors/
update-ca-trust extract
Check supported ciphers
openssl ciphers -v 'ALL' | head -20

```

| **Validation** | openssl s\_client -connect host:443 handshake succeeds |

## Scenario 72: LDAP/AD Authentication Broken After Patch

| Field | Details | |-----|-----| | **Issue** | Users cannot login via LDAP/AD after sssd or PAM update | | **Symptoms** | "Access denied", users not found, cannot sudo | | **Root Cause** | sssd config changed, cache corrupted, or PAM module issue | | **Commands** |

```

Check sssd status
systemctl status sssd
sssctl domain-status <domain>
Check user lookup
id <username>
getent passwd <username>
Clear sssd cache
sss_cache -E
systemctl restart sssd
Check PAM configuration
cat /etc/pam.d/system-auth | grep -i "sss\|ldap"
Check sssd config
cat /etc/sss/sss.conf
Check for .rpmnew in PAM
find /etc/pam.d/ -name "*.rpmnew"
Test LDAP connectivity

```

```
ldapsearch -H ldap://<ldap_server> -D "cn=bind,dc=corp,dc=com" -w
-b "dc=corp,dc=com" "(uid=testuser)"
```

| **Validation** | id <ad\_username> returns user info |

## Scenario 73: Sudo Not Working After PAM Update

| Field | Details | |-----|-----| | **Issue** | sudo authentication fails for all users | | **Symptoms** | "sudo: PAM authentication error" | | **Root Cause** | PAM configuration file replaced during update | | **Commands** | |

```
Check PAM sudo config
cat /etc/pam.d/sudo
Check for replaced files
ls -la /etc/pam.d/sudo*
Restore default sudo PAM config:
cat > /etc/pam.d/sudo << 'EOF'
##PAM-1.0
auth include system-auth
account include system-auth
password include system-auth
session include system-auth
EOF
Verify sudoers file
visudo -c
Expected: /etc/sudoers: parsed OK
Test sudo
sudo -l
sudo whoami
Expected: root
```

| **Validation** | sudo whoami returns "root" |

## Scenario 74: Audit Service Fails After Update

| Field | Details | |-----|-----| | **Issue** | auditd fails to start, compliance requirements not met | | **Symptoms** | systemctl status auditd shows failed | | **Root Cause** | Audit rules syntax error, config incompatibility | | **Commands** | |

```
Check audit status
systemctl status auditd -l
journalctl -u auditd
Test audit rules
augenrules --check
auditctl -l
If rules have errors:
augenrules --load 2>&1 | grep -i error
Fix or remove problematic rule file
ls /etc/audit/rules.d/
Remove/fix file with error
Restart auditd (special - cannot use systemctl restart)
service auditd restart
Or:
auditctl -R /etc/audit/audit.rules
```

| **Validation** | auditctl -l loads rules without errors |

## Scenario 75: Sysstat/SAR Data Collection Stopped

| Field | Details | |-----|-----| | **Issue** | Performance data collection (SAR) stopped after sysstat update | | **Symptoms** | sar shows "Cannot open /var/log/sa/sa<DD>" | | **Root Cause** | sysstat service disabled, data directory permissions | | **Commands** | |

```
Check sysstat service
systemctl status sysstat
systemctl is-enabled sysstat
Enable and start
systemctl enable sysstat
systemctl start sysstat
Check cron entry
cat /etc/cron.d/sysstat
Initialize data collection
/usr/lib64/sa/sa1 1 1
Verify data
sar -u 1 3
ls -la /var/log/sa/
```

| **Validation** | sar displays current performance data |

## 12.7 Automation & Tool Issues (Scenarios 76-90)

### Scenario 76: Ansible Playbook Timeout During Patch

| Field | Details | |-----|-----| | **Issue** | Ansible times out waiting for yum update to complete | | **Symptoms** | "FAILED! => timeout" during yum task | | **Root Cause** | Large number of packages, slow network, default timeout too short | | **Commands** | |

```
Increase task timeout in playbook:
- name: Apply patches
yum:
name: '*'
state: latest
async: 7200 # 2 hours
poll: 60 # Check every 60 seconds
Or set connection timeout:
ansible.cfg:
[defaults]
timeout = 3600
Check if yum is still running on target:
ssh <host> "ps aux | grep yum"
If yum completed but Ansible lost connection:
ssh <host> "yum history info last"
```

| **Validation** | Playbook completes within extended timeout |

### Scenario 77: Ansible "Unreachable" After Reboot Task

| Field | Details | |-----|-----| | **Issue** | Ansible cannot reconnect after reboot module | | **Symptoms** | "UNREACHABLE" on tasks after reboot | | **Root Cause** | Reboot timeout too short, IP changed, or boot slow | | **Commands** | |

```
Fix: Use proper reboot module with adequate timeout
```

```

- name: Reboot and wait
 reboot:
 reboot_timeout: 900 # Wait up to 15 minutes
 connect_timeout: 30
 pre_reboot_delay: 10
 post_reboot_delay: 60 # Wait 60s after boot before
reconnecting
 test_command: uptime
Or use wait_for_connection:
- name: Reboot
 command: /sbin/reboot
 async: 1
 poll: 0
- name: Wait for server to come back
 wait_for_connection:
 delay: 60
 timeout: 600

```

| **Validation** | Post-reboot tasks execute successfully |

## Scenario 78: Ansible "Missing sudo password" Error

| Field | Details | |-----|-----| | **Issue** | Ansible cannot escalate privileges on target host | | **Symptoms** | "Missing sudo password" or "Incorrect sudo password" | | **Root Cause** | sudoers file changed during PAM/sudo update | | **Commands** | |

```

On target host - check sudoers
visudo -c
cat /etc/sudoers | grep <ansible_user>
cat /etc/sudoers.d/ | grep <ansible_user>
Ensure ansible user has NOPASSWD:
/etc/sudoers.d/ansible:
ansible_svc ALL=(ALL) NOPASSWD: ALL
Check for .rpmnew/.rpmsave
find /etc/sudoers* -name "*.rpmnew" -o -name "*.rpmsave"
Fix permissions
chmod 440 /etc/sudoers.d/ansible

```

| **Validation** | ansible <host> -m ping -b succeeds |

## Scenario 79: Ansible Batch Fails - max\_fail\_percentage Exceeded

| Field | Details | |-----|-----| | **Issue** | Ansible stops entire batch because too many hosts failed | | **Symptoms** | "PLAY RECAP" shows multiple failures, remaining hosts skipped | | **Root Cause** | Common issue across hosts (disk space, repo, etc.) | | **Commands** | |

```

Adjust max_fail_percentage:
- hosts: batch4_remaining
 serial: 50
 max_fail_percentage: 20 # Allow up to 20% failures per batch
Or use ignore_unreachable:
- name: Apply patches
 yum:
 name: '*'
 state: latest

```

```

 ignore_unreachable: yes
Investigate common failure:
Check Ansible output for the error pattern
Fix root cause on all hosts first, then re-run
ansible <failed_hosts> -m shell -a "df -h / /boot /var"

```

| **Validation** | Re-run succeeds on previously failed hosts |

## Scenario 80: Ansible Variable Precedence Issue

| Field | Details | |-----|-----| | **Issue** | Wrong patch configuration applied (e.g., all updates instead of security-only) | | **Symptoms** | More packages updated than expected | | **Root Cause** | Variable precedence: extra vars > host vars > group vars | | **Commands** | |

```

Debug variable values
ansible <host> -m debug -a "var=patch_security_only"
Check precedence (highest wins):
1. Extra vars (-e)
2. Task vars
3. Host vars
4. Group vars
5. Role defaults
Fix: Be explicit in command line
ansible-playbook patch.yml -e "patch_security_only=true"
Or verify with --check mode first
ansible-playbook patch.yml --check --diff -v

```

| **Validation** | --check output shows correct behavior before real run |

## Scenario 81: Chef Cookbook Version Conflict

| Field | Details | |-----|-----| | **Issue** | Chef run fails due to cookbook version constraint conflict | | **Symptoms** | "Unable to satisfy constraints" in chef-client output | | **Root Cause** | Environment cookbook version pinning conflicts with dependency | | **Commands** | |

```

Check environment constraints
knife environment show production | grep -A 20 cookbook_versions
Check cookbook dependencies
knife cookbook show linux_patching | grep depends
Upload correct version
knife cookbook upload linux_patching --freeze
Update environment constraint
knife environment edit production
Adjust version constraint: "linux_patching": "= 3.0.0"
Verify
chef-client --why-run

```

| **Validation** | chef-client converges successfully |

## Scenario 82: Chef Run Takes Too Long

| Field | Details | |-----|-----| | **Issue** | Chef-client run exceeds timeout during patching | | **Symptoms** | "Chef::Exceptions::CommandTimeout" | | **Root Cause** | Default timeout (3600s) too short for large update | | **Commands** | |

```
In recipe, set longer timeout:
execute 'apply_patches' do
 command 'yum update --security -y'
 timeout 7200 # 2 hours
 live_stream true
end
Or in client.rb:
chef_client_interval 7200
```

| **Validation** | Chef run completes within extended timeout |

### Scenario 83: SCCM Linux Client Not Reporting

| Field | Details | |-----|-----| | **Issue** | SCCM console shows Linux host as "inactive" | | **Symptoms** | No hardware inventory, compliance unknown | | **Root Cause** | OMI agent crashed after system update | | **Commands** | |

```
Check OMI status
systemctl status omid
/opt/omi/bin/omiserver -s
Restart OMI
systemctl restart omid
/opt/omi/bin/omiserver -r
Check SCCM client
/opt/microsoft/configmgr/bin/ccmexec -status
Force inventory
/opt/microsoft/configmgr/bin/ccmexec -rs inventory
If completely broken, reinstall
rpm -e omi
rpm -e configmgr-client
Reinstall from DP
```

| **Validation** | SCCM console shows host as active with current inventory |

### Scenario 84: Satellite Remote Execution Fails

| Field | Details | |-----|-----| | **Issue** | Cannot run remote commands via Satellite REX | | **Symptoms** | "Failed to initialize: Net::SSH::AuthenticationFailed" | | **Root Cause** | SSH key changed after update, or Rex user disabled | | **Commands** | |

```
Check SSH key on host
cat /root/.ssh/authorized_keys | grep satellite
Re-deploy SSH key from Satellite
hammer host update --name <host> --compute-attributes=""
Or manually add Satellite's public key:
curl https://satellite.corp.com:9090/ssh/pubkey >>
/root/.ssh/authorized_keys
chmod 600 /root/.ssh/authorized_keys
Check SSH from Satellite
ssh -i /usr/share/foreman-proxy/.ssh/id_rsa_foreman_proxy
root@<host>
```

| **Validation** | Satellite REX job executes successfully on host |

### Scenario 85: Patch Report Shows Inconsistent Data

| Field | Details | |-----|-----| | **Issue** | Compliance report doesn't match actual patch status | | **Symptoms** | Satellite shows errata outstanding but host is patched | | **Root Cause** | Satellite host facts not updated, applicability not recalculated | | **Commands** |

```
On host - force upload of package profile
subscription-manager repos --list-enabled
yum clean all
subscription-manager facts --update
On Satellite:
hammer host errata recalculate --host <hostname>
Or force content host refresh:
hammer content-host package list --content-host <host>
--organization "My Org"
Generate fresh report
hammer report-template generate --name "Host - Errata" --inputs
"hosts=<host>"
```

| **Validation** | Satellite errata report matches yum updateinfo list on host |

## 12.8 Memory, CPU & Performance Issues (Scenarios 86-100)

### Scenario 86: High CPU After Kernel Update

| Field | Details | |-----|-----| | **Issue** | CPU utilization abnormally high after kernel update | | **Symptoms** | System sluggish, load average high, processes slow | | **Root Cause** | kcompactd, khugepaged, or other kernel thread consuming CPU | | **Commands** |

```
Identify CPU consumer
top -bn1 | head -20
ps aux --sort=-%cpu | head -10
If kernel thread (kcompactd, khugepaged):
echo 0 > /proc/sys/vm/compact_memory
echo never > /sys/kernel/mm/transparent_hugepage/enabled
If SELinux relabeling:
Wait for completion or check:
ps aux | grep restorecon
If unnecessary services started:
systemctl list-units --type=service --state=running
Disable unnecessary ones
Compare with pre-patch CPU baseline
sar -u 1 10
```

| **Validation** | CPU usage returns to pre-patch baseline |

### Scenario 87: Memory Leak After glibc Update

| Field | Details | |-----|-----| | **Issue** | Memory usage steadily increasing after glibc update | | **Symptoms** | Available memory decreasing over time, swap usage increasing | | **Root Cause** | Application incompatibility with new memory allocator behavior | | **Commands** |

```
Monitor memory over time
watch -n 5 free -h
Identify memory consumer
ps aux --sort=-%mem | head -10
```

```

smem -t -p | head -10
Check for known glibc regression
rpm -q glibc
Search Red Hat KB for glibc issues
Temporary mitigation:
Set malloc tuning
export MALLOC_ARENA_MAX=2 # For specific application
Restart affected applications
systemctl restart <application>
If confirmed bug, downgrade glibc (RISKY - test first):
yum downgrade glibc glibc-common

```

| **Validation** | Memory usage stabilizes over 24-hour period |

## Scenario 88: OOM Killer Activating After Patch

| Field | Details | |-----|-----| | **Issue** | OOM killer terminating critical processes | | **Symptoms** | Processes randomly killed, "Out of memory" in dmesg | | **Root Cause** | New kernel memory overhead, or new services consuming memory | | **Commands** |

```

Check OOM events
dmesg | grep -i "oom\|killed"
journalctl | grep "Out of memory"
Identify what was killed
dmesg | grep "Killed process" | tail -5
Protect critical processes from OOM
echo -1000 > /proc/$(pgrep -x mysqld)/oom_score_adj
echo -1000 > /proc/$(pgrep -x sshd | head -1)/oom_score_adj
Make persistent:
In systemd unit file: OOMScoreAdjust=-1000
systemctl edit mysqld
Add: [Service]
OOMScoreAdjust=-1000
Add swap if needed (temporary)
fallocate -l 4G /tmp/emergency_swap
mkswap /tmp/emergency_swap
swapon /tmp/emergency_swap
Identify new memory consumers since patch
ps aux --sort=-%mem | head -10

```

| **Validation** | No OOM events in dmesg after 24 hours |

## Scenario 89: System Swap Thrashing After Patch

| Field | Details | |-----|-----| | **Issue** | Excessive swap usage causing poor performance | | **Symptoms** | High swap usage, high iowait, system unresponsive | | **Root Cause** | Memory pressure from new services or kernel memory changes | | **Commands** |

```

Check swap usage
free -h
swapon -s
cat /proc/swaps
Check swappiness
cat /proc/sys/vm/swappiness
Reduce swappiness (prefer keeping in RAM)

```

```

sysctl vm.swappiness=10
echo "vm.swappiness=10" >> /etc/sysctl.d/99-tuning.conf
Identify what's in swap
for pid in $(ls /proc/[0-9]*/status 2>/dev/null | cut -d/ -f3); do
 swap=$(grep VmSwap /proc/$pid/status 2>/dev/null | awk '{print $2}')
 if ["${swap:-0}" -gt 1000]; then
 name=$(grep Name /proc/$pid/status | awk '{print $2}')
 echo "$pid $name: ${swap}kB in swap"
 fi
done | sort -k3 -rn | head -10
Restart memory-hungry services to clear swap
systemctl restart <service>

```

| **Validation** | free -h shows swap usage below 20% |

## Scenario 90: Performance Regression After Kernel Update

| Field | Details | |-----|-----| | **Issue** | General system performance degraded after kernel update | | **Symptoms** | Application response times increased, throughput decreased | | **Root Cause** | Kernel parameter defaults changed, security mitigations enabled | | **Commands** | |

```

Check if new CPU mitigations enabled
cat /sys/devices/system/cpu/vulnerabilities/*
grep -i "mitigat" /proc/cmdline
Check kernel parameters that affect performance
sysctl -a | grep -E "vm\.|net\." | head -30
Compare with pre-patch sysctl values
diff /root/pre_patch_backup/sysctl_values.txt <(sysctl -a
2>/dev/null)
If Spectre/Meltdown mitigations causing issues (measure first):
WARNING: Only disable if you understand the security implications
Add to kernel cmdline: mitigations=off
Or specific: nospectre_v2 nopti
Restore custom sysctl values
sysctl -p /etc/sysctl.d/99-performance.conf

```

| **Validation** | Application benchmarks match pre-patch baseline |

## Scenario 91: Zombie Processes After Patch

| Field | Details | |-----|-----| | **Issue** | Accumulation of zombie (defunct) processes | | **Symptoms** | top shows high zombie count, process table filling | | **Root Cause** | Parent process not reaping children after library update | | **Commands** | |

```

Count zombies
ps aux | grep -c defunct
Find parent of zombies
ps -eo pid,ppid,stat,cmd | grep "^.*Z"
Note the PPID
Restart parent process
kill -SIGCHLD <ppid> # Ask parent to reap
Or:
systemctl restart <parent_service>
If parent is init/systemd (PID 1), reboot may be needed

```

| **Validation** | ps aux | grep defunct shows zero zombies |

## Scenario 92: Tuned Profile Reset After Update

| Field | Details | |-----|-----| | **Issue** | Performance tuning profile changed to default | | **Symptoms** | System using "balanced" instead of custom profile | | **Root Cause** | tuned package update reset active profile | | **Commands** | |

```
Check current profile
tuned-adm active
Expected: Current active profile: throughput-performance (or
custom)
If wrong, set correct profile
tuned-adm list
tuned-adm profile throughput-performance
Verify
tuned-adm verify
```

| **Validation** | tuned-adm active shows expected profile |

## Scenario 93: Huge Pages Configuration Lost

| Field | Details | |-----|-----| | **Issue** | Huge pages not allocated after reboot (affects Oracle/databases) | | **Symptoms** | Application fails with "Cannot allocate memory" for huge pages | | **Root Cause** | sysctl.conf changes not applied, or kernel parameter missing | | **Commands** | |

```
Check huge pages
grep Huge /proc/meminfo
If not set:
sysctl -w vm.nr_hugepages=1024
echo "vm.nr_hugepages=1024" >> /etc/sysctl.d/99-hugepages.conf
For kernel boot parameter:
Add to GRUB_CMDLINE_LINUX: hugepagesz=2M hugepages=1024
grub2-mkconfig -o /boot/grub2/grub.cfg
```

| **Validation** | grep HugePages\_Total /proc/meminfo shows expected value |

## Scenario 94: ulimit Values Reset After PAM Update

| Field | Details | |-----|-----| | **Issue** | Process resource limits reverted to defaults | | **Symptoms** | "Too many open files" errors, applications failing | | **Root Cause** | /etc/security/limits.conf overwritten or PAM config changed | | **Commands** | |

```
Check current limits
ulimit -a
cat /proc/$(pgrep <app_name>)/limits
Check limits.conf
cat /etc/security/limits.conf
cat /etc/security/limits.d/*.conf
Restore custom limits:
cat >> /etc/security/limits.d/99-custom.conf << 'EOF'
* soft nofile 65535
* hard nofile 65535
* soft nproc 65535
* hard nproc 65535
```

```

oracle soft memlock unlimited
oracle hard memlock unlimited
EOF
For systemd services, set in unit file:
systemctl edit <service>
[Service]
LimitNOFILE=65535
LimitNPROC=65535
systemctl daemon-reload
systemctl restart <service>

```

| **Validation** | cat /proc/<PID>/limits shows correct values |

## Scenario 95: Network Throughput Reduced (TCP Tuning Lost)

| Field | Details | |-----|-----| | **Issue** | Network performance degraded after kernel update | | **Symptoms** | Lower throughput on large transfers, high latency | | **Root Cause** | TCP buffer sizes and window scaling reset to defaults | | **Commands** | |

```

Check TCP parameters
sysctl net.core.rmem_max
sysctl net.core.wmem_max
sysctl net.ipv4.tcp_rmem
sysctl net.ipv4.tcp_wmem
Restore custom TCP tuning
cat > /etc/sysctl.d/99-network-tuning.conf << 'EOF'
net.core.rmem_max = 16777216
net.core.wmem_max = 16777216
net.ipv4.tcp_rmem = 4096 87380 16777216
net.ipv4.tcp_wmem = 4096 65536 16777216
net.core.netdev_max_backlog = 5000
net.ipv4.tcp_window_scaling = 1
EOF
sysctl -p /etc/sysctl.d/99-network-tuning.conf

```

| **Validation** | Network throughput test matches pre-patch baseline |

## Scenario 96: Journal Storage Full After Patch

| Field | Details | |-----|-----| | **Issue** | systemd journal consuming excessive disk space | | **Symptoms** | /var/log/journal or /run/log/journal consuming GBs | | **Root Cause** | Journal vacuum settings reset, verbose logging from patch | | **Commands** | |

```

Check journal size
journalctl --disk-usage
Vacuum immediately
journalctl --vacuum-size=500M
journalctl --vacuum-time=7d
Set persistent limits
cat > /etc/systemd/journald.conf.d/size.conf << 'EOF'
[Journal]
SystemMaxUse=500M
RuntimeMaxUse=100M
EOF
systemctl restart systemd-journald

```

| **Validation** | journalctl --disk-usage shows acceptable size |

## Scenario 97: Python Version Conflict After Update

| Field | Details | |-----|-----| | **Issue** | System Python changed, breaking scripts and tools | | **Symptoms** | "ImportError", "ModuleNotFoundError", wrong Python version | | **Root Cause** | Python update changed default, or modules not installed for new version | | **Commands** | |

```
Check Python versions
python --version 2>&1
python3 --version
alternatives --list | grep python
Set default Python
alternatives --set python /usr/bin/python3
Or:
alternatives --config python
Install missing modules
pip3 install <module_name>
For Ansible - set interpreter:
In ansible.cfg or inventory:
ansible_python_interpreter=/usr/bin/python3
```

| **Validation** | Scripts and tools run without Python errors |

## Scenario 98: Satellite Capsule Certificate Expired

| Field | Details | |-----|-----| | **Issue** | Capsule cannot communicate with Satellite after cert expiry | | **Symptoms** | "SSL certificate has expired" errors | | **Root Cause** | Certificate auto-renewal failed or expired during patch window | | **Commands** | |

```
Check certificate expiry
openssl x509 -in /etc/foreman-proxy/ssl_cert.pem -noout -enddate
Regenerate Capsule certificates (on Satellite):
capsule-certs-generate --foreman-proxy-fqdn capsule.corp.com \
 --certs-tar /root/capsule-certs.tar
Copy and install on Capsule:
scp /root/capsule-certs.tar capsule:/root/
On Capsule:
satellite-installer --scenario capsule \
 --certs-tar-file /root/capsule-certs.tar \
 --certs-update-all
```

| **Validation** | hammer capsule info shows Capsule connected |

## Scenario 99: Ansible AWX/Tower Job Stuck

| Field | Details | |-----|-----| | **Issue** | Tower patching job stuck in "running" state | | **Symptoms** | Job never completes, output frozen | | **Root Cause** | SSH connection lost, or callback server issue | | **Commands** | |

```
Cancel stuck job via API
curl -X POST -H "Authorization: Bearer <token>" \
 https://tower.corp.com/api/v2/jobs/<job_id>/cancel/
Check Tower task system
awx-manage list_instances
```

| **Validation** | New job launches and completes successfully |

| Field | Details | |-----|-----| | **Issue** | Multiple critical failures, system unusable after patching | | **Symptoms** | Multiple services down, kernel issues, network broken | | **Root Cause** | Major package conflicts, incomplete transaction, corruption | | **Commands** | |

| **Validation** | System fully functional at pre-patch state |

## Section 13 · Rollback Procedures

```

.....
ROLLBACK DECISION FLOWCHART
.....

```



```

rpm -q openssl
Expected: openssl-3.0.7-20.el9_3.x86_64 (previous version)
Step 5: Exclude problematic package from future patches
yum versionlock add openssl-3.0.7-20.el9_3
Step 6: Validate system health
systemctl list-units --state=failed
ping -c 3 <gateway>

```

### 13.3 Rollback: Failed Reboot (Server Won't Boot)

```

=====
ROLLBACK TYPE 2: Server Won't Boot After Reboot
=====
ACCESS: Use iLO/iDRAC/IPMI console or VMware console
METHOD A: Boot Previous Kernel from GRUB
1. At GRUB menu, select "Advanced options"
2. Choose previous kernel version
3. Server boots on old kernel
Once booted on old kernel:
uname -r # Confirm old kernel
Remove problematic new kernel
yum remove kernel-<new_version>
Set old kernel as default
grubby --set-default /boot/vmlinuz-<old_kernel_version>
grub2-mkconfig -o /boot/grub2/grub.cfg
METHOD B: Rescue Mode
1. Boot from RHEL installation media
2. Select "Troubleshooting" . "Rescue a Red Hat Enterprise Linux
system"
3. Select "Continue" to mount system
4. chroot /mnt/sysimage
In chroot:
chroot /mnt/sysimage
grubby --set-default /boot/vmlinuz-<old_kernel>
grub2-mkconfig -o /boot/grub2/grub.cfg
Remove new kernel if needed
rpm -e kernel-<new_version>
exit
reboot
METHOD C: Restore VM Snapshot (Fastest)
From vCenter/Cloud Console:
Revert to pre-patch snapshot
This restores the entire system state

```

### 13.4 Rollback: Application Failure After Patch

```

=====
ROLLBACK TYPE 3: Application Not Working After Patch
=====
Step 1: Identify which update broke the application
yum history info last
Look for application-related packages (java, openssl, glibc,

```

```

etc.)
Step 2: Check application logs for specific error
tail -100 /var/log/<application>/error.log
journalctl -u <app_service> --since "30 minutes ago"
Step 3: Try rolling back specific packages
If Java app broken after openssl update:
yum downgrade openssl openssl-libs
If application broken after glibc update:
yum downgrade glibc glibc-common glibc-minimal-langpack
If application broken after Java update:
alternatives --config java
Select previous Java version
Step 4: Restart application
systemctl restart <app_service>
Step 5: Verify application health
curl -s http://localhost:<port>/health
Expected: 200 OK
Step 6: If cannot identify specific package, undo entire
transaction
yum history undo <transaction_id> -y
reboot # If kernel was part of the transaction

```

## 13.5 Rollback: Kernel Failure

```

=====
ROLLBACK TYPE 4: Kernel Failure (Panic/Instability)
=====
IMMEDIATE ACTION: Boot from GRUB previous kernel entry
Step 1: At GRUB boot menu (press ESC or arrow keys during boot)
Select: "Red Hat Enterprise Linux (4.18.0-513.18.1.el8_9.x86_64)"
(the previous kernel version)
Step 2: Once booted, verify
uname -r # Should show OLD kernel
Step 3: Set old kernel as permanent default
grubby --set-default /boot/vmlinuz-<old_kernel_version>
Example:
grubby --set-default /boot/vmlinuz-4.18.0-513.18.1.el8_9.x86_64
Step 4: Verify default
grubby --default-kernel
grub2-editenv list
Step 5: Optionally remove problematic kernel
yum remove kernel-<problematic_version>
Step 6: Report issue to Red Hat (open support case)
sosreport
Attach sosreport to case
Step 7: Exclude kernel from future updates until fix available
yum versionlock add kernel-<working_version>
ALTERNATIVE: If server won't even show GRUB
Use IPMI/iLO to access console
Boot from rescue media
Mount root filesystem
Edit GRUB config manually

```

## 13.6 Rollback: Package Corruption

```
=====
ROLLBACK TYPE 5: Package Corruption
=====
Symptoms: "error: rpmdbNextIterator: skipping h#"
"rpm: unable to read package"
Step 1: Backup corrupted RPM database
cp -a /var/lib/rpm /var/lib/rpm.corrupted.$(date +%Y%m%d)
Step 2: Remove lock files
rm -f /var/lib/rpm/__db*
Step 3: Rebuild RPM database
rpm --rebuilddb
Step 4: Verify integrity
rpm -qa | wc -l
rpm -Va 2>/dev/null | grep -v "^..5" | head -20
Step 5: If packages are broken, reinstall them
yum reinstall <broken_package>
Step 6: If yum itself is broken
Download RPM manually and install:
curl -O https://satellite.corp.com/pulp/repos/.../yum-<version>.rpm
rpm -Uvh --force yum-<version>.rpm
Step 7: If entire system is corrupted, restore from backup
Use VM snapshot or backup restoration
```

## 13.7 Rollback: Satellite/Ansible/Chef Failure

```
=====
ROLLBACK TYPE 6: Automation Tool Failure During Patch
=====
--- SATELLITE ROLLBACK ---
If Satellite pushed wrong Content View:
1. Promote previous CV version
hammer content-view version promote \
 --content-view "RHEL8-BaseOS" \
 --version <previous_version> \
 --to-lifecycle-environment "Production" \
 --organization "My Org" --force
2. On affected hosts:
yum clean all
yum downgrade <packages_from_wrong_cv>
--- ANSIBLE ROLLBACK ---
If Ansible playbook applied wrong patches:
Run rollback playbook:
ansible-playbook playbooks/patch_rollback.yml \
 -e "target_group=affected_hosts" \
 -e "ticket=CHG-2026-12345"
If Ansible partially completed (some hosts done, some not):
Check which hosts were patched:
ansible affected_hosts -m shell -a "yum history info last" | grep
-A5 "Transaction"
Rollback only completed hosts
--- CHEF ROLLBACK ---
```

```
Run rollback recipe:
knife ssh "name:affected_host" "sudo chef-client -o
'recipe[linux_patching::rollback]'"
Or force specific transaction rollback:
knife ssh "name:affected_host" "sudo yum history undo last -y"
```

## 13.8 Rollback Validation Checklist

```
#!/bin/bash
rollback_validation.sh - Run after any rollback
echo "=== ROLLBACK VALIDATION ==="
echo "Host: $(hostname)"
echo "Date: $(date)"
echo "=====
echo ""
echo "1. Kernel Version:"
uname -r
echo ""
echo "2. Failed Services:"
systemctl list-units --state=failed --no-pager
echo ""
echo "3. Network Connectivity:"
ping -c 2 $(ip route show default | awk '{print $3}') && echo
"Gateway: OK" || echo "Gateway: FAIL"
echo ""
echo "4. DNS Resolution:"
nslookup $(hostname) && echo "DNS: OK" || echo "DNS: FAIL"
echo ""
echo "5. Disk Space:"
df -h / /boot /var | tail -n +2
echo ""
echo "6. Application Health:"
Customize for your applications
for url in "http://localhost/health"
"http://localhost:8080/status"; do
 code=$(curl -s -o /dev/null -w "%{http_code}" $url 2>/dev/null)
 echo "$url: HTTP $code"
done
echo ""
echo "7. Package Status:"
echo "Last yum transaction:"
yum history info last | head -10
echo ""
echo "=== ROLLBACK VALIDATION COMPLETE ==="
```

•**EXPERT TIP:** The fastest rollback method is VM snapshot revert (2-3 minutes). For bare-metal servers without snapshots, maintain current backups and practice rollback procedures quarterly. A failed rollback during an outage is worse than the original failure.

•**COMMON MISTAKE:** Rolling back glibc requires rolling back ALL packages that depend on it. Use `yum history undo` for the entire transaction rather than trying to selectively downgrade individual packages.

---

## Section 14 · Change Management

### 14.1 Change Request Template

```
.....
· CHANGE REQUEST FORM
·
·
·
·
· SECTION A: CHANGE IDENTIFICATION
·
·
· Change ID: CHG-2026-_____
·
· Title: Monthly Linux Security Patching - June 2026
·
· Type: · Standard · Normal · Emergency
· Priority: · P1-Critical · P2-High · P3-Medium · P4-Low
· Category: Security Patching
·
· Requestor: _____
·
· Date Submitted: _____
·
·
· SECTION B: CHANGE DESCRIPTION
·
·
· Summary:
·
· Apply Red Hat security errata (RHSA-2026-XXXX through
RHSA-2026-YYYY)
· to 200 production Linux servers to remediate CVE-2026-XXXXX (CVSS
8.5).
· and 15 additional security vulnerabilities.
·
·
· Business Justification:
·
· Required for PCI-DSS compliance (Requirement 6.3.3 - Critical
patches
· within 30 days). CVE-2026-XXXXX allows remote code execution.
·
·
· SECTION C: SCOPE & IMPACT
·
```

.....  
Affected Systems: 200 Linux servers (see attached list)  
Affected Services: Web Portal, API Gateway, Batch Processing  
Affected Users: Internal (during maintenance window)  
Impact Level: · Low · Medium · High · Critical  
Risk Level: · Low · Medium · High · Critical

SECTION D: IMPLEMENTATION SCHEDULE

.....  
Planned Start: Saturday 2026-06-27 22:00 UTC  
Planned End: Sunday 2026-06-28 06:00 UTC  
Duration: 8 hours  
Maintenance Window: Approved MW-SAT-2200

SECTION E: TESTING

.....  
DEV Testing Date: 2026-06-20  
DEV Test Result: · Pass · Fail  
UAT Testing Date: 2026-06-23  
UAT Test Result: · Pass · Fail  
Test Evidence: Attached (Ansible output, validation reports)

SECTION F: ROLLBACK PLAN

.....  
Rollback Method: yum history undo + VM snapshot restore  
Rollback Time: 30 minutes per server, 2 hours for full batch  
Rollback Trigger: Any service failure, application error, or  
Go/No-Go criteria not met between batches  
Rollback Owner: Linux Team Lead

SECTION G: APPROVALS

.....

Change Manager: \_\_\_\_\_ Date: \_\_\_\_\_

IT Director: \_\_\_\_\_ Date: \_\_\_\_\_

Security Team: \_\_\_\_\_ Date: \_\_\_\_\_

Application Owner: \_\_\_\_\_ Date: \_\_\_\_\_

DBA Team Lead: \_\_\_\_\_ Date: \_\_\_\_\_

Network Team: \_\_\_\_\_ Date: \_\_\_\_\_

.....

## 14.2 Risk Analysis Document

### Risk Register

| # | Risk Description                 | Probability | Impact       | Risk Score | Mitigation                                                                  | Contingency                                                                                                                |
|---|----------------------------------|-------------|--------------|------------|-----------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| 1 | Patch causes application failure | Medium (3)  | High (4)     | 12         | Test in DEV/UAT first   Rollback via yum history                            | 2   Server fails to reboot   Low (2)   Critical (5)   10   Verify GRUB, take snapshot   Boot old kernel / restore snapshot |
| 2 | Kernel panic on new kernel       | Low (2)     | High (4)     | 8          | Test on canary servers   Boot previous kernel                               | 4   Network loss after reboot   Low (2)   High (4)   8   Backup network config   Restore from backup, use console          |
| 3 | Cluster split-brain              | Low (2)     | Critical (5) | 10         | Patch one node at a time   Manual fencing, rebuild node                     | 6   Database corruption   Very Low (1)   Critical (5)   5   Full backup before patch   Restore from backup                 |
| 4 | Patch window exceeded            | Medium (3)  | Medium (3)   | 9          | Batch strategy, parallel exec   Stop at current batch, continue next window | 8   Dependency conflict   Medium (3)   Medium (3)   9   Test in DEV, check conflicts   Exclude package, patch next cycle   |
| 5 | Monitoring false alerts          | High (4)    | Low (2)      | 8          | Suppress alerts during window   Acknowledge and close                       | 10   Staff unavailability   Low (2)   Medium (3)   6   On-call rotation, backup staff   Postpone if insufficient staff     |

### Risk Scoring: Probability (1-5) × Impact (1-5)

- **1-5:** Low Risk (Accept)
- **6-10:** Medium Risk (Mitigate)
- **11-15:** High Risk (Reduce before proceeding)
- **16-25:** Critical Risk (Escalate, consider postponing)

## 14.3 Impact Analysis

### Service Impact Assessment

| Service                | Servers | Impact During Patching                 | User Impact                             | Mitigation                                                                                      |
|------------------------|---------|----------------------------------------|-----------------------------------------|-------------------------------------------------------------------------------------------------|
| Web Portal             | 20      | Rolling restart (< 30s per server)     | None (behind LB)   Drain before restart | API Gateway   15   Rolling restart (< 30s per server)   None (behind LB)   Drain before restart |
| Authentication Service | 4       | Brief failover (< 5s)   Possible retry | HA cluster, patch standby first         |                                                                                                 |

Batch Processing | 10 | Stopped during window | Delayed reports | Schedule after batch completion | | Email Service | 4 | Queue during restart | Delayed delivery (< 5 min) | Queue holds messages | | Database (Primary) | 2 | Failover (< 30s) | Brief connection retry | Patch standby, failover, patch primary | | Monitoring | 5 | Brief gap in monitoring | None (suppressed) | Suppress alerts, validate after |

## 14.4 Communication Templates

### Pre-Maintenance Notification (5 days before)

Subject: [ADVANCE NOTICE] Linux Security Patching - Sat Jun 27, 22:00-06:00 UTC  
Dear Stakeholders,  
This is an advance notification of planned maintenance:  
ACTIVITY: Monthly Linux Security Patching  
DATE: Saturday, June 27, 2026  
TIME: 22:00 - 06:00 UTC (8-hour window)  
SYSTEMS: 200 Production Linux Servers  
PATCHES INCLUDED:  
- RHSA-2026-1234: kernel security update (CVSS 8.5)  
- RHSA-2026-1235: openssl security update (CVSS 7.8)  
- RHSA-2026-1236: httpd security update (CVSS 6.5)  
- Plus 12 additional security and bug fix updates  
EXPECTED IMPACT:  
- Services behind load balancers: NO user impact  
- Standalone services: Brief restart (< 2 minutes)  
- Batch processing: Suspended 22:00-06:00 UTC  
ACTION REQUIRED BY YOU:  
1. Confirm no critical batch jobs run Saturday 22:00-06:00 UTC  
2. Acknowledge this notification by replying to this email  
3. Provide contact number for escalation if needed  
Change Ticket: CHG-2026-12345  
Bridge Call: Dial-in details will be sent day-of  
Questions? Contact: linux-team@corp.com  
Regards,  
Linux Infrastructure Team

### Maintenance Start Notification

Subject: [IN PROGRESS] Linux Patching Started - CHG-2026-12345  
Maintenance has STARTED.  
Start Time: 22:00 UTC  
Expected End: 06:00 UTC  
Status: Batch 1 (50 servers) in progress  
Bridge Call: <dial-in number> PIN: <pin>  
Updates will be sent every 2 hours or upon significant events.  
- Linux Team

### Maintenance Completion Notification

Subject: [COMPLETED] Linux Patching Complete - CHG-2026-12345  
Maintenance has been COMPLETED SUCCESSFULLY.  
SUMMARY:

```

- Servers Patched: 200/200 (100%)
- Rebooted: 195/200 (5 did not require reboot)
- Rollbacks: 0
- Issues Encountered: 0
- End Time: 05:45 UTC (15 minutes early)
All services verified operational. Monitoring confirmed normal.
Business sign-off obtained from: <Name>
Change ticket CHG-2026-12345 will be closed.
Thank you for your cooperation.
- Linux Infrastructure Team

```

## 14.5 Go/No-Go Checklist

```

.....
. GO / NO-GO DECISION CHECKLIST .
. Time: _____ Date: _____ .
.....
.
. PRE-REQUISITES: .
. . 1. CAB Approval obtained and documented .
. . 2. Business approval obtained (if required) .
. . 3. DEV testing passed (date: _____) .
. . 4. UAT testing passed (date: _____) .
. . 5. Backups verified for all target servers .
. . 6. VM snapshots taken (where applicable) .
. . 7. Rollback plan documented and tested .
. . 8. Communication sent to all stakeholders .
. . 9. Bridge call established and attendees confirmed .
. . 10. No active P1/P2 incidents affecting target systems .
.
. READINESS: .
. . 11. All team members present and available .
. . 12. Console/IPMI access verified for critical servers .
. . 13. Monitoring alerts suppression configured .
. . 14. Ansible/automation tools tested and ready .
. . 15. Satellite content synced and verified .
. . 16. No conflicting changes in current window .
. . 17. Escalation contacts confirmed available .
.
. ENVIRONMENTAL: .
. . 18. No severe weather/power concerns .
. . 19. No pending security incidents .
. . 20. No critical business events (month-end, etc.) .
.
.
. DECISION: .
.
. . GO - All criteria met, proceed with maintenance .
. . NO-GO - Criteria not met, postpone .
.
. Reason (if No-Go): _____ .
.
. Decision Made By: _____ Time: _____ .

```

· Witnessed By: \_\_\_\_\_ ·  
·  
.....

## 14.6 Implementation Plan

### Phase Execution Timeline

| Time (UTC) | Activity | Duration | Responsible | Status | |-----|-----|-----|-----|-----| | 22:00 | Bridge call start, Go/No-Go decision | 15 min | Team Lead | · | 22:15 | Suppress monitoring alerts | 5 min | NOC | · | 22:20 | Start Batch 1 (50 canary servers) | 45 min | Engineer 1 | · | 23:05 | Batch 1 validation | 25 min | Engineer 2 | · | 23:30 | Batch 1 Go/No-Go for Batch 2 | 5 min | Team Lead | · | 23:35 | Start Batch 2 (100 servers) | 60 min | Engineer 1 | · | 00:35 | Batch 2 validation | 25 min | Engineer 2 | · | 01:00 | Batch 2 Go/No-Go for Batch 3 | 5 min | Team Lead | · | 01:05 | Start Batch 3 (250 servers) | 90 min | Engineer 1 | · | 02:35 | Batch 3 validation | 25 min | Engineer 2 | · | 03:00 | Batch 3 Go/No-Go for Batch 4 | 5 min | Team Lead | · | 03:05 | Start Batch 4 (1600 servers, parallel) | 150 min | Engineer 1 + 2 | · | 05:35 | Final validation all servers | 20 min | Both Engineers | · | 05:55 | Re-enable monitoring | 5 min | NOC | · | 06:00 | Bridge call end, send completion notice | 10 min | Team Lead | · |

## 14.7 Backout Plan

### Backout Triggers

- 1 More than 5% of servers in any batch fail validation
- 2 Any critical application unavailable for more than 15 minutes
- 3 Any data integrity issue detected
- 4 Cluster split-brain or multiple failovers
- 5 Team Lead or management decision

### Backout Procedure

#### BACKOUT STEPS:

1. STOP all in-progress patching activities
2. ASSESS which servers are affected
3. For servers still booting: Wait for completion
4. For servers with failed services: Attempt restart
5. If restart fails: Execute rollback
  - a. VM servers: Revert to snapshot
  - b. Bare metal: yum history undo
  - c. Kernel issues: Boot previous kernel
6. VALIDATE all rolled-back servers
7. RE-ENABLE monitoring
8. NOTIFY stakeholders of backout
9. DOCUMENT issues and root cause
10. SCHEDULE new window after fix identified

## 14.8 Closure Report Template

.....  
· CHANGE CLOSURE REPORT ·  
.....

|       |                                               |                                     |               |           |   |
|-------|-----------------------------------------------|-------------------------------------|---------------|-----------|---|
| .     |                                               |                                     |               |           |   |
| ·     | Change ID:                                    | CHG-2026-12345                      | ·             |           |   |
| ·     | Title:                                        | Monthly Linux Security Patching     | ·             |           |   |
| ·     | Implementation Date:                          | June 27-28, 2026                    | ·             |           |   |
| ·     | Status:                                       | · Successful · Partially Successful | ·             |           |   |
| ·     |                                               | · Failed · Backed Out               | ·             |           |   |
| .     |                                               |                                     |               |           |   |
| ·     | RESULTS SUMMARY:                              |                                     | ·             |           |   |
| ·     | Total Servers Targeted:                       | 200                                 | ·             |           |   |
| ·     | Successfully Patched:                         | 200                                 | ·             |           |   |
| ·     | Failed/Rolled Back:                           | 0                                   | ·             |           |   |
| ·     | Deferred to Next Window:                      | 0                                   | ·             |           |   |
| .     |                                               |                                     |               |           |   |
| ·     | TIMELINE:                                     |                                     | ·             |           |   |
| ·     | Planned Start:                                | 22:00 UTC                           | Actual Start: | 22:00 UTC | · |
| ·     | Planned End:                                  | 06:00 UTC                           | Actual End:   | 05:45 UTC | · |
| ·     | Variance:                                     | 15 minutes early                    |               |           | · |
| .     |                                               |                                     |               |           |   |
| ·     | ISSUES ENCOUNTERED:                           |                                     | ·             |           |   |
| ·     | None / [Description of issues and resolution] |                                     | ·             |           |   |
| .     |                                               |                                     |               |           |   |
| ·     | LESSONS LEARNED:                              |                                     | ·             |           |   |
| ·     | 1. [What went well]                           |                                     | ·             |           |   |
| ·     | 2. [What could be improved]                   |                                     | ·             |           |   |
| ·     | 3. [Action items for next cycle]              |                                     | ·             |           |   |
| .     |                                               |                                     |               |           |   |
| ·     | SIGN-OFF:                                     |                                     | ·             |           |   |
| ·     | Implementation Lead:                          | _____                               | Date:         | _____     | · |
| ·     | Application Owner:                            | _____                               | Date:         | _____     | · |
| ·     | Change Manager:                               | _____                               | Date:         | _____     | · |
| .     |                                               |                                     |               |           |   |
| ..... |                                               |                                     |               |           |   |

·**EXPERT TIP:** Maintain a "Lessons Learned" database across patch cycles. Over time, this becomes invaluable for training new team members and identifying recurring issues that need systemic fixes.

## Section 15 · Appendix

### 15.1 YUM/DNF Cheat Sheet

|                                     |                                      |                          |                         |                                      |                          |                       |
|-------------------------------------|--------------------------------------|--------------------------|-------------------------|--------------------------------------|--------------------------|-----------------------|
| Command                             | Description                          | RHEL Version             | ----- ----- -----       | yum check-update                     | List available updates   | 7, 8                  |
| dnf check-update                    | List available updates               | 8, 9, 10                 | yum update -y           | Install all updates                  | 7, 8                     | dnf update -y         |
| Install all updates                 | 8, 9, 10                             | yum update --security -y | Security updates only   | 7, 8                                 | dnf update --security -y | Security updates only |
| 8, 9, 10                            | yum update --advisory=RHSA-2026:1234 | Specific advisory        | 7, 8                    | dnf update --advisory=RHSA-2026:1234 | Specific advisory        | 8, 9, 10              |
| yum update <package>                | Update specific package              | 7, 8                     | yum downgrade <package> | Downgrade package                    | 7, 8                     | yum history list      |
| List transactions                   | 7, 8                                 | yum history info <id>    | Transaction details     | 7, 8                                 | yum history undo <id>    | Undo transaction      |
| 7, 8                                | yum history rollback <id>            | Rollback to state        | 7                       | yum updateinfo list security         | List security errata     | 7, 8                  |
| dnf updateinfo list --type=security | List security errata                 |                          |                         |                                      |                          |                       |

8,9,10||yum versionlock add <pkg>|Lock package version|7,8||dnf versionlock add <pkg>|Lock package version|8,9,10||yum clean all|Clear all caches|7,8||dnf clean all|Clear all caches|8,9,10||yum repolist enabled|Show enabled repos|7,8||dnf repolist enabled|Show enabled repos|8,9,10||rpm -qa --last \|| head -20|Recently installed packages|All||rpm -q <package>|Check installed version|All||rpm -Va|Verify all packages|All||rpm --rebuilddb|Rebuild RPM database|All||needs-restarting -r|Check if reboot needed|7,8,9||needs-restarting -s|Services needing restart|7,8,9||package-cleanup --oldkernels --count=2|Remove old kernels|7||dnf remove --oldinstallonly|Remove old kernels|8,9,10|

## 15.2 Ansible Cheat Sheet

```
... INVENTORY ...
ansible-inventory --list # Show full
inventory
ansible-inventory --graph # Show inventory
tree
ansible all --list-hosts # List all hosts
... AD-HOC COMMANDS ...
ansible all -m ping # Test
connectivity
ansible all -m shell -a "uname -r" # Run command
ansible all -m yum -a "name=* state=latest security=yes" -b #
Patch all
ansible all -m service -a "name=httpd state=restarted" -b #
Restart service
ansible all -m setup -a "filter=ansible_kernel" # Get facts
... PLAYBOOK EXECUTION ...
ansible-playbook playbook.yml # Run playbook
ansible-playbook playbook.yml --check # Dry run
ansible-playbook playbook.yml --diff # Show changes
ansible-playbook playbook.yml --limit host1 # Limit to host
ansible-playbook playbook.yml -e "var=value" # Extra variables
ansible-playbook playbook.yml --tags "patch" # Run tagged tasks
only
ansible-playbook playbook.yml --step # Step through
tasks
ansible-playbook playbook.yml -f 50 # 50 parallel
forks
... VAULT ...
ansible-vault create secrets.yml # Create encrypted
file
ansible-vault edit secrets.yml # Edit encrypted
file
ansible-vault encrypt secrets.yml # Encrypt existing
file
ansible-playbook playbook.yml --ask-vault-pass # Run with vault
... TROUBLESHOOTING ...
ansible all -m ping -vvv # Verbose output
ansible-playbook playbook.yml -vvvv # Maximum
verbosity
ANSIBLE_DEBUG=1 ansible-playbook playbook.yml # Debug mode
ansible-config dump --changed # Show non-default
config
```

## 15.3 Chef Cheat Sheet

```
... COOKBOOK MANAGEMENT ...
knife cookbook list # List cookbooks
on server
knife cookbook upload linux_patching # Upload cookbook
knife cookbook show linux_patching # Show cookbook
details
knife cookbook download linux_patching # Download
cookbook
... NODE MANAGEMENT ...
knife node list # List all nodes
knife node show <node> # Show node
details
knife node edit <node> # Edit node
attributes
knife node run_list add <node> "recipe[linux_patching]" # Add to
run list
knife node run_list remove <node> "recipe[linux_patching]" #
Remove
... SEARCH ...
knife search node "platform:redhat" # Find RHEL nodes
knife search node "chef_environment:production" # Production nodes
knife search node "role:webserver AND platform_version:8*" # RHEL8
webservers
... REMOTE EXECUTION ...
knife ssh "role:webserver" "sudo chef-client" # Run chef on
webservers
knife ssh "name:host1" "sudo chef-client -o
'recipe[linux_patching]'" # Override run list
knife ssh "chef_environment:production" "uname -r" --concurrency 10
... ENVIRONMENT ...
knife environment list # List
environments
knife environment show production # Show environment
knife environment edit production # Edit environment
... CLIENT ...
chef-client # Normal run
chef-client --once # Run once (no
daemon)
chef-client --why-run # Dry run
chef-client -o "recipe[linux_patching]" # Override run
list
chef-client -l debug # Debug output
```

## 15.4 Red Hat Satellite (Hammer CLI) Cheat Sheet

```
... ORGANIZATION ...
hammer organization list
hammer organization info --name "My Org"
... CONTENT MANAGEMENT ...
```

```

hammer product list --organization "My Org"
hammer repository list --organization "My Org"
hammer repository synchronize --organization "My Org" --product
"RHEL 8" --name "BaseOS"
hammer content-view list --organization "My Org"
hammer content-view publish --name "RHEL8-CV" --organization "My
Org"
hammer content-view version promote --content-view "RHEL8-CV"
--version 15 --to-lifecycle-environment "Production" --organization
"My Org"
... HOST MANAGEMENT ...
hammer host list --organization "My Org"
hammer host info --name "webserver01.corp.com"
hammer host errata list --host "webserver01.corp.com"
hammer host errata apply --host "webserver01.corp.com" --errata-ids
"RHSA-2026:1234"
hammer host-collection list --organization "My Org"
... ERRATA ...
hammer erratum list --organization "My Org" --search "type =
security"
hammer erratum info --id "RHSA-2026:1234" --organization "My Org"
... ACTIVATION KEYS ...
hammer activation-key list --organization "My Org"
hammer activation-key info --name "ak-rhel8-prod" --organization
"My Org"
... LIFECYCLE ...
hammer lifecycle-environment list --organization "My Org"
... REPORTS ...
hammer report-template list
hammer report-template generate --name "Host - Errata"
--organization "My Org"

```

## 15.5 Systemctl Cheat Sheet

```

... SERVICE MANAGEMENT ...
systemctl start <service> # Start service
systemctl stop <service> # Stop service
systemctl restart <service> # Restart service
systemctl reload <service> # Reload config
(no downtime)
systemctl status <service> # Check status
systemctl enable <service> # Enable at boot
systemctl disable <service> # Disable at boot
systemctl is-active <service> # Check if running
systemctl is-enabled <service> # Check if enabled
... LISTING ...
systemctl list-units --type=service # All loaded
services
systemctl list-units --state=running # Running services
systemctl list-units --state=failed # Failed services
systemctl list-unit-files --state=enabled # Enabled services
systemctl list-dependencies <service> # Dependencies
... TROUBLESHOOTING ...

```

|                                              |                   |
|----------------------------------------------|-------------------|
| systemctl cat <service>                      | # Show unit file  |
| systemctl show <service>                     | # Show all        |
| properties                                   |                   |
| journalctl -u <service>                      | # Service logs    |
| journalctl -u <service> --since "1 hour ago" | # Recent logs     |
| journalctl -u <service> -f                   | # Follow logs     |
| systemctl daemon-reload                      | # Reload unit     |
| files                                        |                   |
| systemctl reset-failed                       | # Clear failed    |
| state                                        |                   |
| # ... TARGETS ...                            |                   |
| systemctl get-default                        | # Current default |
| target                                       |                   |
| systemctl set-default multi-user.target      | # Set text mode   |
| boot                                         |                   |
| systemctl isolate rescue.target              | # Enter rescue    |
| mode                                         |                   |

## 15.6 Journalctl Cheat Sheet

|                                                          |                    |
|----------------------------------------------------------|--------------------|
| # ... VIEWING LOGS ...                                   |                    |
| journalctl                                               | # All logs         |
| journalctl -b                                            | # Current boot     |
| only                                                     |                    |
| journalctl -b -1                                         | # Previous boot    |
| journalctl --since "2026-06-27 22:00"                    | # Since specific   |
| time                                                     |                    |
| journalctl --since "1 hour ago"                          | # Last hour        |
| journalctl --since "1 hour ago" --until "30 minutes ago" | # Time range       |
| # ... FILTERING ...                                      |                    |
| journalctl -u sshd                                       | # Specific unit    |
| journalctl -p err                                        | # Error priority   |
| and above                                                |                    |
| journalctl -p crit                                       | # Critical only    |
| journalctl _PID=1234                                     | # Specific PID     |
| journalctl _UID=1000                                     | # Specific user    |
| journalctl -k                                            | # Kernel messages  |
| only                                                     |                    |
| # ... OUTPUT ...                                         |                    |
| journalctl -o verbose                                    | # Verbose format   |
| journalctl -o json-pretty                                | # JSON format      |
| journalctl --no-pager                                    | # Don't page       |
| output                                                   |                    |
| journalctl -f                                            | # Follow (like     |
| tail -f)                                                 |                    |
| journalctl -n 50                                         | # Last 50 lines    |
| # ... MAINTENANCE ...                                    |                    |
| journalctl --disk-usage                                  | # Check journal    |
| size                                                     |                    |
| journalctl --vacuum-size=500M                            | # Limit to 500MB   |
| journalctl --vacuum-time=7d                              | # Keep only 7 days |
| journalctl --verify                                      | # Verify journal   |

integrity

## 15.7 LVM Cheat Sheet

|                                              |                    |
|----------------------------------------------|--------------------|
| # ... PHYSICAL VOLUMES ...                   |                    |
| pvs                                          | # List PVs         |
| pvdisplay                                    | # Detailed PV info |
| pvscan                                       | # Scan for PVs     |
| pvcreate /dev/sdb1                           | # Create PV        |
| # ... VOLUME GROUPS ...                      |                    |
| vgs                                          | # List VGs         |
| vgdisplay                                    | # Detailed VG info |
| vgscan                                       | # Scan for VGs     |
| vgextend vg_name /dev/sdb1                   | # Add PV to VG     |
| vgchange -ay                                 | # Activate all VGs |
| # ... LOGICAL VOLUMES ...                    |                    |
| lvs                                          | # List LVs         |
| lvdisplay                                    | # Detailed LV info |
| lvcreate -L 10G -n lv_name vg_name           | # Create LV        |
| lvextend -L +5G /dev/vg_name/lv_name         | # Extend LV        |
| lvreduce -L -5G /dev/vg_name/lv_name         | # Reduce LV        |
| (DANGEROUS)                                  |                    |
| lvremove /dev/vg_name/lv_name                | # Remove LV        |
| # ... SNAPSHOTS (IMPORTANT FOR PATCHING) ... |                    |
| lvcreate -L 5G -s -n snap_root /dev/vg/root  | # Create snapshot  |
| lvs   grep snap                              | # List snapshots   |
| lvremove /dev/vg/snap_root                   | # Remove snapshot  |
| # Revert to snapshot (merge):                |                    |
| lvconvert --merge /dev/vg/snap_root          | # Merge (requires  |
| reboot)                                      |                    |
| # ... FILESYSTEM EXTENSION ...               |                    |
| lvextend -L +5G /dev/vg/root                 | # Extend LV        |
| xfs_growfs /                                 | # Extend XFS       |
| filesystem                                   |                    |
| resize2fs /dev/vg/root                       | # Extend ext4      |
| filesystem                                   |                    |

## 15.8 Network Commands Cheat Sheet

|                                         |                   |
|-----------------------------------------|-------------------|
| # ... IP CONFIGURATION ...              |                   |
| ip addr show                            | # Show all IPs    |
| ip addr add 10.0.0.50/24 dev eth0       | # Add IP          |
| ip addr del 10.0.0.50/24 dev eth0       | # Remove IP       |
| ip link show                            | # Show interfaces |
| ip link set eth0 up                     | # Bring interface |
| up                                      |                   |
| ip link set eth0 down                   | # Bring interface |
| down                                    |                   |
| # ... ROUTING ...                       |                   |
| ip route show                           | # Show routing    |
| table                                   |                   |
| ip route add 172.16.0.0/16 via 10.0.0.1 | # Add route       |

|                                                   |                    |
|---------------------------------------------------|--------------------|
| ip route del 172.16.0.0/16                        | # Delete route     |
| ip route get 8.8.8.8                              | # Check route to   |
| host                                              |                    |
| # ... DNS ...                                     |                    |
| nslookup <hostname>                               | # DNS lookup       |
| dig <hostname>                                    | # Detailed DNS     |
| lookup                                            |                    |
| dig <hostname> +short                             | # Short DNS lookup |
| host <hostname>                                   | # Reverse lookup   |
| cat /etc/resolv.conf                              | # DNS              |
| configuration                                     |                    |
| # ... CONNECTIVITY ...                            |                    |
| ping -c 3 <host>                                  | # Basic            |
| connectivity                                      |                    |
| traceroute <host>                                 | # Trace route      |
| mtr <host>                                        | # Combined         |
| ping/traceroute                                   |                    |
| curl -v https://<host>                            | # HTTP(S) test     |
| telnet <host> <port>                              | # Port             |
| connectivity                                      |                    |
| nc -zv <host> <port>                              | # Port scan        |
| # ... NETWORK MANAGER ...                         |                    |
| nmcli device status                               | # Show device      |
| status                                            |                    |
| nmcli connection show                             | # Show connections |
| nmcli connection up <name>                        | # Activate         |
| connection                                        |                    |
| nmcli connection modify <name> ipv4.dns "8.8.8.8" | # Modify DNS       |
| nmcli general status                              | # NetworkManager   |
| status                                            |                    |
| # ... SOCKETS ...                                 |                    |
| ss -tuln                                          | # Listening        |
| TCP/UDP ports                                     |                    |
| ss -tn                                            | # Established      |
| connections                                       |                    |
| ss -s                                             | # Socket           |
| statistics                                        |                    |
| lsof -i :<port>                                   | # Process using    |
| port                                              |                    |

## 15.9 GRUB & Kernel Management Cheat Sheet

|                                              |                   |
|----------------------------------------------|-------------------|
| # ... GRUB ...                               |                   |
| grubby --default-kernel                      | # Show default    |
| boot kernel                                  |                   |
| grubby --info=ALL                            | # Show all kernel |
| entries                                      |                   |
| grubby --set-default /boot/vmlinuz-<version> | # Set default     |
| kernel                                       |                   |
| grub2-editenv list                           | # Show GRUB       |
| environment                                  |                   |
| grub2-mkconfig -o /boot/grub2/grub.cfg       | # Regenerate GRUB |
| config                                       |                   |

|                         |                    |
|-------------------------|--------------------|
| grub2-install /dev/sda  | # Reinstall GRUB   |
| (BIOS)                  |                    |
| # ... KERNEL ...        |                    |
| uname -r                | # Current kernel   |
| uname -a                | # Full kernel info |
| rpm -qa kernel          | # Installed        |
| kernels                 |                    |
| cat /proc/cmdline       | # Boot parameters  |
| sysctl -a               | # All kernel       |
| parameters              |                    |
| sysctl -p               | # Reload           |
| sysctl.conf             |                    |
| modprobe <module>       | # Load kernel      |
| module                  |                    |
| lsmod                   | # List loaded      |
| modules                 |                    |
| modinfo <module>        | # Module           |
| information             |                    |
| dracut -f               | # Rebuild          |
| initramfs               |                    |
| # ... LIVE PATCHING ... |                    |
| yum install kpatch      | # Install kpatch   |
| kpatch list             | # List active      |
| patches                 |                    |
| kpatch load <module.ko> | # Load patch       |
| kpatch unload <module>  | # Unload patch     |

## 15.10 Quick Reference: Critical Pre/Post Patch Commands

```
.....
QUICK PRE-PATCH (copy-paste ready)
.....
echo "=== PRE-PATCH: $(hostname) $(date) ==="
uname -r
cat /etc/redhat-release
uptime
free -h
df -h / /boot /var
df -i / | tail -1
systemctl list-units --state=failed --no-pager
ip route show default
subscription-manager status 2>/dev/null | grep Status
yum check-update 2>/dev/null | wc -l
echo "=== END PRE-PATCH ==="
.....
QUICK POST-PATCH (copy-paste ready)
.....
echo "=== POST-PATCH: $(hostname) $(date) ==="
uname -r
cat /etc/redhat-release
uptime
free -h
df -h / /boot /var
```

```
systemctl list-units --state=failed --no-pager
ip route show default
ping -c 2 $(ip route show default | awk '{print $3}') > /dev/null
&& echo "Gateway: OK" || echo "Gateway: FAIL"
needs-restarting -r 2>/dev/null; echo "Reboot needed: $([$? -eq 1
] && echo YES || echo NO)"
echo "=== END POST-PATCH ==="
```

## 15.11 RHEL Version-Specific Notes

### RHEL 6 (Legacy - End of Life)

```
Package manager: yum (older version)
Init system: SysVinit (not systemd)
Service management: service / chkconfig
service httpd status
chkconfig httpd on
Network config: /etc/sysconfig/network-scripts/ifcfg-*
Kernel: 2.6.32.x
NOTE: RHEL 6 ELS ended June 2024. Migrate immediately!
```

### RHEL 7 (Maintenance Support until June 2024, ELS until 2028)

```
Package manager: yum
Init system: systemd
Default filesystem: XFS
Firewall: firewalld
Network: NetworkManager + ifcfg scripts
Python: 2.7 (default), 3.6 (optional)
Kernel: 3.10.x
```

### RHEL 8 (Full Support until May 2024, Maintenance until 2029)

```
Package manager: dnf (yum is symlink to dnf)
Module streams: dnf module
Default filesystem: XFS
Firewall: firewalld + nftables backend
Network: NetworkManager (nmcli)
Python: 3.6/3.8/3.9 (no default python command)
Kernel: 4.18.x
Container: podman (no docker by default)
```

### RHEL 9 (Current)

```
Package manager: dnf
Default filesystem: XFS
Firewall: firewalld + nftables
Network: NetworkManager only (no legacy scripts)
Python: 3.9+
Kernel: 5.14.x
Crypto: OpenSSL 3.0
```

```
SELinux: Enforcing by default
Notable: No ifcfg scripts, no iptables
```

## RHEL 10 (Latest - 2025+)

```
Package manager: dnf5
Default filesystem: XFS
Kernel: 6.x
Python: 3.12+
Notable changes:
- dnf5 replaces dnf4 (faster, C++ based)
- New image-based deployment options
- Enhanced container integration
- Stronger crypto defaults (TLS 1.3 minimum)
```

## 15.12 Emergency Contact Template

| Role                | Name                     | Phone | Email | Availability    | ----- ----- ----- ----- ----- | Linux Team Lead | _____          |
|---------------------|--------------------------|-------|-------|-----------------|-------------------------------|-----------------|----------------|
|                     | _____                    | _____ | _____ | Primary on-call | Linux Engineer (Primary)      | _____           | _____          |
| During window       | Linux Engineer (Backup)  | _____ | _____ | _____           | On standby                    | DBA On-Call     | _____          |
| _____               | _____                    | _____ | _____ | 15 min response | Network On-Call               | _____           | _____          |
| 15 min response     | Application Owner        | _____ | _____ | _____           | For sign-off                  | IT Director     | _____          |
| _____               | _____                    | _____ | _____ | Escalation only | CISO                          | _____           | _____          |
| Security escalation | Vendor Support (Red Hat) | _____ | _____ | 1-888-REDHAT1   | _____                         | Severity 1      | VMware Support |
| _____               | _____                    | _____ | _____ | If VM issues    | _____                         | _____           | _____          |

## 15.13 Glossary

| Term | Definition | ----- ----- | CAB | Change Advisory Board - approves production changes | CVE | Common Vulnerabilities and Exposures - unique vulnerability ID | CVSS | Common Vulnerability Scoring System - severity rating (0-10) | RHSA | Red Hat Security Advisory - security patch notification | RHBA | Red Hat Bug Advisory - bug fix notification | RHEA | Red Hat Enhancement Advisory - feature enhancement | Errata | Collective term for advisories (patches) from Red Hat | Content View | Satellite concept - curated set of repos for a purpose | Lifecycle Environment | Satellite concept - stage in patch promotion path | Activation Key | Satellite concept - registration configuration template | Host Collection | Satellite concept - group of hosts for batch operations | RPM | Red Hat Package Manager - package format for RHEL | YUM | Yellowdog Updater Modified - package manager (RHEL 6/7) | DNF | Dandified YUM - next-gen package manager (RHEL 8+) | kpatch | Kernel live patching technology (no reboot needed) | initramfs | Initial RAM filesystem loaded during boot | GRUB | Grand Unified Bootloader - boot manager | LVM | Logical Volume Manager - flexible disk management | SELinux | Security-Enhanced Linux - mandatory access control | SLA | Service Level Agreement | RPO | Recovery Point Objective - max acceptable data loss | RTO | Recovery Time Objective - max acceptable downtime |
|------|------------|-------------|-----|-----------------------------------------------------|-----|----------------------------------------------------------------|------|--------------------------------------------------------------|------|---------------------------------------------------------|------|---------------------------------------------|------|----------------------------------------------------|--------|-------------------------------------------------------|--------------|--------------------------------------------------------|-----------------------|---------------------------------------------------|----------------|---------------------------------------------------------|-----------------|---------------------------------------------------------|-----|---------------------------------------------------|-----|---------------------------------------------------------|-----|----------------------------------------------------|--------|----------------------------------------------------|-----------|-------------------------------------------|------|-----------------------------------------|-----|---------------------------------------------------|---------|----------------------------------------------------|-----|-------------------------|-----|-----------------------------------------------------|-----|---------------------------------------------------|
|------|------------|-------------|-----|-----------------------------------------------------|-----|----------------------------------------------------------------|------|--------------------------------------------------------------|------|---------------------------------------------------------|------|---------------------------------------------|------|----------------------------------------------------|--------|-------------------------------------------------------|--------------|--------------------------------------------------------|-----------------------|---------------------------------------------------|----------------|---------------------------------------------------------|-----------------|---------------------------------------------------------|-----|---------------------------------------------------|-----|---------------------------------------------------------|-----|----------------------------------------------------|--------|----------------------------------------------------|-----------|-------------------------------------------|------|-----------------------------------------|-----|---------------------------------------------------|---------|----------------------------------------------------|-----|-------------------------|-----|-----------------------------------------------------|-----|---------------------------------------------------|

## Document End

```
.....
.
. ENTERPRISE LINUX PATCHING RUNBOOK - VERSION 3.0 .
.
. © 2026 Linux Infrastructure Team .
. Classification: INTERNAL - CONFIDENTIAL .
```

|       |                                  |   |
|-------|----------------------------------|---|
| .     |                                  | . |
| .     | Next Review Date: September 2026 | . |
| .     | Document Owner: Linux Team Lead  | . |
| .     |                                  | . |
| ..... |                                  |   |

---

*End of Document*